MTE Managed-Key Encryption Add-On Developer's Guide

MTE Version 2.1

Table of Contents

1 Introduction
- 1.1 Encoder/Decoder Mode
- 1.2 Chunk Mode
  - 1.2.1 Chunk Encrypt Process
  - 1.2.2 Chunk Decrypt Process

2 Terms and Abbreviations

3 Prerequisites

4 Caveats

5 Architecture

6 SDK Structure

7 Options
- 7.1 Choosing a Cipher
- 7.2 Choosing a Hash

8 Status

9 Basic C API
- 9.1 Overview
  - 9.1.1 Encoder
  - 9.1.2 Decoder
  - 9.1.3 Chunk Encrypt
  - 9.1.4 Chunk Decrypt
- 9.2 Files
- 9.3 Functions
  - 9.3.1 Decoder
    - 9.3.1.1 `mte_mke_dec_state_bytes` (C)
    - 9.3.1.2 `mte_mke_dec_state_bytes_d` (C)
    - 9.3.1.3 `mte_mke_dec_state_bytes_c` (C)
    - 9.3.1.4 `mte_mke_dec_state_bytes_h` (C)
    - 9.3.1.5 `mte_mke_dec_state_bytes_dc` (C)
    - 9.3.1.6 `mte_mke_dec_state_bytes_dh` (C)
    - 9.3.1.7 `mte_mke_dec_state_bytes_ch` (C)
    - 9.3.1.8 `mte_mke_dec_state_bytes_dch` (C)
    - 9.3.1.9 `mte_mke_dec_state_init` (C)
    - 9.3.1.10 `mte_mke_dec_state_init_d` (C)
    - 9.3.1.11 `mte_mke_dec_state_init_c` (C)
    - 9.3.1.12 `mte_mke_dec_state_init_h` (C)
    - 9.3.1.13 `mte_mke_dec_state_init_dc` (C)
    - 9.3.1.14 `mte_mke_dec_state_init_dh` (C)
    - 9.3.1.15 `mte_mke_dec_state_init_ch` (C)
    - 9.3.1.16 `mte_mke_dec_state_init_dch` (C)
    - 9.3.1.17 `mte_mke_dec_instantiate` (C)
    - 9.3.1.18 `mte_mke_dec_reseed_counter` (C)
    - 9.3.1.19 `mte_mke_dec_save_bytes` (C)

9.3.1.20 `mte_mke_dec_save_bytes_b64` (C)
9.3.1.21 `mte_mke_dec_state_save` (C)
9.3.1.22 `mte_mke_dec_state_save_b64` (C)
9.3.1.23 `mte_mke_dec_state_restore` (C)
9.3.1.24 `mte_mke_dec_state_restore_b64` (C)
9.3.1.25 `mte_mke_dec_buff_bytes` (C)
9.3.1.26 `mte_mke_dec_buff_bytes_b64` (C)
9.3.1.27 `mte_mke_dec_decode` (C)
9.3.1.28 `mte_mke_dec_decode_b64` (C)
9.3.1.29 `mte_mke_dec_decrypt_state_bytes` (C)
9.3.1.30 `mte_mke_dec_decrypt_start` (C)
9.3.1.31 `mte_mke_dec_decrypt_chunk` (C)
9.3.1.32 `mte_mke_dec_decrypt_finish` (C)
9.3.1.33 `mte_mke_dec_enc_ts` (C)
9.3.1.34 `mte_mke_dec_dec_ts` (C)
9.3.1.35 `mte_mke_dec_msg_skipped` (C)
9.3.1.36 `mte_mke_dec_uninstantiate` (C)
9.3.2 Encoder
9.3.2.1 `mte_mke_enc_state_bytes` (C)
9.3.2.2 `mte_mke_enc_state_bytes_d` (C)
9.3.2.3 `mte_mke_enc_state_bytes_c` (C)
9.3.2.4 `mte_mke_enc_state_bytes_h` (C)
9.3.2.5 `mte_mke_enc_state_bytes_dc` (C)
9.3.2.6 `mte_mke_enc_state_bytes_dh` (C)
9.3.2.7 `mte_mke_enc_state_bytes_ch` (C)
9.3.2.8 `mte_mke_enc_state_bytes_dch` (C)
9.3.2.9 `mte_mke_enc_state_init` (C)
9.3.2.10 `mte_mke_enc_state_init_d` (C)
9.3.2.11 `mte_mke_enc_state_init_c` (C)
9.3.2.12 `mte_mke_enc_state_init_h` (C)
9.3.2.13 `mte_mke_enc_state_init_dc` (C)
9.3.2.14 `mte_mke_enc_state_init_dh` (C)
9.3.2.15 `mte_mke_enc_state_init_ch` (C)
9.3.2.16 `mte_mke_enc_state_init_dch` (C)
9.3.2.17 `mte_mke_enc_instantiate` (C)
9.3.2.18 `mte_mke_enc_reseed_counter` (C)
9.3.2.19 `mte_mke_enc_save_bytes` (C)
9.3.2.20 `mte_mke_enc_save_bytes_b64` (C)
9.3.2.21 `mte_mke_enc_state_save` (C)
9.3.2.22 `mte_mke_enc_state_save_b64` (C)
9.3.2.23 `mte_mke_enc_state_restore` (C)
9.3.2.24 `mte_mke_enc_state_restore_b64` (C)
9.3.2.25 `mte_mke_enc_buff_bytes` (C)
9.3.2.26 `mte_mke_enc_buff_bytes_b64` (C)
9.3.2.27 `mte_mke_enc_encode` (C)
9.3.2.28 `mte_mke_enc_encode_b64` (C)
9.3.2.29 `mte_mke_enc_encrypt_finish_bytes` (C)
9.3.2.30 `mte_mke_enc_encrypt_state_bytes` (C)
9.3.2.31 `mte_mke_enc_encrypt_start` (C)
9.3.2.32 `mte_mke_enc_encrypt_chunk` (C)
9.3.2.33 `mte_mke_enc_encrypt_finish` (C)

9.3.2.34 `mte_mke_enc_uninstantiate` (C)

10 C Support API

10.1 Overview

10.1.1 Encoder

10.1.2 Decoder

10.1.3 Chunk Encrypt

10.1.4 Chunk Decrypt

10.2 Files

10.3 Functions

10.3.1 Encoder/Decoder Support

10.3.1.1 `mte_supp_mke_enc_alloc` (C)

10.3.1.2 `mte_supp_mke_enc_state_save` (C)

10.3.1.3 `mte_supp_mke_enc_state_save_b64` (C)

10.3.1.4 `mte_supp_mke_enc_encode` (C)

10.3.1.5 `mte_supp_mke_enc_encode_b64` (C)

10.3.1.6 `mte_supp_mke_enc_free` (C)

10.3.1.7 `mte_supp_mke_dec_alloc` (C)

10.3.1.8 `mte_supp_mke_dec_state_save` (C)

10.3.1.9 `mte_supp_mke_dec_state_save_b64` (C)

10.3.1.10 `mte_supp_mke_dec_decode` (C)

10.3.1.11 `mte_supp_mke_dec_decode_b64` (C)

10.3.1.12 `mte_supp_mke_dec_free` (C)

11 C++ Language Interface

11.1 Overview

11.1.1 Encoder

11.1.2 Decoder

11.1.3 Chunk Encrypt

11.1.4 Chunk Decrypt

11.2 Files

11.2.1 Classes

11.2.1.1 `MteMkeDec`

11.2.1.2 `MteMkeEnc`

11.3 Functions

11.3.1 Decoder

11.3.1.1 `MteMkeDec` Constructor (default options) (CPP)

11.3.1.2 `MteMkeDec` Constructor (runtime options) (CPP)

11.3.1.3 `MteMkeDec` Destructor (CPP)

11.3.1.4 `MteMkeDec::instantiate` (`void *`) (CPP)

11.3.1.5 `MteMkeDec::instantiate` (`std::string`) (CPP)

11.3.1.6 `MteMkeDec::getReseedCounter` (CPP)

11.3.1.7 `MteMkeDec::saveState` (CPP)

11.3.1.8 `MteMkeDec::saveStateB64` (CPP)

11.3.1.9 `MteMkeDec::restoreState` (CPP)

11.3.1.10 `MteMkeDec::restoreStateB64` (CPP)

11.3.1.11 `MteMkeDec::getBuffBytes` (CPP)

11.3.1.12 `MteMkeDec::getBuffBytesB64` (CPP)

11.3.1.13 `MteMkeDec::decode` (`void *`) (CPP)

11.3.1.14 `MteMkeDec::decodeB64` (`void *`) (CPP)

11.3.1.15 `MteMkeDec::decode` (`std::string`) (CPP)
11.3.1.16 `MteMkeDec::decodeB64` (`std::string`) (CPP)
11.3.1.17 `MteMkeDec::decode` (external buffers) (CPP)
11.3.1.18 `MteMkeDec::decodeB64` (external buffers) (CPP)
11.3.1.19 `MteMkeDec::startDecrypt` (CPP)
11.3.1.20 `MteMkeDec::decryptChunk` (CPP)
11.3.1.21 `MteMkeDec::decryptChunk` (external buffers) (CPP)
11.3.1.22 `MteMkeDec::finishDecrypt` (CPP)
11.3.1.23 `MteMkeDec::getEncTs` (CPP)
11.3.1.24 `MteMkeDec::getDecTs` (CPP)
11.3.1.25 `MteMkeDec::getMsgSkipped` (CPP)
11.3.1.26 `MteMkeDec::uninstantiate` (CPP)
11.3.2 Encoder
11.3.2.1 `MteMkeEnc` Constructor (default options) (CPP)
11.3.2.2 `MteMkeEnc` Constructor (runtime options) (CPP)
11.3.2.3 `MteMkeEnc` Destructor (CPP)
11.3.2.4 `MteMkeEnc::instantiate` (`void *`) (CPP)
11.3.2.5 `MteMkeEnc::instantiate` (`std::string`) (CPP)
11.3.2.6 `MteMkeEnc::getReseedCounter` (CPP)
11.3.2.7 `MteMkeEnc::saveState` (CPP)
11.3.2.8 `MteMkeEnc::saveStateB64` (CPP)
11.3.2.9 `MteMkeEnc::restoreState` (CPP)
11.3.2.10 `MteMkeEnc::restoreStateB64` (CPP)
11.3.2.11 `MteMkeEnc::getBuffBytes` (CPP)
11.3.2.12 `MteMkeEnc::getBuffBytesB64` (CPP)
11.3.2.13 `MteMkeEnc::encode` (`void *`) (CPP)
11.3.2.14 `MteMkeEnc::encodeB64` (`void *`) (CPP)
11.3.2.15 `MteMkeEnc::encode` (`std::string`) (CPP)
11.3.2.16 `MteMkeEnc::encodeB64` (`std::string`) (CPP)
11.3.2.17 `MteMkeEnc::encode` (external buffers) (CPP)
11.3.2.18 `MteMkeEnc::encodeB64` (external buffers) (CPP)
11.3.2.19 `MteMkeEnc::encryptFinishBytes` (CPP)
11.3.2.20 `MteMkeEnc::startEncrypt` (CPP)
11.3.2.21 `MteMkeEnc::encryptChunk` (CPP)
11.3.2.22 `MteMkeEnc::finishEncrypt` (CPP)
11.3.2.23 `MteMkeEnc::uninstantiate` (CPP)
12 C# Language Interface
12.1 Overview
12.1.1 Encoder
12.1.2 Decoder
12.1.3 Chunk Encrypt
12.1.4 Chunk Decrypt
12.2 Files
12.3 Types
12.3.1 Classes
12.3.1.1 `MteMkeDec`
12.3.1.2 `MteMkeEnc`
12.4 Functions
12.4.1 Decoder

12.4.1.1 `MteMkeDec` Constructor (default options) (CS)
12.4.1.2 `MteMkeDec` Constructor (interop, default options) (CS)
12.4.1.3 `MteMkeDec` Constructor (runtime options) (CS)
12.4.1.4 `MteMkeDec` Constructor (interop, runtime options) (CS)
12.4.1.5 `MteMkeDec` Destructor (CS)
12.4.1.6 `MteMkeDec.Instantiate` (`byte[]`) (CS)
12.4.1.7 `MteMkeDec.Instantiate` (`string`) (CS)
12.4.1.8 `MteMkeDec.GetReseedCounter` (CS)
12.4.1.9 `MteMkeDec.SaveState` (CS)
12.4.1.10 `MteMkeDec.SaveStateB64` (CS)
12.4.1.11 `MteMkeDec.RestoreState` (CS)
12.4.1.12 `MteMkeDec.RestoreStateB64` (CS)
12.4.1.13 `MteMkeDec.GetBuffBytes` (CS)
12.4.1.14 `MteMkeDec.GetBuffBytesB64` (CS)
12.4.1.15 `MteMkeDec.Decode` (CS)
12.4.1.16 `MteMkeDec.DecodeB64` (CS)
12.4.1.17 `MteMkeDec.DecodeStr` (CS)
12.4.1.18 `MteMkeDec.DecodeStrB64` (CS)
12.4.1.19 `MteMkeDec.Decode` (external buffers) (CS)
12.4.1.20 `MteMkeDec.DecodeB64` (external buffers) (CS)
12.4.1.21 `MteMkeDec.StartDecrypt` (CS)
12.4.1.22 `MteMkeDec.DecryptChunk` (CS)
12.4.1.23 `MteMkeDec.DecryptChunk` (external buffers) (CS)
12.4.1.24 `MteMkeDec.FinishDecrypt` (CS)
12.4.1.25 `MteMkeDec.FinishDecrypt` (external buffers) (CS)
12.4.1.26 `MteMkeDec.GetEncTs` (CS)
12.4.1.27 `MteMkeDec.GetDecTs` (CS)
12.4.1.28 `MteMkeDec.GetMsgSkipped` (CS)
12.4.1.29 `MteMkeDec.Uninstantiate` (CS)
12.4.2 Encoder
12.4.2.1 `MteMkeEnc` Constructor (default options) (CS)
12.4.2.2 `MteMkeEnc` Constructor (interop, default options) (CS)
12.4.2.3 `MteMkeEnc` Constructor (runtime options) (CS)
12.4.2.4 `MteMkeEnc` Constructor (interop, runtime options) (CS)
12.4.2.5 `MteMkeEnc` Destructor (CS)
12.4.2.6 `MteMkeEnc.Instantiate` (byte[]) (CS)
12.4.2.7 `MteMkeEnc.Instantiate` (`string`) (CS)
12.4.2.8 `MteMkeEnc.GetReseedCounter` (CS)
12.4.2.9 `MteMkeEnc.SaveState` (CS)
12.4.2.10 `MteMkeEnc.SaveStateB64` (CS)
12.4.2.11 `MteMkeEnc.RestoreState` (CS)
12.4.2.12 `MteMkeEnc.RestoreStateB64` (CS)
12.4.2.13 `MteMkeEnc.GetBuffBytes` (CS)
12.4.2.14 `MteMkeEnc.GetBuffBytesB64` (CS)
12.4.2.15 `MteMkeEnc.Encode` (`byte[]`) (CS)
12.4.2.16 `MteMkeEnc.EncodeB64` (`byte[]`) (CS)
12.4.2.17 `MteMkeEnc.Encode` (`string`) (CS)
12.4.2.18 `MteMkeEnc.EncodeB64` (`string`) (CS)
12.4.2.19 `MteMkeEnc.Encode` (external buffers) (CS)
12.4.2.20 `MteMkeEnc.EncodeB64` (external buffers) (CS)
12.4.2.21 `MteMkeEnc.EncryptFinishBytes` (CS)

12.4.2.22 `MteMkeEnc.StartEncrypt` (CS)
12.4.2.23 `MteMkeEnc.EncryptChunk` (CS)
12.4.2.24 `MteMkeEnc.EncryptChunk` (slice) (CS)
12.4.2.25 `MteMkeEnc.FinishEncrypt` (CS)
12.4.2.26 `MteMkeEnc.Uninstantiate` (CS)

13 Go Language Interface
13.1 Overview
13.1.1 Encoder
13.1.2 Decoder
13.1.3 Chunk Encrypt
13.1.4 Chunk Decrypt
13.2 Files
13.3 Types
13.3.1 Structures
13.3.1.1 `MkeDec`
13.3.1.2 `MkeEnc`
13.4 Functions
13.4.1 Decoder
13.4.1.1 `NewMkeDecDef` (Go)
13.4.1.2 `NewMkeDecWin` (Go)
13.4.1.3 `NewMkeDecOpt` (Go)
13.4.1.4 `(*MkeDec)Destroy` (Go)
13.4.1.5 `(*MkeDec)Instantiate` (Go)
13.4.1.6 `(*MkeDec)InstantiateStr` (Go)
13.4.1.7 `(*MkeDec)GetReseedCounter` (Go)
13.4.1.8 `(*MkeDec)SaveState` (Go)
13.4.1.9 `(*MkeDec)SaveStateB64` (Go)
13.4.1.10 `(*MkeDec)RestoreState` (Go)
13.4.1.11 `(*MkeDec)RestoreStateB64` (Go)
13.4.1.12 `(*MkeDec)Decode` (Go)
13.4.1.13 `(*MkeDec)DecodeB64` (Go)
13.4.1.14 `(*MkeDec)DecodeStr` (Go)
13.4.1.15 `(*MkeDec)DecodeStrB64` (Go)
13.4.1.16 `(*MkeDec)StartDecrypt` (Go)
13.4.1.17 `(*MkeDec)DecryptChunk` (Go)
13.4.1.18 `(*MkeDec)FinishDecrypt` (Go)
13.4.1.19 `(*MkeDec)GetEncTs` (Go)
13.4.1.20 `(*MkeDec)GetMkeDecTs` (Go)
13.4.1.21 `(*MkeDec)GetMsgSkipped` (Go)
13.4.1.22 `(*MkeDec)Uninstantiate` (Go)
13.4.2 Encoder
13.4.2.1 `NewMkeEncDef` (Go)
13.4.2.2 `NewMkeEncOpt` (Go)
13.4.2.3 `(*MkeEnc)Destroy` (Go)
13.4.2.4 `(*MkeEnc)Instantiate` (Go)
13.4.2.5 `(*MkeEnc)InstantiateStr` (Go)
13.4.2.6 `(*MkeEnc)GetReseedCounter` (Go)
13.4.2.7 `(*MkeEnc)SaveState` (Go)
13.4.2.8 `(*MkeEnc)SaveStateB64` (Go)

13.4.2.9 `(*MkeEnc)RestoreState` (Go)
13.4.2.10 `(*MkeEnc)RestoreStateB64` (Go)
13.4.2.11 `(*MkeEnc)MkeEncode` (Go)
13.4.2.12 `(*MkeEnc)MkeEncodeB64` (Go)
13.4.2.13 `(*MkeEnc)MkeEncodeStr` (Go)
13.4.2.14 `(*MkeEnc)MkeEncodeStrB64` (Go)
13.4.2.15 `(*MkeEnc)EncryptFinishBytes` (Go)
13.4.2.16 `(*MkeEnc)StartEncrypt` (Go)
13.4.2.17 `(*MkeEnc)EncryptChunk` (Go)
13.4.2.18 `(*MkeEnc)FinishEncrypt` (Go)
13.4.2.19 `(*MkeEnc)Uninstantiate` (Go)

14 Java Language Interface
14.1 Overview
14.1.1 Encoder
14.1.2 Decoder
14.1.3 Chunk Encrypt
14.1.4 Chunk Decrypt
14.2 Files
14.3 Types
14.3.1 Classes
14.3.1.1 `MteMkeDec`
14.3.1.2 `MteMkeEnc`
14.4 Functions
14.4.1 Decoder
14.4.1.1 `MteMkeDec` Constructor (default options) (Java)
14.4.1.2 `MteMkeDec` Constructor (window options) (Java)
14.4.1.3 `MteMkeDec` Constructor (runtime options) (Java)
14.4.1.4 `MteMkeDec.instantiate` (`byte[]`) (Java)
14.4.1.5 `MteMkeDec.instantiate` (`String`) (Java)
14.4.1.6 `MteMkeDec.getReseedCounter` (Java)
14.4.1.7 `MteMkeDec.saveState` (Java)
14.4.1.8 `MteMkeDec.saveStateB64` (Java)
14.4.1.9 `MteMkeDec.restoreState` (Java)
14.4.1.10 `MteMkeDec.restoreStateB64` (Java)
14.4.1.11 `MteMkeDec.getBuffBytes` (Java)
14.4.1.12 `MteMkeDec.getBuffBytesB64` (Java)
14.4.1.13 `MteMkeDec.decode` (Java)
14.4.1.14 `MteMkeDec.decodeB64` (Java)
14.4.1.15 `MteMkeDec.decodeStr` (Java)
14.4.1.16 `MteMkeDec.decodeStrB64` (Java)
14.4.1.17 `MteMkeDec.decode` (external buffers) (Java)
14.4.1.18 `MteMkeDec.decodeB64` (external buffers) (Java)
14.4.1.19 `MteMkeDec.startDecrypt` (Java)
14.4.1.20 `MteMkeDec.decryptChunk` (Java)
14.4.1.21 `MteMkeDec.decryptChunk` (external buffers) (Java)
14.4.1.22 `MteMkeDec.finishDecrypt` (Java)
14.4.1.23 `MteMkeDec.finishDecrypt` (external buffers) (Java)
14.4.1.24 `MteMkeDec.getEncTs` (Java)
14.4.1.25 `MteMkeDec.getDecTs` (Java)

14.4.1.26 `MteMkeDec.getMsgSkipped` (Java)
14.4.1.27 `MteMkeDec.uninstantiate` (Java)
14.4.2 Encoder
14.4.2.1 `MteMkeEnc` Constructor (default options) (Java)
14.4.2.2 `MteMkeEnc` Constructor (runtime options) (Java)
14.4.2.3 `MteMkeEnc.instantiate` (`byte[]`) (Java)
14.4.2.4 `MteMkeEnc.instantiate` (`String`) (Java)
14.4.2.5 `MteMkeEnc.getReseedCounter` (Java)
14.4.2.6 `MteMkeEnc.saveState` (Java)
14.4.2.7 `MteMkeEnc.saveStateB64` (Java)
14.4.2.8 `MteMkeEnc.restoreState` (Java)
14.4.2.9 `MteMkeEnc.restoreStateB64` (Java)
14.4.2.10 `MteMkeEnc.getBuffBytes` (Java)
14.4.2.11 `MteMkeEnc.getBuffBytesB64` (Java)
14.4.2.12 `MteMkeEnc.encode` (`byte[]`) (Java)
14.4.2.13 `MteMkeEnc.encodeB64` (`byte[]`) (Java)
14.4.2.14 `MteMkeEnc.encode` (`String`) (Java)
14.4.2.15 `MteMkeEnc.encodeB64` (`String`) (Java)
14.4.2.16 `MteMkeEnc.encode` (external buffers) (Java)
14.4.2.17 `MteMkeEnc.encodeB64` (external buffers) (Java)
14.4.2.18 `MteMkeEnc.encryptFinishBytes` (Java)
14.4.2.19 `MteMkeEnc.startEncrypt` (Java)
14.4.2.20 `MteMkeEnc.encryptChunk` (Java)
14.4.2.21 `MteMkeEnc.encryptChunk` (slice) (Java)
14.4.2.22 `MteMkeEnc.finishEncrypt` (Java)
14.4.2.23 `MteMkeEnc.uninstantiate` (Java)
15 Objective-C Language Interface
15.1 Overview
15.1.1 Encoder
15.1.2 Decoder
15.1.3 Chunk Encrypt
15.1.4 Chunk Decrypt
15.2 Files
15.3 Types
15.3.1 Classes
15.3.1.1 `MteMkeDec`
15.3.1.2 `MteMkeEnc`
15.4 Functions
15.4.1 Decoder
15.4.1.1 `MteMkeDec -init` (ObjC)
15.4.1.2 `MteMkeDec -initWithTWindow:sWindow:` (ObjC)
15.4.1.3 `MteMkeDec -initWithDrbg:tokBytes:verifiers:cipher:hash:tWindow:sWindow:` (ObjC)
15.4.1.4 `MteMkeDec -dealloc` (ObjC)
15.4.1.5 `MteMkeDec -instantiate:bytes:` (ObjC)
15.4.1.6 `MteMkeDec -instantiate:` (ObjC)
15.4.1.7 `MteMkeDec -getReseedCounter` (ObjC)
15.4.1.8 `MteMkeDec -saveState:` (ObjC)
15.4.1.9 `MteMkeDec -saveStateB64` (ObjC)
15.4.1.10 `MteMkeDec -restoreState:` (ObjC)

15.4.1.11 `MteMkeDec -restoreStateB64:` (ObjC)
15.4.1.12 `MteMkeDec -getBuffBytes:encOff:encBytes` (ObjC)
15.4.1.13 `MteMkeDec -getBuffBytesB64:encOff:encBytes` (ObjC)
15.4.1.14 `MteMkeDec -decode:encodedBytes:decodedBytes:status:` (ObjC)
15.4.1.15 `MteMkeDec -decodeB64:decodedBytes:status:` (ObjC)
15.4.1.16 `MteMkeDec -decode:encodedBytes:status:` (ObjC)
15.4.1.17 `MteMkeDec -decodeB64:status:` (ObjC)
15.4.1.18 `MteMkeDec -decode:encOff:encBytes:decoded:decOff:decBytes:` (ObjC)
15.4.1.19 `MteMkeDec -decodeB64:encOff:encBytes:decoded:decOff:decBytes:` (ObjC)
15.4.1.20 `MteMkeDec -startDecrypt` (ObjC)
15.4.1.21 `MteMkeDec -decryptChunk:encryptedBytes:decryptedBytes:` (ObjC)
15.4.1.22 `MteMkeDec -decryptChunk:encOff:encBytes:decrypted:decOff:` (ObjC)
15.4.1.23 `MteMkeDec -finishDecrypt:status:` (ObjC)
15.4.1.24 `MteMkeDec -getEncTs` (ObjC)
15.4.1.25 `MteMkeDec -getDecTs` (ObjC)
15.4.1.26 `MteMkeDec -getMsgSkipped` (ObjC)
15.4.1.27 `MteMkeDec -uninstantiate` (ObjC)
15.4.2 Encoder
15.4.2.1 `MteMkeEnc -init` (ObjC)
15.4.2.2 `MteMkeEnc -initWithDrbg:tokBytes:verifiers:cipher:hash:` (ObjC)
15.4.2.3 `MteMkeEnc -dealloc` (ObjC)
15.4.2.4 `MteMkeEnc -instantiate:bytes:` (ObjC)
15.4.2.5 `MteMkeEnc -instantiate:` (ObjC)
15.4.2.6 `MteMkeEnc -getReseedCounter` (ObjC)
15.4.2.7 `MteMkeEnc -saveState:` (ObjC)
15.4.2.8 `MteMkeEnc -saveStateB64` (ObjC)
15.4.2.9 `MteMkeEnc -restoreState:` (ObjC)
15.4.2.10 `MteMkeEnc -restoreStateB64:` (ObjC)
15.4.2.11 `MteMkeEnc -getBuffBytes:` (ObjC)
15.4.2.12 `MteMkeEnc -getBuffBytesB64:` (ObjC)
15.4.2.13 `MteMkeEnc -encode:dataBytes:encodedBytes:status:` (ObjC)
15.4.2.14 `MteMkeEnc -encodeB64:dataBytes:status:` (ObjC)
15.4.2.15 `MteMkeEnc -encode:encodedBytes:status:` (ObjC)
15.4.2.16 `MteMkeEnc -encodeB64:status:` (ObjC)
15.4.2.17 `MteMkeEnc -encode:dataOff:dataBytes:encoded:encOff:encBytes` (ObjC)
15.4.2.18 `MteMkeEnc -encodeB64:dataOff:dataBytes:encoded:encOff:encBytes` (ObjC)
15.4.2.19 `MteMkeEnc -encryptFinishBytes` (ObjC)
15.4.2.20 `MteMkeEnc -startEncrypt` (ObjC)
15.4.2.21 `MteMkeEnc -encryptChunk:dataBytes:` (ObjC)
15.4.2.22 `MteMkeEnc -finishEncrypt:status:` (ObjC)
15.4.2.23 `MteMkeEnc -uninstantiate` (ObjC)
16 Python Language Interface
16.1 Overview
16.1.1 Encoder
16.1.2 Decoder
16.1.3 Chunk Encrypt
16.1.4 Chunk Decrypt
16.2 Files
16.3 Types
16.3.1 Classes

16.3.1.1 `MteMkeDec`
16.3.1.2 `MteMkeEnc`
16.4 Functions
16.4.1 Decoder
16.4.1.1 `MteMkeDec` Initializer (Python)
16.4.1.2 `MteMkeDec.fromdefault` (Python)
16.4.1.3 `MteMkeDec` Delete (Python)
16.4.1.4 `MteMkeDec.instantiate` (Python)
16.4.1.5 `MteMkeDec.get_reseed_counter` (Python)
16.4.1.6 `MteMkeDec.save_state` (Python)
16.4.1.7 `MteMkeDec.save_state_b64` (Python)
16.4.1.8 `MteMkeDec.restore_state` (Python)
16.4.1.9 `MteMkeDec.restore_state_b64` (Python)
16.4.1.10 `MteMkeDec.decode` (Python)
16.4.1.11 `MteMkeDec.decode_b64` (Python)
16.4.1.12 `MteMkeDec.decode_str` (Python)
16.4.1.13 `MteMkeDec.decode_str_b64` (Python)
16.4.1.14 `MteMkeDec.start_decrypt` (Python)
16.4.1.15 `MteMkeDec.decrypt_chunk` (Python)
16.4.1.16 `MteMkeDec.finish_decrypt` (Python)
16.4.1.17 `MteMkeDec.get_enc_ts` (Python)
16.4.1.18 `MteMkeDec.get_dec_ts` (Python)
16.4.1.19 `MteMkeDec.get_msg_skipped` (Python)
16.4.1.20 `MteMkeDec.uninstantiate` (Python)
16.4.2 Encoder
16.4.2.1 `MteMkeEnc` Initializer (Python)
16.4.2.2 `MteMkeEnc.fromdefault` (Python)
16.4.2.3 `MteMkeEnc` Delete (Python)
16.4.2.4 `MteMkeEnc.instantiate` (Python)
16.4.2.5 `MteMkeEnc.get_reseed_counter` (Python)
16.4.2.6 `MteMkeEnc.save_state` (Python)
16.4.2.7 `MteMkeEnc.save_state_b64` (Python)
16.4.2.8 `MteMkeEnc.restore_state` (Python)
16.4.2.9 `MteMkeEnc.restore_state_b64` (Python)
16.4.2.10 `MteMkeEnc.encode` (Python)
16.4.2.11 `MteMkeEnc.encode_b64` (Python)
16.4.2.12 `MteMkeEnc.encrypt_finish_bytes` (Python)
16.4.2.13 `MteMkeEnc.start_encrypt` (Python)
16.4.2.14 `MteMkeEnc.encrypt_chunk` (Python)
16.4.2.15 `MteMkeEnc.finish_encrypt` (Python)
16.4.2.16 `MteMkeEnc.uninstantiate` (Python)
17 **Swift Language Interface**
17.1 Overview
17.1.1 Encoder
17.1.2 Decoder
17.1.3 Chunk Encrypt
17.1.4 Chunk Decrypt
17.2 Files
17.3 Types

17.3.1 Classes
17.3.1.1 `MteMkeDec`
17.3.1.2 `MteMkeEnc`
17.4 Functions
17.4.1 Decoder
17.4.1.1 `MteMkeDec` Initializer (default options) (Swift)
17.4.1.2 `MteMkeDec` Initializer (runtime options) (Swift)
17.4.1.3 `MteMkeDec` Deinitializer (Swift)
17.4.1.4 `MteMkeDec.instantiate` (`[UInt8]`) (Swift)
17.4.1.5 `MteMkeDec.instantiate` (`String`) (Swift)
17.4.1.6 `MteMkeDec.getReseedCounter` (Swift)
17.4.1.7 `MteMkeDec.saveState` (Swift)
17.4.1.8 `MteMkeDec.saveStateB64` (Swift)
17.4.1.9 `MteMkeDec.restoreState` (Swift)
17.4.1.10 `MteMkeDec.restoreStateB64` (Swift)
17.4.1.11 `MteMkeDec.getBuffBytes` (Swift)
17.4.1.12 `MteMkeDec.getBuffBytesB64` (Swift)
17.4.1.13 `MteMkeDec.decode` (Swift)
17.4.1.14 `MteMkeDec.decodeB64` (Swift)
17.4.1.15 `MteMkeDec.decodeStr` (Swift)
17.4.1.16 `MteMkeDec.decodeStrB64` (Swift)
17.4.1.17 `MteMkeDec.startDecrypt` (Swift)
17.4.1.18 `MteMkeDec.decryptChunk` (Swift)
17.4.1.19 `MteMkeDec.decryptChunk` (external buffers) (Swift)
17.4.1.20 `MteMkeDec.finishDecrypt` (Swift)
17.4.1.21 `MteMkeDec.getEncTs` (Swift)
17.4.1.22 `MteMkeDec.getDecTs` (Swift)
17.4.1.23 `MteMkeDec.getMsgSkipped` (Swift)
17.4.1.24 `MteMkeDec.uninstantiate` (Swift)
17.4.2 Encoder
17.4.2.1 `MteMkeEnc` Initializer (default options) (Swift)
17.4.2.2 `MteMkeEnc` Initializer (runtime options) (Swift)
17.4.2.3 `MteMkeEnc` Deinitializer (Swift)
17.4.2.4 `MteMkeEnc.instantiate` (`[UInt8]`) (Swift)
17.4.2.5 `MteMkeEnc.instantiate` (`String`) (Swift)
17.4.2.6 `MteMkeEnc.getReseedCounter` (Swift)
17.4.2.7 `MteMkeEnc.saveState` (Swift)
17.4.2.8 `MteMkeEnc.saveStateB64` (Swift)
17.4.2.9 `MteMkeEnc.restoreState` (Swift)
17.4.2.10 `MteMkeEnc.restoreStateB64` (Swift)
17.4.2.11 `MteMkeEnc.getBuffBytes` (Swift)
17.4.2.12 `MteMkeEnc.getBuffBytesB64` (Swift)
17.4.2.13 `MteMkeEnc.encode` (`[UInt8]`) (Swift)
17.4.2.14 `MteMkeEnc.encodeB64` (`[UInt8]`) (Swift)
17.4.2.15 `MteMkeEnc.encode` (`String`) (Swift)
17.4.2.16 `MteMkeEnc.encodeB64` (`String`) (Swift)
17.4.2.17 `MteMkeEnc.encode` (external buffers) (Swift)
17.4.2.18 `MteMkeEnc.encodeB64` (external buffers) (Swift)
17.4.2.19 `MteMkeEnc.encryptFinishBytes` (Swift)
17.4.2.20 `MteMkeEnc.startEncrypt` (Swift)
17.4.2.21 `MteMkeEnc.encryptChunk` (Swift)

17.4.2.22 `MteMkeEnc.encryptChunk` (partial) (Swift)
17.4.2.23 `MteMkeEnc.finishEncrypt` (Swift)
17.4.2.24 `MteMkeEnc.uninstantiate` (Swift)

18 WASM Language Interface
18.1 Overview
18.1.1 Encoder
18.1.2 Decoder
18.1.3 Chunk Encrypt
18.1.4 Chunk Decrypt
18.2 Files
18.3 Types
18.3.1 Classes
18.3.1.1 `MteMkeDec`
18.3.1.2 `MteMkeEnc`
18.4 Functions
18.4.1 Decoder
18.4.1.1 `MteMkeDec` Constructor (WASM)
18.4.1.2 `MteMkeDec.fromdefault` (WASM)
18.4.1.3 `MteMkeDec.destruct` (WASM)
18.4.1.4 `MteMkeDec.instantiate` (WASM)
18.4.1.5 `MteMkeDec.getReseedCounter` (WASM)
18.4.1.6 `MteMkeDec.saveState` (WASM)
18.4.1.7 `MteMkeDec.saveStateB64` (WASM)
18.4.1.8 `MteMkeDec.restoreState` (WASM)
18.4.1.9 `MteMkeDec.restoreStateB64` (WASM)
18.4.1.10 `MteMkeDec.decode` (WASM)
18.4.1.11 `MteMkeDec.decodeB64` (WASM)
18.4.1.12 `MteMkeDec.decodeStr` (WASM)
18.4.1.13 `MteMkeDec.decodeStrB64` (WASM)
18.4.1.14 `MteMkeDec.startDecrypt` (WASM)
18.4.1.15 `MteMkeDec.decryptChunk` (WASM)
18.4.1.16 `MteMkeDec.finishDecrypt` (WASM)
18.4.1.17 `MteMkeDec.getEncTs` (WASM)
18.4.1.18 `MteMkeDec.getDecTs` (WASM)
18.4.1.19 `MteMkeDec.getMsgSkipped` (WASM)
18.4.1.20 `MteMkeDec.uninstantiate` (WASM)
18.4.2 Encoder
18.4.2.1 `MteMkeEnc` Constructor (WASM)
18.4.2.2 `MteMkeEnc.fromdefault` (WASM)
18.4.2.3 `MteMkeEnc.destruct` (WASM)
18.4.2.4 `MteMkeEnc.instantiate` (WASM)
18.4.2.5 `MteMkeEnc.getReseedCounter` (WASM)
18.4.2.6 `MteMkeEnc.saveState` (WASM)
18.4.2.7 `MteMkeEnc.saveStateB64` (WASM)
18.4.2.8 `MteMkeEnc.restoreState` (WASM)
18.4.2.9 `MteMkeEnc.restoreStateB64` (WASM)
18.4.2.10 `MteMkeEnc.encode` (WASM)
18.4.2.11 `MteMkeEnc.encodeB64` (WASM)
18.4.2.12 `MteMkeEnc.encodeStr` (WASM)

18.4.2.13 `MteMkeEnc.encodeStrB64` (WASM)
18.4.2.14 `MteMkeEnc.encryptFinishBytes` (WASM)
18.4.2.15 `MteMkeEnc.startEncrypt` (WASM)
18.4.2.16 `MteMkeEnc.encryptChunk` (WASM)
18.4.2.17 `MteMkeEnc.finishEncrypt` (WASM)
18.4.2.18 `MteMkeEnc.uninstantiate` (WASM)
19 Contact Eclypses

1 Introduction

The Managed-Key Encryption Add-On replaces the core encoder and decoder, which only do tokenization, with an encoder and decoder that combine standard encryption with tokenization. This allows much larger data to take advantage of the MTE technology without significantly increasing the data size.

Using the core MTE technology, the Managed-Key Encryption Add-On can generate a unique random encryption key for each piece of data, encrypt the data, then decrypt it without ever transmitting or sharing that key in any way. That key is then discarded and the next piece of data will use a new, unrelated key. The encrypted data is hashed and that hash is tokenized with the core MTE algorithm to provide a superior level of authentication.

The Managed-Key Encryption Add-On can be used in two different ways. The first, most common use is encoder/decoder mode. The other way to use the add-on is in chunk encrypt/decrypt mode. Both are described below. Regardless of which mode is used, the normal instantiation process must occur before using the encoder or decoder. The modes are not compatible with each other, so an encode must pair with a decode and a chunk encrypt must pair with a chunk decrypt.

1.1 Encoder/Decoder Mode

This mode uses the Managed-Key Encryption Add-On just like a core encoder and decoder. The interface is almost identical to the core encoder and decoder interface. An encoder calls an encode function to encode an input and a decoder calls a decode function to decode that encoded version. Each encode call will use a different encryption key.

The encoder automatically pads inputs as necessary for the size, cipher, and mode of operation in use. The decoder automatically removes any added padding.

1.2 Chunk Mode

This mode uses the Managed-Key Encryption Add-On to encrypt very large pieces of data that are not suitable for a single encode operation. The interface allows the data to be streamed through in chunks to keep memory usage reasonable. The interface requires a call to start a new chunk session, calls to encrypt/decrypt each chunk, then a call to finalize the session, which adds a tokenized hash for authentication. Each new session will use a different encryption key.

1.2.1 Chunk Encrypt Process

1. Start the chunk session.
2. Encrypt each chunk. The chunks must be a multiple of the cipher's block size for the chosen mode of operation.
3. Finish the chunk session. Ensure a successful status.

Steps 2 and 3 produce outputs (the encrypted data and the tokenized hash with optional verifiers). The outputs from these steps, concatenated together in order, are the full result of the chunked encryption process. This concatenated output must be fed in full to the chunk decrypt process to succeed. A common mistake is to neglect to include the result of step 3 as part of the result.

1.2.2 Chunk Decrypt Process

1. Start the chunk session.
2. Decrypt each chunk. The chunks can be any size, unlike the encryption process. The chunks must include all of the encrypted data in order as well as the tokenized hash produced in step 3 of the chunk encrypt process.
3. Finish the chunk session. Check the status to make sure the result is authentic and that any optional verifiers did not indicate problems.

After step 3, the normal informational functions like retrieving the timestamps (if the timestamp verifier is enabled) are available, just like in normal encoder/decoder mode.

2 Terms and Abbreviations

Refer to the *MTE Core Developer's Guide* terms and abbreviations.

3 Prerequisites

Refer to the *MTE Core Developer's Guide* prerequisites.

4 Caveats

Refer to the *MTE Core Developer's Guide* caveats.

5 Architecture

Refer to the *MTE Core Developer's Guide* architecture.

6 SDK Structure

Refer to the *MTE Core Developer's Guide* SDK structure.

7 Options

Refer to the *MTE Core Developer's Guide* options. The Managed-Key Encryption Add-On adds some additional options. The options must be the same for a decoder to successfully decode what the encoder has encoded. The following table lists the options the Managed-Key Encryption Add-On adds to the core and whether they apply to encoders, decoders, or both. Subsequent subsections go in to further detail on the options.

Option	Encoder/Decoder	Description
Cipher	E/D	The cipher. The choice will determine the security strength. The choices are: `None`: No MTE-provided cipher is selected. The SDK user must provide a cipher via an MTE-defined callback interface. This is intended for highly specialized use cases, and the SDK user must ensure the cipher is cryptographically secure. `AES-128-CBC`: AES-128 using the Cipher Block Chaining mode of operation. `AES-128-CTR`: AES-128 using the Counter mode of operation. `AES-128-ECB`: AES-128 using the Electronic Code Book mode of operation. `AES-192-CBC`: AES-192 using the Cipher Block Chaining mode of operation. `AES-192-CTR`: AES-192 using the Counter mode of operation. `AES-192-ECB`: AES-192 using the Electronic Code Book mode of operation. `AES-256-CBC`: AES-256 using the Cipher Block Chaining mode of operation. `AES-256-CTR`: AES-256 using the Counter mode of operation. `AES-256-ECB`: AES-256 using the Electronic Code Book mode of operation.
Hash	E/D	The hash. The choice will determine the authentication strength. The choices are: `None`: No MTE-provided hash is selected. The SDK user must provide a hash via an MTE-defined callback interface. This is intended for highly specialized use cases, and the SDK user must ensure the hash is cryptographically secure. `CRC-32`: CRC-32. This is not cryptographically secure. `SHA-1`: SHA-1. This is no longer considered cryptographically secure for hashing. `SHA-256`: SHA-256. `SHA-512`: SHA-512.

7.1 Choosing a Cipher

To choose a cipher, first determine the required security strength of your system or application. There may be multiple choices for that security strength. Next, determine if the DRBG is cipher-based and can use the same cipher as that may help decide between competing choices. For example, if you are using an AES-256 DRBG, you could choose any of the AES-256 options for the cipher, which will

help reduce code size.

Next you need to determine the mode of operation to use with the cipher. Note that some modes of operation have strict limitations (e.g., ECB should only be used for encrypting a single block of data). If you are using chunk mode, you will be required to ensure the data is chunked to a multiple of the block size, which may necessitate a padding algorithm. If not using chunk mode, the Managed-Key Encryption Add-On will pad inputs if necessary, but is able to remove the padding, so the process is transparent.

Beyond those guidelines, it comes down to preferences for the organization.

7.2 Choosing a Hash

To choose a hash, first determine the required security strength of your system or application. There may be multiple choices for that security strength. Next, determine if the DRBG is hash-based and can use the same hash as that may help decide between competing choices. For example, if you are using a SHA-256 DRBG, you could choose SHA-256 for the hash, which will help reduce code size.

Beyond those guidelines, it comes down to preferences for the organization.

8 Status

Refer to the *MTE Core Developer's Guide* status.

9 Basic C API

9.1 Overview

9.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, get the state size with `mte_mke_enc_state_bytes()`, initialize the state with `mte_mke_enc_state_init()`, and instantiate with `mte_mke_enc_instantiate()`.

To encode, use `mte_mke_enc_encode()` or `mte_mke_enc_encode_b64()`.

To destroy, use `mte_mke_enc_uninstantiate()`.

9.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and destroy it.

To create, get the state size with `mte_mke_dec_state_bytes()`, initialize the state with `mte_mke_dec_state_init()`, and instantiate with `mte_mke_dec_instantiate()`.

To decode, use `mte_mke_dec_decode()` or `mte_mke_dec_decode_b64()`, depending on which encode was used.

To get additional information, use `mte_mke_dec_enc_ts()`, `mte_mke_dec_dec_ts()`, and `mte_mke_dec_msg_skipped()`, depending on which verifiers are enabled.

To destroy, use `mte_mke_dec_uninstantiate()`.

9.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `mte_mke_enc_encrypt_state_bytes()` to get the chunk state size, `mte_mke_enc_encrypt_start()` to start a chunk session, `mte_mke_enc_encrypt_chunk()` repeatedly to encrypt each chunk, then `mte_mke_enc_encrypt_finish()` to finish the session.

9.1.4 Chunk Decrypt

The chunk decrypt follows the flow of a decoder, but instead of using a decode method, use `mte_mke_dec_decrypt_state_bytes()` to get the chunk state size, `mte_mke_dec_decrypt_start()` to start a chunk session, `mte_mke_dec_decrypt_chunk()` repeatedly to decrypt each chunk (and the result of the encrypt finish), then `mte_mke_dec_decrypt_finish()` to finish the session.

9.2 Files

The MTE Managed-Key Encryption Add-On uses the core header files (other than `mte_enc.h` and `mte_dec.h`) documented in the MTE Core Developer's Guide, as well as the following:

File	Description
mte_mke_dec.h	This defines the core decoder functions.
mte_mke_enc.h	This defines the core encoder functions.

9.3 Functions

9.3.1 Decoder

Decoder-specific functions are declared in `mte_mke_dec.h`.

9.3.1.1 `mte_mke_dec_state_bytes` (C)

```
uint32_t mte_mke_dec_state_bytes(mte_drbgs drbg, uint32_t tok_bytes, mte_verifiers
verifiers, mte_ciphers cipher, mte_hashes hash)
```

Returns the decoder state size for the given DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided DRBG, cipher, and hash are going to be used. The associated initialization function is `mte_mke_dec_state_init()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_dec_state_bytes_d()`, `mte_mke_dec_state_bytes_dc()`, `mte_mke_dec_state_bytes_dh()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_dec_state_bytes_c()`, `mte_mke_dec_state_bytes_dc()`, `mte_mke_dec_state_bytes_ch()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided cipher is not going to be used.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_dec_state_bytes_h()`, `mte_mke_dec_state_bytes_dh()`, `mte_mke_dec_state_bytes_ch()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided hash is not going to be used.

9.3.1.2 `mte_mke_dec_state_bytes_d` (C)

```
uint32_t mte_mke_dec_state_bytes_d(uint32_t tok_bytes, mte_verifiers verifiers,
mte_ciphers cipher, mte_hashes hash)
```

Returns the decoder state size for the given token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm. Returns `0` if the combination of arguments is not usable. Use this function when an MTE-provided DRBG is not going to be used and MTE-provided cipher and hash are going to be used. The associated initialization function is `mte_mke_dec_state_init_d()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_dec_state_bytes_c()`, `mte_mke_dec_state_bytes_dc()`, `mte_mke_dec_state_bytes_ch()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided cipher is not going to be used.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_dec_state_bytes_h()`, `mte_mke_dec_state_bytes_dh()`, `mte_mke_dec_state_bytes_ch()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided hash is not going to be used.

9.3.1.3 `mte_mke_dec_state_bytes_c` (C)

```
uint32_t mte_mke_dec_state_bytes_c(mte_drbgs drbg, uint32_t tok_bytes,
mte_verifiers verifiers, mte_hashes hash)
```

Returns the decoder state size for the given DRBG algorithm, token size in bytes, verifiers algorithm, and hash algorithm. Returns `0` if the combination of arguments is not usable. Use this function when an MTE-provided cipher is not going to be used and MTE-provided DRBG and hash are going to be used. The associated initialization function is `mte_mke_dec_state_init_c()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_dec_state_bytes_d()`, `mte_mke_dec_state_bytes_dc()`, `mte_mke_dec_state_bytes_dh()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_dec_state_bytes_h()`, `mte_mke_dec_state_bytes_dh()`, `mte_mke_dec_state_bytes_ch()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided hash is

not going to be used.

9.3.1.4 `mte_mke_dec_state_bytes_h` (C)

```
uint32_t mte_mke_dec_state_bytes_h(mte_drbgs drbg, uint32_t tok_bytes,
mte_verifiers verifiers, mte_ciphers cipher)
```

Returns the decoder state size for the given DRBG algorithm, token size in bytes, verifiers algorithm, and cipher algorithm. Returns `0` if the combination of arguments is not usable. Use this function when an MTE-provided hash is not going to be used and MTE-provided DRBG and cipher are going to be used. The associated initialization function is `mte_mke_dec_state_init_h()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_dec_state_bytes_d()`, `mte_mke_dec_state_bytes_dc()`, `mte_mke_dec_state_bytes_dh()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_dec_state_bytes_c()`, `mte_mke_dec_state_bytes_dc()`, `mte_mke_dec_state_bytes_ch()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided cipher is not going to be used.

9.3.1.5 `mte_mke_dec_state_bytes_dc` (C)

```
uint32_t mte_mke_dec_state_bytes_dc(uint32_t tok_bytes, mte_verifiers verifiers,
mte_hashes hash)
```

Returns the decoder state size for the given token size in bytes, verifiers algorithm, and hash algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided DRBG and cipher are not going to be used and an MTE-provided hash is going to be used. The associated initialization function is `mte_mke_dec_state_init_dc()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_dec_state_bytes_h()`, `mte_mke_dec_state_bytes_dh()`, `mte_mke_dec_state_bytes_ch()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided hash is not going to be used.

9.3.1.6 `mte_mke_dec_state_bytes_dh` (C)

```
uint32_t mte_mke_dec_state_bytes_dh(uint32_t tok_bytes, mte_verifiers verifiers,
mte_ciphers cipher)
```

Returns the decoder state size for the given token size in bytes, verifiers algorithm, and cipher algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided DRBG and hash are not going to be used and an MTE-provided cipher is going to be used. The associated initialization function is `mte_mke_dec_state_init_dh()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_dec_state_bytes_c()`, `mte_mke_dec_state_bytes_dc()`, `mte_mke_dec_state_bytes_ch()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided cipher is not going to be used.

9.3.1.7 `mte_mke_dec_state_bytes_ch` (C)

```
uint32_t mte_mke_dec_state_bytes_ch(mte_drbgs drbg, uint32_t tok_bytes,
mte_verifiers verifiers)
```

Returns the decoder state size for the given DRBG algorithm, token size in bytes, and verifiers algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided cipher and hash are not going to be used and an MTE-provided DRBG is going to be used. The associated initialization function is `mte_mke_dec_state_init_ch()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_dec_state_bytes_d()`, `mte_mke_dec_state_bytes_dc()`, `mte_mke_dec_state_bytes_dh()`, or `mte_mke_dec_state_bytes_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.

9.3.1.8 `mte_mke_dec_state_bytes_dch` (C)

```
uint32_t mte_mke_dec_state_bytes_dch(uint32_t tok_bytes, mte_verifiers verifiers)
```

Returns the decoder state size for the given token size in bytes and verifiers algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided DRBG, cipher, and hash are not going to be used. The associated initialization function is `mte_mke_dec_state_init_dch()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.

9.3.1.9 `mte_mke_dec_state_init` (C)

```
mte_status mte_mke_dec_state_init(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, mte_ciphers cipher, mte_hashes hash, uint64_t
t_window, int32_t s_window)
```

Initialize the decoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window. Returns the status. Use this function when MTE-provided DRBG, cipher, and hash are going to be used.

The state buffer must be of sufficient length to hold the decoder state. See `mte_mke_dec_state_bytes()`.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_dec_state_init_d()`, `mte_mke_dec_state_init_dc()`, `mte_mke_dec_state_init_dh()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_dec_state_init_c()`, `mte_mke_dec_state_init_dc()`, `mte_mke_dec_state_init_ch()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided cipher is not going to be used.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_dec_state_init_h()`, `mte_mke_dec_state_init_dh()`, `mte_mke_dec_state_init_ch()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided hash is not going to be used.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

9.3.1.10 `mte_mke_dec_state_init_d` (C)

```
mte_status mte_mke_dec_state_init_d(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, mte_ciphers cipher, mte_hashes hash, uint64_t t_window, int32_t
s_window)
```

Initialize the decoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window. Returns the status. Use this function when an MTE-provided DRBG is not going to be used and MTE-provided cipher and hash are going to be used.

The state buffer must be of sufficient length to hold the decoder state. See `mte_mke_dec_state_bytes_d()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.
`g_cb`: DRBG generate callback.
`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_dec_state_init_c()`, `mte_mke_dec_state_init_dc()`, `mte_mke_dec_state_init_ch()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided cipher is not going to be used.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_dec_state_init_h()`, `mte_mke_dec_state_init_dh()`, `mte_mke_dec_state_init_ch()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided hash is not going to be used.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

9.3.1.11 `mte_mke_dec_state_init_c` (C)

```
mte_status mte_mke_dec_state_init_c(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, mte_cipher_key cdk_cb, mte_cipher_enc_dec
cd_cb, mte_cipher_uninit cu_cb, uint32_t key_bytes, uint32_t block_bytes, void
*cipher_state, mte_hashes hash, uint64_t t_window, int32_t s_window)
```

Initialize the decoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher callbacks, cipher key/block size, cipher state pointer, hash algorithm, timestamp window, and sequence window. Returns the status. Use this function when an MTE-provided cipher is not going to be used and MTE-provided DRBG and hash are going to be used.

The state buffer must be of sufficient length to hold the decoder state. See `mte_mke_dec_state_bytes_c()`.

The `cipher_state` must point at the external cipher state and must remain in scope as long as the state will be used.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, and `hash` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_dec_state_init_d()`, `mte_mke_dec_state_init_dc()`, `mte_mke_dec_state_init_dh()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cdk_cb`: cipher decrypt key callback.
`cd_cb`: cipher decrypt callback.
`cu_cb`: cipher uninit callback.
`key_bytes`: cipher key size in bytes.
`block_bytes`: cipher block size in bytes.
`cipher_state`: cipher state buffer.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_dec_state_init_h()`, `mte_mke_dec_state_init_dh()`, `mte_mke_dec_state_init_ch()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided hash is not going to be used.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

9.3.1.12 `mte_mke_dec_state_init_h` (C)

```
mte_status mte_mke_dec_state_init_h(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, mte_ciphers cipher, mte_hash_calc hc_cb,
mte_hash_start hs_cb, mte_hash_feed hf_cb, mte_hash_finish hfin_cb, uint32_t
digest_bytes, void *hash_state, uint64_t t_window, int32_t s_window)
```

Initialize the decoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash callbacks, hash digest size, hash state pointer, timestamp window, and sequence window. Returns the status. Use this function when an MTE-provided hash is not going to be used and MTE-provided DRBG and cipher are going to be used.

The state buffer must be of sufficient length to hold the decoder state. See `mte_mke_dec_state_bytes_h()`.

The `hash_state` must point at the external hash state and must remain in scope as long as the state will be used.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, and `cipher` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_dec_state_init_d()`, `mte_mke_dec_state_init_dc()`, `mte_mke_dec_state_init_dh()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_dec_state_init_c()`, `mte_mke_dec_state_init_dc()`, `mte_mke_dec_state_init_ch()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided cipher is not going to be used.
`hc_cb`: hash calculate callback.
`hs_cb`: hash incremental start callback.
`hf_cb`: hash incremental feed callback.
`hfin_cb`: hash incremental finish callback.
`digest_bytes`: hash digest size in bytes.
`hash_state`: hash state buffer.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

9.3.1.13 `mte_mke_dec_state_init_dc` (C)

```
mte_status mte_mke_dec_state_init_dc(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, mte_cipher_key cdk_cb, mte_cipher_enc_dec cd_cb, mte_cipher_uninit
cu_cb, uint32_t key_bytes, uint32_t block_bytes, void *cipher_state, mte_hashes
hash, uint64_t t_window, int32_t s_window)
```

Initialize the decoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, cipher callbacks, cipher key/block size, cipher state pointer, hash algorithm, timestamp window, and sequence window. Returns the status. Use this function when MTE-provided DRBG and cipher are not going to be used and an MTE-provided hash is going to be used.

The state buffer must be of sufficient length to hold the decoder state. See `mte_mke_dec_state_bytes_dc()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

The `cipher_state` must point at the external cipher state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes`, `verifiers`, and `hash` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.
`g_cb`: DRBG generate callback.
`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cdk_cb`: cipher decrypt key callback.
`cd_cb`: cipher decrypt callback.
`cu_cb`: cipher uninit callback.
`key_bytes`: cipher key size in bytes.
`block_bytes`: cipher block size in bytes.
`cipher_state`: cipher state buffer.

`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_dec_state_init_h()`, `mte_mke_dec_state_init_dh()`, `mte_mke_dec_state_init_ch()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided hash is not going to be used.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

9.3.1.14 `mte_mke_dec_state_init_dh` (C)

```
mte_status mte_mke_dec_state_init_dh(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, mte_ciphers cipher, mte_hash_calc hc_cb, mte_hash_start hs_cb,
mte_hash_feed hf_cb, mte_hash_finish hfin_cb, uint32_t digest_bytes, void
*hash_state, uint64_t t_window, int32_t s_window)
```

Initialize the decoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, cipher algorithm, hash callbacks, hash digest size, hash state pointer, timestamp window, and sequence window. Returns the status. Use this function when MTE-provided DRBG and hash are not going to be used and an MTE-provided cipher is going to be used.

The state buffer must be of sufficient length to hold the decoder state. See `mte_mke_dec_state_bytes_dh()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

The `hash_state` must point at the external hash state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes`, `verifiers`, and `cipher` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.
`g_cb`: DRBG generate callback.
`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use

`mte_mke_dec_state_init_c()`, `mte_mke_dec_state_init_dc()`, `mte_mke_dec_state_init_ch()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided cipher is not going to be used.
`hc_cb`: hash calculate callback.
`hs_cb`: hash incremental start callback.
`hf_cb`: hash incremental feed callback.
`hfin_cb`: hash incremental finish callback.
`digest_bytes`: hash digest size in bytes.
`hash_state`: hash state buffer.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

9.3.1.15 `mte_mke_dec_state_init_ch` (C)

```
mte_status mte_mke_dec_state_init_ch(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, mte_cipher_key cdk_cb, mte_cipher_enc_dec
cd_cb, mte_cipher_uninit cu_cb, uint32_t key_bytes, uint32_t block_bytes, void
*cipher_state, mte_hash_calc hc_cb, mte_hash_start hs_cb, mte_hash_feed hf_cb,
mte_hash_finish hfin_cb, uint32_t digest_bytes, void *hash_state, uint64_t
t_window, int32_t s_window)
```

Initialize the decoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher callbacks, cipher key/block size, cipher state pointer, hash callbacks, hash digest size, hash state pointer, timestamp window, and sequence window. Returns the status. Use this function when MTE-provided cipher and hash are not going to be used and an MTE-provided DRBG is going to be used.

The state buffer must be of sufficient length to hold the decoder state. See `mte_mke_dec_state_bytes_ch()`.

The `cipher_state` must point at the external cipher state and must remain in scope as long as the state will be used.

The `hash_state` must point at the external hash state and must remain in scope as long as the state will be used.

Buildtime options: the `drbg`, `tok_bytes`, and `verifiers` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_dec_state_init_d()`, `mte_mke_dec_state_init_dc()`, `mte_mke_dec_state_init_dh()`, or `mte_mke_dec_state_init_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cdk_cb`: cipher decrypt key callback.
`cd_cb`: cipher decrypt callback.

`cu_cb`: cipher uninit callback.
`key_bytes`: cipher key size in bytes.
`block_bytes`: cipher block size in bytes.
`cipher_state`: cipher state buffer.
`hc_cb`: hash calculate callback.
`hs_cb`: hash incremental start callback.
`hf_cb`: hash incremental feed callback.
`hfin_cb`: hash incremental finish callback.
`digest_bytes`: hash digest size in bytes.
`hash_state`: hash state buffer.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

9.3.1.16 `mte_mke_dec_state_init_dch` (C)

```
mte_status mte_mke_dec_state_init_dch(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, mte_cipher_key cdk_cb, mte_cipher_enc_dec cd_cb, mte_cipher_uninit
cu_cb, uint32_t key_bytes, uint32_t block_bytes, void *cipher_state, mte_hash_calc
hc_cb, mte_hash_start hs_cb, mte_hash_feed hf_cb, mte_hash_finish hfin_cb, uint32_t
digest_bytes, void *hash_state, uint64_t t_window, int32_t s_window)
```

Initialize the decoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, cipher callbacks, cipher key/block size, cipher state pointer, hash callbacks, hash digest size, hash state pointer, timestamp window, and sequence window. Returns the status. Use this function when MTE-provided DRBG, cipher, and hash are not going to be used.

The state buffer must be of sufficient length to hold the decoder state. See `mte_mke_dec_state_bytes_dch()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

The `cipher_state` must point at the external cipher state and must remain in scope as long as the state will be used.

The `hash_state` must point at the external hash state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes` and `verifiers` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.
`g_cb`: DRBG generate callback.
`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cdk_cb`: cipher decrypt key callback.
`cd_cb`: cipher decrypt callback.
`cu_cb`: cipher uninit callback.
`key_bytes`: cipher key size in bytes.
`block_bytes`: cipher block size in bytes.
`cipher_state`: cipher state buffer.
`hc_cb`: hash calculate callback.
`hs_cb`: hash incremental start callback.
`hf_cb`: hash incremental feed callback.
`hfin_cb`: hash incremental finish callback.
`digest_bytes`: hash digest size in bytes.
`hash_state`: hash state buffer.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

9.3.1.17 `mte_mke_dec_instantiate` (C)

```
mte_status mte_mke_dec_instantiate(MTE_HANDLE state, mte_drbg_get_entropy_input
ei_cb, void *ei_cb_context, mte_drbg_get_nonce n_cb, void *n_cb_context, const void
*ps, uint32_t ps_bytes)
```

Instantiate the decoder given the entropy input callback/context, nonce callback/context, personalization string, and length of the personalization string in bytes. Returns the status.

`state`: handle to the decoder state.
`ei_cb`: entropy input callback.
`ei_cb_context`: pointer that will be passed through to the `ei_cb` as the context argument.
`n_cb`: nonce callback.
`n_cb_context`: pointer that will be passed through to the `n_cb` as the context argument.
`ps`: personalization string.
`ps_bytes`: length of the personalization string in bytes.

9.3.1.18 `mte_mke_dec_reseed_counter` (C)

```
uint64_t mte_mke_dec_reseed_counter(MTE_CHANDLE state)
```

Returns the decoder's DRBG reseed counter.

`state`: handle to the decoder state.

9.3.1.19 `mte_mke_dec_save_bytes` (C)

```
uint32_t mte_mke_dec_save_bytes(MTE_CHANDLE state)
```

Returns the state save size for raw state. The associated state save function is `mte_mke_dec_state_save()`.

`state`: handle to the decoder state.

9.3.1.20 `mte_mke_dec_save_bytes_b64` (C)

```
uint32_t mte_mke_dec_save_bytes_b64(MTE_CHANDLE state)
```

Returns the state save size for Base64-encoded state. The associated state save function is `mte_mke_dec_state_save_b64()`.

`state`: handle to the decoder state.

9.3.1.21 `mte_mke_dec_state_save` (C)

```
mte_status mte_mke_dec_state_save(MTE_CHANDLE state, void *saved)
```

Saves the decoder state to the given buffer in raw form. The size of the buffer must be `mte_mke_dec_save_bytes()` and that is the length of the raw saved state. Returns the status.

`state`: handle to the decoder state.
`saved`: the buffer to save the state to.

9.3.1.22 `mte_mke_dec_state_save_b64` (C)

```
mte_status mte_mke_dec_state_save_b64(MTE_CHANDLE state, void *saved)
```

Saves the decoder state to the given buffer in Base64-encoded form. The size of the buffer must be `mte_mke_dec_save_bytes_b64()` and the encoded saved state is null-terminated. Returns the status.

`state`: handle to the decoder state.
`saved`: the buffer to save the state to.

9.3.1.23 `mte_mke_dec_state_restore` (C)

```
mte_status mte_mke_dec_state_restore(MTE_HANDLE state, const void *saved)
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`state`: handle to the decoder state.
`saved`: the buffer to restore the state from.

9.3.1.24 `mte_mke_dec_state_restore_b64` (C)

```
mte_status mte_mke_dec_state_restore_b64(MTE_HANDLE state, void *saved)
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`state`: handle to the decoder state.
`saved`: the buffer to restore the state from. The Base64 is overwritten with the decoded form.

9.3.1.25 `mte_mke_dec_buff_bytes` (C)

```
uint32_t mte_mke_dec_buff_bytes(MTE_CHANDLE state, uint32_t encoded_bytes)
```

Returns the decode buffer size in bytes given the encoded length in bytes for raw form decoding. The decode buffer provided to `mte_mke_dec_decode()` must be of at least this length.

`state`: handle to the decoder state.
`encoded_bytes`: the length of the data to be decoded (in bytes).

9.3.1.26 `mte_mke_dec_buff_bytes_b64` (C)

```
uint32_t mte_mke_dec_buff_bytes_b64(MTE_CHANDLE state, uint32_t encoded_bytes)
```

Returns the decode buffer size in bytes given the encoded length in bytes for Base64-encoded form decoding. The decode buffer provided to `mte_mke_dec_decode_b64()` must be of at least this length.

`state`: handle to the decoder state.
`encoded_bytes`: the length of the data to be decoded (in bytes).

9.3.1.27 `mte_mke_dec_decode` (C)

```
mte_status mte_mke_dec_decode(MTE_HANDLE state, mte_verifier_get_timestamp64 t_cb,
void *t_cb_context, const void *encoded, uint32_t encoded_bytes, void *decoded,
uint32_t *decoded_off, uint32_t *decoded_bytes)
```

Decode the given raw encoded data of the given byte length to the given decode buffer. Returns the status. Sets `*decoded_off` to the offset in the `decoded` buffer where the decoded version can be found and sets `*decoded_bytes` to the decoded data length in bytes. The decoded version is valid only if `!mte_base_status_is_error(status)`. A null terminator is appended to the decoded data but is not included in `*decoded_bytes`.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

The `decoded` buffer must be of sufficient length to hold the decoded version. See `mte_mke_dec_buff_bytes()`.

`state`: handle to the decoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`encoded`: the encoded data to decode.
`encoded_bytes`: the length of the encoded data to be decoded (in bytes).
`decoded`: the buffer to write the decoded version of the input to.
`decoded_off`: pointer to the offset of the decoded version.
`decoded_bytes`: pointer to the length of the decoded version.

9.3.1.28 `mte_mke_dec_decode_b64` (C)

```
mte_status mte_mke_dec_decode_b64(MTE_HANDLE state, mte_verifier_get_timestamp64
t_cb, void *t_cb_context, const void *encoded, uint32_t encoded_bytes, void
*decoded, uint32_t *decoded_off, uint32_t *decoded_bytes)
```

Decode the given Base64-encoded encoded data of the given byte length to the given decode buffer. Returns the status. Sets `*decoded_off` to the offset in the `decoded` buffer where the Base64 version can be found and sets `*decoded_bytes` to the decoded version length in bytes. The decoded version is null terminated, but `*decoded_bytes` excludes the null terminator. The decoded version is valid only if `!mte_base_status_is_error(status)`.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

The `decoded` buffer must be of sufficient length to hold the decoded version. See `mte_mke_dec_buff_bytes_b64()`.

`state`: handle to the decoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`encoded`: the encoded data to decode.
`encoded_bytes`: the length of the encoded data to be decoded (in bytes).
`decoded`: the buffer to write the decoded version of the input to.
`decoded_off`: pointer to the offset of the decoded version.
`decoded_bytes`: pointer to the length of the decoded version.

9.3.1.29 `mte_mke_dec_decrypt_state_bytes` (C)

```
uint32_t mte_mke_dec_decrypt_state_bytes(MTE_CHANDLE state)
```

Returns the chunk-based decryption state size in bytes. The `c_state` buffer provided to the `mte_mke_dec_decrypt_start()`, `mte_mke_dec_decrypt_chunk()`, and `mte_mke_dec_decrypt_finish()` functions must be of at least this length.

`state`: handle to the decoder state.

9.3.1.30 `mte_mke_dec_decrypt_start` (C)

```
mte_status mte_mke_dec_decrypt_start(MTE_HANDLE state, MTE_HANDLE c_state)
```

Start a chunk-based decryption session. Initializes the state to the `c_state` buffer. This state is used by the other chunk-based functions. Returns the status.

The `c_state` buffer must be of sufficient length. See `mte_mke_dec_decrypt_state_bytes()`.

`state`: handle to the decoder state.
`c_state`: handle to the chunk-based decryption session state.

9.3.1.31 `mte_mke_dec_decrypt_chunk` (C)

```
mte_status mte_mke_dec_decrypt_chunk(MTE_HANDLE state, MTE_HANDLE c_state, const
void *encrypted, uint32_t encrypted_bytes, void *decrypted, uint32_t
*decrypted_bytes)
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data of length `encrypted_bytes` is used as input and some decrypted data is written to the `decrypted` buffer and `*decrypted_bytes` is set to the amount of decrypted data written. The amount decrypted may be greater or less than the input size. Returns the status.

The `decrypted` buffer must be at least `encrypted_bytes` plus the cipher block size bytes in length to hold the decrypted version. The `decrypted` buffer cannot overlap the `encrypted` buffer.

`state`: handle to the decoder state.
`c_state`: handle to the chunk-based decryption session state.
`encrypted`: the encrypted data to decrypt.
`encrypted_bytes`: the length of the encrypted data to be decrypted (in bytes).
`decrypted`: the buffer to write the decrypted data to.
`decrypted_bytes`: pointer to the length of data decrypted. This may be greater or less than `encrypted_bytes` due to cipher block boundaries and buffering for the tokenized hash.

9.3.1.32 `mte_mke_dec_decrypt_finish` (C)

```
mte_status mte_mke_dec_decrypt_finish(MTE_HANDLE state, MTE_HANDLE c_state,
mte_verifier_get_timestamp64 t_cb, void *t_cb_context, uint32_t *decrypted_off,
uint32_t *decrypted_bytes)
```

Finish the chunk-based decryption session. Returns the status. Writes the final decrypted data to the `c_state` buffer and sets `*decrypted_off` to the offset in the `c_state` buffer of the final decrypted data and sets `*decrypted_bytes` to the length of the final part of the decrypted data. The `c_state` is no longer usable for a chunk-based decryption session until `mte_mke_dec_decrypt_start()` is called.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

`state`: handle to the decoder state.
`c_state`: handle to the chunk-based decryption session state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`decrypted_off`: pointer to the offset of the data decrypted.
`decrypted_bytes`: pointer to the length of data decrypted.

9.3.1.33 `mte_mke_dec_enc_ts` (C)

```
uint64_t mte_mke_dec_enc_ts(MTE_CHANDLE state)
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

`state`: handle to the decoder state.

9.3.1.34 `mte_mke_dec_dec_ts` (C)

```
uint64_t mte_mke_dec_dec_ts(MTE_CHANDLE state)
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

`state`: handle to the decoder state.

9.3.1.35 `mte_mke_dec_msg_skipped` (C)

```
uint32_t mte_mke_dec_msg_skipped(MTE_CHANDLE state)
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

`state`: handle to the decoder state.

9.3.1.36 `mte_mke_dec_uninstantiate` (C)

```
mte_status mte_mke_dec_uninstantiate(MTE_HANDLE state)
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The state must not be used until instantiated again.

`state`: handle to the decoder state.

9.3.2 Encoder

Encoder-specific functions are declared in `mte_mke_enc.h`.

9.3.2.1 `mte_mke_enc_state_bytes` (C)

```
uint32_t mte_mke_enc_state_bytes(mte_drbgs drbg, uint32_t tok_bytes, mte_verifiers
verifiers, mte_ciphers cipher, mte_hashes hash)
```

Returns the encoder state size for the given DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided DRBG, cipher, and hash are going to be used. The associated initialization function is `mte_mke_enc_state_init()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_enc_state_bytes_d()`, `mte_mke_enc_state_bytes_dc()`, `mte_mke_enc_state_bytes_dh()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_enc_state_bytes_c()`, `mte_mke_enc_state_bytes_dc()`, `mte_mke_enc_state_bytes_ch()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided cipher is not going to be used.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_enc_state_bytes_h()`, `mte_mke_enc_state_bytes_dh()`, `mte_mke_enc_state_bytes_ch()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided hash is not going to be used.

9.3.2.2 `mte_mke_enc_state_bytes_d` (C)

```
uint32_t mte_mke_enc_state_bytes_d(uint32_t tok_bytes, mte_verifiers verifiers,
mte_ciphers cipher, mte_hashes hash)
```

Returns the encoder state size for the given token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm. Returns `0` if the combination of arguments is not usable. Use this function when an MTE-provided DRBG is not going to be used and MTE-provided cipher and hash are going to be used. The associated initialization function is `mte_mke_enc_state_init_d()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_enc_state_bytes_c()`, `mte_mke_enc_state_bytes_dc()`, `mte_mke_enc_state_bytes_ch()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided cipher is not going to be used.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_enc_state_bytes_h()`, `mte_mke_enc_state_bytes_dh()`, `mte_mke_enc_state_bytes_ch()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided hash is not going to be used.

9.3.2.3 `mte_mke_enc_state_bytes_c` (C)

```
uint32_t mte_mke_enc_state_bytes_c(mte_drbgs drbg, uint32_t tok_bytes,
mte_verifiers verifiers, mte_hashes hash)
```

Returns the encoder state size for the given DRBG algorithm, token size in bytes, verifiers algorithm, and hash algorithm. Returns `0` if the combination of arguments is not usable. Use this function when an MTE-provided cipher is not going to be used and MTE-provided DRBG and hash are going to be used. The associated initialization function is `mte_mke_enc_state_init_c()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_enc_state_bytes_d()`, `mte_mke_enc_state_bytes_dc()`, `mte_mke_enc_state_bytes_dh()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_enc_state_bytes_h()`, `mte_mke_enc_state_bytes_dh()`, `mte_mke_enc_state_bytes_ch()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided hash is not going to be used.

9.3.2.4 `mte_mke_enc_state_bytes_h` (C)

```
uint32_t mte_mke_enc_state_bytes_h(mte_drbgs drbg, uint32_t tok_bytes,
mte_verifiers verifiers, mte_ciphers cipher)
```

Returns the encoder state size for the given DRBG algorithm, token size in bytes, verifiers algorithm, and cipher algorithm. Returns `0` if the combination of arguments is not usable. Use this function when an MTE-provided hash is not going to be used and MTE-provided DRBG and cipher are going to be used. The associated initialization function is `mte_mke_enc_state_init_h()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_enc_state_bytes_d()`, `mte_mke_enc_state_bytes_dc()`, `mte_mke_enc_state_bytes_dh()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_enc_state_bytes_c()`, `mte_mke_enc_state_bytes_dc()`, `mte_mke_enc_state_bytes_ch()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided cipher is

not going to be used.

9.3.2.5 `mte_mke_enc_state_bytes_dc` (C)

```
uint32_t mte_mke_enc_state_bytes_dc(uint32_t tok_bytes, mte_verifiers verifiers,
mte_hashes hash)
```

Returns the encoder state size for the given token size in bytes, verifiers algorithm, and hash algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided DRBG and cipher are not going to be used and an MTE-provided hash is going to be used. The associated initialization function is `mte_mke_enc_state_init_dc()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_enc_state_bytes_h()`, `mte_mke_enc_state_bytes_dh()`, `mte_mke_enc_state_bytes_ch()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided hash is not going to be used.

9.3.2.6 `mte_mke_enc_state_bytes_dh` (C)

```
uint32_t mte_mke_enc_state_bytes_dh(uint32_t tok_bytes, mte_verifiers verifiers,
mte_ciphers cipher)
```

Returns the encoder state size for the given token size in bytes, verifiers algorithm, and cipher algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided DRBG and hash are not going to be used and an MTE-provided cipher is going to be used. The associated initialization function is `mte_mke_enc_state_init_dh()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_enc_state_bytes_c()`, `mte_mke_enc_state_bytes_dc()`, `mte_mke_enc_state_bytes_ch()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided cipher is not going to be used.

9.3.2.7 `mte_mke_enc_state_bytes_ch` (C)

```
uint32_t mte_mke_enc_state_bytes_ch(mte_drbgs drbg, uint32_t tok_bytes,
mte_verifiers verifiers)
```

Returns the encoder state size for the given DRBG algorithm, token size in bytes, and verifiers algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided cipher and hash are not going to be used and an MTE-provided DRBG is going to be used. The associated initialization function is `mte_mke_enc_state_init_ch()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_enc_state_bytes_d()`, `mte_mke_enc_state_bytes_dc()`, `mte_mke_enc_state_bytes_dh()`, or `mte_mke_enc_state_bytes_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.

9.3.2.8 `mte_mke_enc_state_bytes_dch` (C)

```
uint32_t mte_mke_enc_state_bytes_dch(uint32_t tok_bytes, mte_verifiers verifiers)
```

Returns the encoder state size for the given token size in bytes and verifiers algorithm. Returns `0` if the combination of arguments is not usable. Use this function when MTE-provided DRBG, cipher, and hash are not going to be used. The associated initialization function is `mte_mke_enc_state_init_dch()`.

Buildtime options: the arguments must match the buildtime-chosen options.

`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.

9.3.2.9 `mte_mke_enc_state_init` (C)

```
mte_status mte_mke_enc_state_init(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, mte_ciphers cipher, mte_hashes hash)
```

Initialize the encoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm. Returns the status. Use this function when MTE-provided DRBG, cipher, and hash are going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_mke_enc_state_bytes()`.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_enc_state_init_d()`, `mte_mke_enc_state_init_dc()`, `mte_mke_enc_state_init_dh()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_enc_state_init_c()`, `mte_mke_enc_state_init_dc()`, `mte_mke_enc_state_init_ch()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided cipher is not going to be used.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_enc_state_init_h()`, `mte_mke_enc_state_init_dh()`, `mte_mke_enc_state_init_ch()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided hash is not going to be used.

9.3.2.10 `mte_mke_enc_state_init_d` (C)

```
mte_status mte_mke_enc_state_init_d(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, mte_ciphers cipher, mte_hashes hash)
```

Initialize the encoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm. Returns the status. Use this function when an MTE-provided DRBG is not going to be used and MTE-provided cipher and hash are going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_mke_enc_state_bytes_d()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.

`g_cb`: DRBG generate callback.
`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_enc_state_init_c()`, `mte_mke_enc_state_init_dc()`, `mte_mke_enc_state_init_ch()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided cipher is not going to be used.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_enc_state_init_h()`, `mte_mke_enc_state_init_dh()`, `mte_mke_enc_state_init_ch()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided hash is not going to be used.

9.3.2.11 `mte_mke_enc_state_init_c` (C)

```
mte_status mte_mke_enc_state_init_c(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, mte_cipher_key cek_cb, mte_cipher_enc_enc
ce_cb, mte_cipher_uninit cu_cb, uint32_t key_bytes, uint32_t block_bytes, void
*cipher_state, mte_hashes hash)
```

Initialize the encoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher callbacks, cipher key/block size, cipher state pointer, and hash algorithm. Returns the status. Use this function when an MTE-provided cipher is not going to be used and MTE-provided DRBG and hash are going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_mke_enc_state_bytes_c()`.

The `cipher_state` must point at the external cipher state and must remain in scope as long as the state will be used.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, and `hash` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_enc_state_init_d()`, `mte_mke_enc_state_init_dc()`, `mte_mke_enc_state_init_dh()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cek_cb`: cipher encrypt key callback.
`ce_cb`: cipher encrypt callback.
`cu_cb`: cipher uninit callback.
`key_bytes`: cipher key size in bytes.
`block_bytes`: cipher block size in bytes.

`cipher_state`: cipher state buffer.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_enc_state_init_h()`, `mte_mke_enc_state_init_dh()`, `mte_mke_enc_state_init_ch()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided hash is not going to be used.

9.3.2.12 `mte_mke_enc_state_init_h` (C)

```
mte_status mte_mke_enc_state_init_h(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, mte_ciphers cipher, mte_hash_calc hc_cb,
mte_hash_start hs_cb, mte_hash_feed hf_cb, mte_hash_finish hfin_cb, uint32_t
digest_bytes, void *hash_state)
```

Initialize the encoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash callbacks, hash digest size, and hash state pointer. Returns the status. Use this function when an MTE-provided hash is not going to be used and MTE-provided DRBG and cipher are going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_mke_enc_state_bytes_h()`.

The `hash_state` must point at the external hash state and must remain in scope as long as the state will be used.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, and `cipher` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_enc_state_init_d()`, `mte_mke_enc_state_init_dc()`, `mte_mke_enc_state_init_dh()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_enc_state_init_c()`, `mte_mke_enc_state_init_dc()`, `mte_mke_enc_state_init_ch()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided cipher is not going to be used.
`hc_cb`: hash calculate callback.
`hs_cb`: hash incremental start callback.
`hf_cb`: hash incremental feed callback.
`hfin_cb`: hash incremental finish callback.
`digest_bytes`: hash digest size in bytes.
`hash_state`: hash state buffer.

9.3.2.13 `mte_mke_enc_state_init_dc` (C)

```
mte_status mte_mke_enc_state_init_dc(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, mte_cipher_key cek_cb, mte_cipher_enc_enc ce_cb, mte_cipher_uninit
cu_cb, uint32_t key_bytes, uint32_t block_bytes, void *cipher_state, mte_hashes
hash)
```

Initialize the encoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, cipher callbacks, cipher key/block size, cipher state pointer, and hash algorithm. Returns the status. Use this function when MTE-provided DRBG and cipher are not going to be used and an MTE-provided hash is going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_mke_enc_state_bytes_dc()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

The `cipher_state` must point at the external cipher state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes`, `verifiers`, and `hash` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.
`g_cb`: DRBG generate callback.
`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cek_cb`: cipher encrypt key callback.
`ce_cb`: cipher encrypt callback.
`cu_cb`: cipher uninit callback.
`key_bytes`: cipher key size in bytes.
`block_bytes`: cipher block size in bytes.
`cipher_state`: cipher state buffer.

`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`. Use `mte_mke_enc_state_init_h()`, `mte_mke_enc_state_init_dh()`, `mte_mke_enc_state_init_ch()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided hash is not going to be used.

9.3.2.14 `mte_mke_enc_state_init_dh` (C)

```
mte_status mte_mke_enc_state_init_dh(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, mte_ciphers cipher, mte_hash_calc hc_cb, mte_hash_start hs_cb,
mte_hash_feed hf_cb, mte_hash_finish hfin_cb, uint32_t digest_bytes, void
*hash_state)
```

Initialize the encoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, cipher algorithm, hash callbacks, hash digest size, and hash state pointer. Returns the status. Use this function when MTE-provided DRBG and hash are not going to be used and an MTE-provided cipher is going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_mke_enc_state_bytes_dh()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

The `hash_state` must point at the external hash state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes`, `verifiers`, and `cipher` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.
`g_cb`: DRBG generate callback.
`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`. Use `mte_mke_enc_state_init_c()`, `mte_mke_enc_state_init_dc()`, `mte_mke_enc_state_init_ch()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided cipher is not going to be used.

`hc_cb`: hash calculate callback.
`hs_cb`: hash incremental start callback.
`hf_cb`: hash incremental feed callback.
`hfin_cb`: hash incremental finish callback.
`digest_bytes`: hash digest size in bytes.
`hash_state`: hash state buffer.

9.3.2.15 `mte_mke_enc_state_init_ch` (C)

```
mte_status mte_mke_enc_state_init_ch(MTE_HANDLE state, mte_drbgs drbg, uint32_t
tok_bytes, mte_verifiers verifiers, mte_cipher_key cek_cb, mte_cipher_enc_enc
ce_cb, mte_cipher_uninit cu_cb, uint32_t key_bytes, uint32_t block_bytes, void
*cipher_state, mte_hash_calc hc_cb, mte_hash_start hs_cb, mte_hash_feed hf_cb,
mte_hash_finish hfin_cb, uint32_t digest_bytes, void *hash_state)
```

Initialize the encoder state given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher callbacks, cipher key/block size, cipher state pointer, hash callbacks, hash digest size, and hash state pointer. Returns the status. Use this function when MTE-provided cipher and hash are not going to be used and an MTE-provided DRBG is going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_mke_enc_state_bytes_ch()`.

The `cipher_state` must point at the external cipher state and must remain in scope as long as the state will be used.

The `hash_state` must point at the external hash state and must remain in scope as long as the state will be used.

Buildtime options: the `drbg`, `tok_bytes`, and `verifiers` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`. Use `mte_mke_enc_state_init_d()`, `mte_mke_enc_state_init_dc()`, `mte_mke_enc_state_init_dh()`, or `mte_mke_enc_state_init_dch()` if an MTE-provided DRBG is not going to be used.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cek_cb`: cipher encrypt key callback.
`ce_cb`: cipher encrypt callback.
`cu_cb`: cipher uninit callback.
`key_bytes`: cipher key size in bytes.
`block_bytes`: cipher block size in bytes.
`cipher_state`: cipher state buffer.

`hc_cb`: hash calculate callback.
`hs_cb`: hash incremental start callback.
`hf_cb`: hash incremental feed callback.
`hfin_cb`: hash incremental finish callback.
`digest_bytes`: hash digest size in bytes.
`hash_state`: hash state buffer.

9.3.2.16 `mte_mke_enc_state_init_dch` (C)

```
mte_status mte_mke_enc_state_init_dch(MTE_HANDLE state, mte_drbg_instantiate i_cb,
mte_drbg_reseed_counter rc_cb, mte_drbg_state_save_bytes ssb_cb,
mte_drbg_state_save ss_cb, mte_drbg_state_restore sr_cb, mte_drbg_generate g_cb,
mte_drbg_uninstantiate u_cb, void *drbg_state, uint32_t tok_bytes, mte_verifiers
verifiers, mte_cipher_key cek_cb, mte_cipher_enc_enc ce_cb, mte_cipher_uninit
cu_cb, uint32_t key_bytes, uint32_t block_bytes, void *cipher_state, mte_hash_calc
hc_cb, mte_hash_start hs_cb, mte_hash_feed hf_cb, mte_hash_finish hfin_cb, uint32_t
digest_bytes, void *hash_state)
```

Initialize the encoder state given the DRBG callbacks, DRBG state pointer, token size in bytes, verifiers algorithm, cipher callbacks, cipher key/block size, cipher state pointer, hash callbacks, hash digest size, and hash state pointer. Returns the status. Use this function when MTE-provided DRBG, cipher, and hash are not going to be used.

The state buffer must be of sufficient length to hold the encoder state. See `mte_mke_enc_state_bytes_dch()`.

The `drbg_state` must point at the external DRBG state and must remain in scope as long as the state will be used.

The `cipher_state` must point at the external cipher state and must remain in scope as long as the state will be used.

The `hash_state` must point at the external hash state and must remain in scope as long as the state will be used.

Buildtime options: the `tok_bytes` and `verifiers` arguments must match the buildtime-chosen options.

`state`: handle to the state to initialize.
`i_cb`: DRBG instantiate callback.
`rc_cb`: DRBG reseed callback. Can be `NULL` if reseed information will not be requested.
`ssb_cb`: DRBG state save bytes callback. Can be `NULL` if state save/restore will not be used.
`ss_cb`: DRBG state save callback. Can be `NULL` if state save will not be used.
`sr_cb`: DRBG state restore callback. Can be `NULL` if state restore will not be used.
`g_cb`: DRBG generate callback.

`u_cb`: DRBG uninstantiate callback.
`drbg_state`: DRBG state buffer.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cek_cb`: cipher encrypt key callback.
`ce_cb`: cipher encrypt callback.
`cu_cb`: cipher uninit callback.
`key_bytes`: cipher key size in bytes.
`block_bytes`: cipher block size in bytes.
`cipher_state`: cipher state buffer.
`hc_cb`: hash calculate callback.
`hs_cb`: hash incremental start callback.
`hf_cb`: hash incremental feed callback.
`hfin_cb`: hash incremental finish callback.
`digest_bytes`: hash digest size in bytes.
`hash_state`: hash state buffer.

9.3.2.17 `mte_mke_enc_instantiate` (C)

```
mte_status mte_mke_enc_instantiate(MTE_HANDLE state, mte_drbg_get_entropy_input
ei_cb, void *ei_cb_context, mte_drbg_get_nonce n_cb, void *n_cb_context, const void
*ps, uint32_t ps_bytes)
```

Instantiate the encoder given the entropy input callback/context, nonce callback/context, personalization string, and length of the personalization string in bytes. Returns the status.

`state`: handle to the encoder state.
`ei_cb`: entropy input callback.
`ei_cb_context`: pointer that will be passed through to the `ei_cb` as the context argument.
`n_cb`: nonce callback.
`n_cb_context`: pointer that will be passed through to the `n_cb` as the context argument.
`ps`: personalization string.
`ps_bytes`: length of the personalization string in bytes.

9.3.2.18 `mte_mke_enc_reseed_counter` (C)

```
uint64_t mte_mke_enc_reseed_counter(MTE_CHANDLE state)
```

Returns the encoder's DRBG reseed counter.

`state`: handle to the encoder state.

9.3.2.19 `mte_mke_enc_save_bytes` (C)

```
uint32_t mte_mke_enc_save_bytes(MTE_CHANDLE state)
```

Returns the state save size for raw state. The associated state save function is `mte_mke_enc_state_save()`.

`state`: handle to the encoder state.

9.3.2.20 `mte_mke_enc_save_bytes_b64` (C)

```
uint32_t mte_mke_enc_save_bytes_b64(MTE_CHANDLE state)
```

Returns the state save size for Base64-encoded state. The associated state save function is `mte_mke_enc_state_save_b64()`.

`state`: handle to the encoder state.

9.3.2.21 `mte_mke_enc_state_save` (C)

```
mte_status mte_mke_enc_state_save(MTE_CHANDLE state, void *saved)
```

Saves the encoder state to the given buffer in raw form. The size of the buffer must be `mte_mke_enc_save_bytes()` and that is the length of the raw saved state. Returns the status.

`state`: handle to the encoder state.
`saved`: the buffer to save the state to.

9.3.2.22 `mte_mke_enc_state_save_b64` (C)

```
mte_status mte_mke_enc_state_save_b64(MTE_CHANDLE state, void *saved)
```

Saves the encoder state to the given buffer in Base64-encoded form. The size of the buffer must be `mte_mke_enc_save_bytes_b64()` and the encoded saved state is null-terminated. Returns the status.

`state`: handle to the encoder state.
`saved`: the buffer to save the state to.

9.3.2.23 `mte_mke_enc_state_restore` (C)

```
mte_status mte_mke_enc_state_restore(MTE_HANDLE state, const void *saved)
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`state`: handle to the encoder state.
`saved`: the buffer to restore the state from.

9.3.2.24 `mte_mke_enc_state_restore_b64` (C)

```
mte_status mte_mke_enc_state_restore_b64(MTE_HANDLE state, void *saved)
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`state`: handle to the encoder state.
`saved`: the buffer to restore the state from. The Base64 is overwritten with the decoded form.

9.3.2.25 `mte_mke_enc_buff_bytes` (C)

```
uint32_t mte_mke_enc_buff_bytes(MTE_CHANDLE state, uint32_t data_bytes)
```

Returns the encode buffer size in bytes given the data length in bytes for raw form encoding. The encode buffer provided to `mte_mke_enc_encode()` must be of at least this length.

`state`: handle to the encoder state.
`data_bytes`: the length of the data to be encoded (in bytes).

9.3.2.26 `mte_mke_enc_buff_bytes_b64` (C)

```
uint32_t mte_mke_enc_buff_bytes_b64(MTE_CHANDLE state, uint32_t data_bytes)
```

Returns the encode buffer size in bytes given the data length in bytes for Base64-encoded form encoding. The encode buffer provided to `mte_mke_enc_encode_b64()` must be of at least this length.

`state`: handle to the encoder state.
`data_bytes`: the length of the data to be encoded (in bytes).

9.3.2.27 `mte_mke_enc_encode` (C)

```
mte_status mte_mke_enc_encode(MTE_HANDLE state, mte_verifier_get_timestamp64 t_cb,
void *t_cb_context, const void *data, uint32_t data_bytes, void *encoded, uint32_t
*encoded_off, uint32_t *encoded_bytes)
```

Encode the given data of the given byte length to the given encode buffer in raw form. Returns the status. Sets `*encoded_off` to the offset in the `encoded` buffer where the encoded version can be found and sets `*encoded_bytes` to the encoded version length in bytes. The encoded version is valid only if `mte_status_success` is returned.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

The `encoded` buffer must be of sufficient length to hold the encoded version. See `mte_mke_enc_buff_bytes()`.

`state`: handle to the encoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`data`: the data to encode. It must contain only byte values allowed by the input range.
`data_bytes`: the length of the data to be encoded (in bytes).
`encoded`: the buffer to write the encoded version of the input to.
`encoded_off`: pointer to the offset of the encoded version.
`encoded_bytes`: pointer to the length of the encoded version.

9.3.2.28 `mte_mke_enc_encode_b64` (C)

```
mte_status mte_mke_enc_encode_b64(MTE_HANDLE state, mte_verifier_get_timestamp64
t_cb, void *t_cb_context, const void *data, uint32_t data_bytes, void *encoded,
uint32_t *encoded_off, uint32_t *encoded_bytes)
```

Encode the given data of the given byte length to the given encode buffer in Base64-encoded form. Returns the status. Sets `*encoded_off` to the offset in the `encoded` buffer where the Base64 version can be found and sets `*encoded_bytes` to the encoded version length in bytes. The encoded version is null terminated, but `*encoded_bytes` excludes the null terminator. The encoded version is valid only if `mte_status_success` is returned.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

The `encoded` buffer must be of sufficient length to hold the encoded version. See `mte_mke_enc_buff_bytes_b64()`.

`state`: handle to the encoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`data`: the data to encode.
`data_bytes`: the length of the data to be encoded (in bytes).
`encoded`: the buffer to write the encoded version of the input to.
`encoded_off`: pointer to the offset of the encoded version.
`encoded_bytes`: pointer to the length of the encoded version.

9.3.2.29 `mte_mke_enc_encrypt_finish_bytes` (C)

```
uint32_t mte_mke_enc_encrypt_finish_bytes(MTE_CHANDLE state)
```

Returns the length of the result `mte_mke_enc_encrypt_finish()` will produce. Use this if you need to know that size before you can call it.

`state`: handle to the encoder state.

9.3.2.30 `mte_mke_enc_encrypt_state_bytes` (C)

```
uint32_t mte_mke_enc_encrypt_state_bytes(MTE_CHANDLE state)
```

Returns the chunk-based encryption state size in bytes. The `c_state` buffer provided to the `mte_mke_enc_encrypt_start()`, `mte_mke_enc_encrypt_chunk()`, and `mte_mke_enc_encrypt_finish()` functions must be of at least this length.

`state`: handle to the encoder state.

9.3.2.31 `mte_mke_enc_encrypt_start` (C)

```
mte_status mte_mke_enc_encrypt_start(MTE_HANDLE state, MTE_HANDLE c_state)
```

Start a chunk-based encryption session. Initializes the state to the `c_state` buffer. This state is used by the other chunk-based functions. Returns the status.

The `c_state` buffer must be of sufficient length. See `mte_mke_enc_encrypt_state_bytes()`.

`state`: handle to the encoder state.
`c_state`: handle to the chunk-based encryption session state.

9.3.2.32 `mte_mke_enc_encrypt_chunk` (C)

```
mte_status mte_mke_enc_encrypt_chunk(MTE_HANDLE state, MTE_HANDLE c_state, const
void *data, uint32_t data_bytes, void *encrypted)
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` of length `data_bytes` is used as input and the encrypted data is written to the `encrypted` buffer. The `data_bytes` argument must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

The `encrypted` buffer must be at least `data_bytes` in length to hold the encrypted version. The `encrypted` buffer may be the same as the `data` buffer to overwrite the data with the encrypted version. The encrypted version length is `data_bytes`.

`state`: handle to the encoder state.
`c_state`: handle to the chunk-based encryption session state.
`data`: the data to encrypt.
`data_bytes`: the length of the data to be encrypted (in bytes).
`encrypted`: the buffer to write the encrypted data to.

9.3.2.33 `mte_mke_enc_encrypt_finish` (C)

```
mte_status mte_mke_enc_encrypt_finish(MTE_HANDLE state, MTE_HANDLE c_state,
mte_verifier_get_timestamp64 t_cb, void *t_cb_context, uint32_t *result_off,
uint32_t *result_bytes)
```

Finish the chunk-based encryption session. Returns the status. Writes the tokenized hash to the `c_state` buffer and sets `*result_off` to the offset of the tokenized hash in the `c_state` buffer and sets `*result_bytes` to the length of the tokenized hash. The tokenized hash must be appended to the encrypted data so that both the encrypted data and tokenized hash are sent to the chunk-based decryption session.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

The `buff` buffer must be of sufficient length. See `mte_mke_enc_encrypt_buff_bytes()`.

`state`: handle to the encoder state.
`c_state`: handle to the chunk-based encryption session state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`result_off`: pointer to the offset of the tokenized hash.
`result_bytes`: pointer to the length of the tokenized hash.

9.3.2.34 `mte_mke_enc_uninstantiate` (C)

```
mte_status mte_mke_enc_uninstantiate(MTE_HANDLE state)
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The state must not be used until instantiated again.

`state`: handle to the encoder state.

10 C Support API

10.1 Overview

10.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use `mte_supp_mke_enc_alloc()` to allocate the encoder and `mte_mke_enc_instantiate()` to instantiate it.

To encode, use `mte_supp_mke_enc_encode()` or `mte_supp_mke_enc_encode_b64()`. Use `mte_supp_buff_free()` to free the encoded version after use.

To destroy, use `mte_supp_mke_enc_free()`.

10.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and destroy it.

To create, use `mte_supp_mke_dec_alloc()` to allocate the decoder and `mte_mke_dec_instantiate()` to instantiate it.

To decode, use `mte_supp_mke_dec_decode()` or `mte_supp_mke_dec_decode_b64()`. Use `mte_supp_buff_free()` to free the decoded version after use.

To get additional information, use `mte_mke_dec_enc_ts()`, `mte_mke_dec_dec_ts()`, and `mte_mke_dec_msg_skipped()`, depending on which verifiers are enabled.

To destroy, use `mte_supp_mke_dec_free()`.

10.1.3 Chunk Encrypt

Other than for creating and destroying the encoder, the Basic C API must be used for chunk encrypt since it is designed for efficiency with large data.

10.1.4 Chunk Decrypt

Other than for creating and destroying the decoder, the Basic C API must be used for chunk decrypt since it is designed for efficiency with large data.

10.2 Files

The MTE Fixed-Length Add-On uses the core header files documented in the MTE Core Developer's Guide, as well as the following:

File	Description
mte_supp_mke.h	This defines the Managed-Key Encryption Support API.

10.3 Functions

10.3.1 Encoder/Decoder Support

Support functions are declared in `mte_supp_mke.h`.

10.3.1.1 `mte_supp_mke_enc_alloc` (C)

```
MTE_HANDLE mte_supp_mke_enc_alloc(mte_drbgs drbg, uint32_t tok_bytes, mte_verifiers
verifiers, mte_ciphers cipher, mte_hashes hash, uint32_t min_encr_bytes)
```

Create an encoder given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, and minimum size to encrypt. Returns a handle to the encoder on success or `NULL` on failure. You must call `mte_supp_mke_enc_free()` when you are done with the encoder.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`.
`min_to_encrypt`: the minimum size to encrypt. Inputs smaller than this will be tokenized instead of encrypted.

10.3.1.2 `mte_supp_mke_enc_state_save` (C)

```
void *mte_supp_mke_enc_state_save(MTE_CHANDLE encoder, uint32_t *bytes)
```

Saves the encoder state to the given buffer in raw form. Returns `NULL` on failure; otherwise returns the saved state and sets `*bytes` to the length of the saved state. You must call `mte_supp_buff_free()` when you are done with the saved state.

`encoder`: handle to the encoder state.
`bytes`: pointer to the length of the saved state.

10.3.1.3 `mte_supp_mke_enc_state_save_b64` (C)

```
char *mte_mke_enc_state_save_b64(MTE_CHANDLE encoder, uint32_t *bytes)
```

Saves the encoder state to the given buffer in Base64-encoded form. Returns `NULL` on failure; otherwise returns the saved state and sets `*bytes` to the length of the saved state. You must call `mte_supp_buff_free()` when you are done with the saved state.

`encoder`: handle to the encoder state.
`bytes`: pointer to the length of the saved state.

10.3.1.4 `mte_supp_mke_enc_encode` (C)

```
void *mte_supp_mke_enc_encode(MTE_HANDLE encoder, mte_verifier_get_timestamp64
t_cb, void *t_cb_context, const void *data, uint32_t data_bytes, uint32_t
*encoded_bytes, mte_status *status)
```

Encode the given data of the given byte length in raw form. Returns the encoded version on success or `NULL` on failure. Sets `*encoded_bytes` to the encoded version length in bytes. Sets `*status` to the status. The encoded version is valid only if `*status == mte_status_success`. You must call `mte_supp_buff_free()` when you are done with the encoded version.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

`encoder`: handle to the encoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`data`: the data to encode. It must contain only byte values allowed by the input range.
`data_bytes`: the length of the data to be encoded (in bytes).
`encoded_bytes`: pointer to the length of the encoded version.
`status`: pointer to the status.

10.3.1.5 `mte_supp_mke_enc_encode_b64` (C)

```
char *mte_supp_mke_enc_encode_b64(MTE_HANDLE encoder, mte_verifier_get_timestamp64
t_cb, void *t_cb_context, const void *data, uint32_t data_bytes, uint32_t
*encoded_bytes, mte_status *status)
```

Encode the given data of the given byte length in Base64-encoded form. Returns the encoded version on success or `NULL` on failure. Sets `*encoded_bytes` to the encoded version length in bytes. Sets `*status` to the status. The encoded version is valid only if `*status == mte_status_success`. You must call `mte_supp_buff_free()` when you are done with the encoded version.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

`encoder`: handle to the encoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`data`: the data to encode.
`data_bytes`: the length of the data to be encoded (in bytes).
`encoded_bytes`: pointer to the length of the encoded version.
`status`: pointer to the status.

10.3.1.6 `mte_supp_mke_enc_free` (C)

```
mte_status mte_supp_mke_enc_free(MTE_HANDLE encoder)
```

Free an encoder allocated with `mte_supp_mke_enc_alloc()`. Uninstantiates the encoder before freeing, so the encoder's DRBG state is zeroized. Returns the status.

`encoder`: handle to the encoder state.

10.3.1.7 `mte_supp_mke_dec_alloc` (C)

```
MTE_HANDLE mte_supp_mke_dec_alloc(mte_drbgs drbg, uint32_t tok_bytes, mte_verifiers
verifiers, mte_ciphers cipher, mte_hashes hash, uint64_t t_window, int32_t
s_window)
```

Create a decoder given the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window. Returns a handle to the decoder on success or NULL on failure. You must call `mte_supp_mke_dec_free()` when you are done with the decoder.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

10.3.1.8 `mte_supp_mke_dec_state_save` (C)

```
void *mte_supp_mke_dec_state_save(MTE_CHANDLE decoder, uint32_t *bytes)
```

Saves the decoder state to the given buffer in raw form. Returns `NULL` on failure; otherwise returns the saved state and sets `*bytes` to the length of the saved state. You must call `mte_supp_buff_free()` when you are done with the saved state.

`decoder`: handle to the decoder state.
`bytes`: pointer to the length of the saved state.

10.3.1.9 `mte_supp_mke_dec_state_save_b64` (C)

```
char *mte_mke_dec_state_save_b64(MTE_CHANDLE decoder, uint32_t *bytes)
```

Saves the decoder state to the given buffer in Base64-encoded form. Returns `NULL` on failure; otherwise returns the saved state and sets `*bytes` to the length of the saved state. You must call `mte_supp_buff_free()` when you are done with the saved state.

`decoder`: handle to the decoder state.
`bytes`: pointer to the length of the saved state.

10.3.1.10 `mte_supp_mke_dec_decode` (C)

```
void *mte_supp_mke_dec_decode(MTE_HANDLE decoder, mte_verifier_get_timestamp64
t_cb, void *t_cb_context, const void *encoded, uint32_t encoded_bytes, uint32_t
*decoded_bytes, mte_status *status)
```

Decodes the given raw encoded data of the given byte length. Returns the decoded version on success or `NULL` on failure. Sets `*decoded_bytes` to the decoded data length in bytes. Sets `*status` to the status. The decoded version is valid only if `!mte_base_status_is_error(status)`. A null terminator is appended to the decoded data but is not included in `*decoded_bytes`. You must call `mte_supp_buff_free()` when you are done with the decoded version.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

`decoder`: handle to the decoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`encoded`: the encoded data to decode.
`encoded_bytes`: the length of the encoded data to be decoded (in bytes).
`decoded_bytes`: pointer to the length of the decoded version.
`status`: pointer to the status.

10.3.1.11 `mte_supp_mke_dec_decode_b64` (C)

```
void *mte_supp_mke_dec_decode_b64(MTE_HANDLE decoder, mte_verifier_get_timestamp64
t_cb, void *t_cb_context, const void *encoded, uint32_t encoded_bytes, uint32_t
*decoded_bytes, mte_status *status)
```

Decodes the given Base64-encoded encoded data of the given byte length. Returns the decoded version on success or `NULL` on failure. Sets `*decoded_bytes` to the decoded data length in bytes. Sets `*status` to the status. The decoded version is valid only if `!mte_base_status_is_error(status)`. A null terminator is appended to the decoded data but is not included in `*decoded_bytes`. You must call `mte_supp_buff_free()` when you are done with the decoded version.

The timestamp callback and context are used to obtain the timestamp if the timestamp verifier is enabled.

`decoder`: handle to the decoder state.
`t_cb`: timestamp callback. If the timestamp verifier is not enabled, this can be `NULL`.
`t_cb_context`: pointer that will be passed through to the `t_cb` as the context argument.
`encoded`: the encoded data to decode.
`encoded_bytes`: the length of the encoded data to be decoded (in bytes).
`decoded_bytes`: pointer to the length of the decoded version.
`status`: pointer to the status.

10.3.1.12 `mte_supp_mke_dec_free` (C)

```
mte_status mte_supp_mke_dec_free(MTE_HANDLE decoder)
```

Frees a decoder allocated with `mte_supp_mke_dec_alloc()`. Uninstantiates the decoder before freeing, so the decoder's DRBG state is zeroized. Returns the status.

`decoder`: handle to the decoder state.

11 C++ Language Interface

11.1 Overview

11.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use an `MteMkeEnc` constructor to create an encoder object and one of the `MteMkeEnc::instantiate()` overloads to instantiate it.

To encode, use any of the `MteMkeEnc::encode()` or `MteMkeEnc::encodeB64()` overloads.

To destroy, cause the `MteMkeEnc` destructor to be invoked (e.g., by going out of scope if on the stack or `delete` if dynamically allocated).

11.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and destroy it.

To create, use an `MteMkeDec` constructor to create a decoder object and one of the `MteMkeDec::instantiate()` overloads to instantiate it.

To decode, use any of the `MteMkeDec::decode()` or `MteMkeDec::decodeB64()` overloads.

To get additional information, use `MteMkeDec::getEncTs()`, `MteMkeDec::getDecTs()`, and `MteMkeDec::getMsgSkipped()`, depending on which verifiers are enabled.

To destroy, cause the `MteMkeDec` destructor to be invoked (e.g., by going out of scope if on the stack or `delete` if dynamically allocated).

11.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `MteMkeEnc::startEncrypt()` to start a chunk session, `MteMkeEnc::encryptChunk()` repeatedly to encrypt each chunk, then `MteMkeEnc::finishEncrypt()` to finish the session.

11.1.4 Chunk Decrypt

The chunk encrypt follows the flow of a decoder, but instead of using a decode method, use `MteMkeDec::startDecrypt()` to start a chunk session, `MteMkeDec::decryptChunk()` repeatedly to decrypt each chunk, then `MteMkeEnc::finishDecrypt()` to finish the session.

11.2 Files

The MTE Managed-Key Encryption Add-On uses the core header files (other than `MteEnc.h` and `MteDec.h`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteMkeDec.h	The MteMkeDec class declaration.
MteMkeEnc.h	The MteMkeEnc class declaration.

The MTE Managed-Key Encryption Add-On uses the core source files (other than `MteEnc.cpp` and `MteDec.cpp`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteMkeDec.cpp	The MteMkeDec class definition.
MteMkeEnc.cpp	The MteMkeEnc class definition.

11.2.1 Classes

11.2.1.1 `MteMkeDec`

Defines decoder-specific methods.

11.2.1.2 `MteMkeEnc`

Defines encoder-specific methods.

11.3 Functions

11.3.1 Decoder

Decoder-specific functions are declared in `MteMkeDec.h`.

11.3.1.1 `MteMkeDec` Constructor (default options) (CPP)

```
MteMkeDec(uint64_t tWindow = 0, int32_t sWindow = 0)
```

Constructor using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase::ourDefault`* constants.

`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

11.3.1.2 `MteMkeDec` Constructor (runtime options) (CPP)

```
MteMkeDec(mte_drbgs drbg, size_t tokBytes, mte_verifiers verifiers, mte_ciphers
cipher, mte_hashes hash, uint64_t tWindow, int32_t sWindow)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`.
`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

11.3.1.3 `MteMkeDec` Destructor (CPP)

```
virtual ~MteMkeDec()
```

Destructor. The `uninstantiate()` method is called.

11.3.1.4 `MteMkeDec::instantiate(void *)` (CPP)

```
mte_status instantiate(const void *ps, size_t psBytes)
```

Instantiate the decoder given the personalization string, and length of the personalization string in bytes. The `entropyCallback()` and `nonceCallback()` virtual functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.
`psBytes`: length of the personalization string in bytes.

11.3.1.5 `MteMkeDec::instantiate(std::string)` (CPP)

```
mte_status instantiate(const std::string& ps)
```

Calls `instantiate(ps.data(), ps.size())`. For interoperability, it is assumed the string is UTF-8 encoded.

`ps`: personalization string.

11.3.1.6 `MteMkeDec::getReseedCounter` (CPP)

```
uint64_t getReseedCounter() const
```

Returns the decoder's DRBG reseed counter.

11.3.1.7 `MteMkeDec::saveState` (CPP)

```
const void *saveState(size_t& stateBytes)
```

Returns the saved state in raw form and sets `stateBytes` to the length of the saved state in bytes. Returns `NULL` on error.

`stateBytes`: the length of the saved state.

11.3.1.8 `MteMkeDec::saveStateB64` (CPP)

```
char *saveStateB64()
```

Returns the saved state in Base64-encoded form. Returns `NULL` on error.

11.3.1.9 `MteMkeDec::restoreState` (CPP)

```
mte_status restoreState(const void *saved)
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

11.3.1.10 `MteMkeDec::restoreStateB64` (CPP)

```
mte_status restoreStateB64(const char *saved)
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

11.3.1.11 `MteMkeDec::getBuffBytes` (CPP)

```
size_t getBuffBytes(const void *encoded, size_t encOff, size_t encBytes)
```

Returns the decode buffer size in bytes given the raw encoded message at the given offset and size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

11.3.1.12 `MteMkeDec::getBuffBytesB64` (CPP)

```
size_t getBuffBytesB64(const void *encoded, size_t encOff, size_t encBytes)
```

Returns the decode buffer size in bytes given the Base64-encoded message at the given offset and size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

11.3.1.13 `MteMkeDec::decode(void *)` (CPP)

```
void *decode(const void *encoded, size_t encodedBytes, size_t& decodedBytes,
mte_status& status)
```

Decode the given raw encoded data of the given byte length. Returns the decoded data and sets `decodedBytes` to the length of the decoded data in bytes and `status` to the status. The decoded version is valid only if `!statusIsError(status)`. A null terminator is appended to the decoded data but is not included in `decodedBytes`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`encodedBytes`: the length of the encoded data to be decoded (in bytes).
`decodedBytes`: the length of the decoded version.
`status`: the status.

11.3.1.14 `MteMkeDec::decodeB64(void *)` (CPP)

```
void *decodeB64(const char *encoded, size_t& decodedBytes, mte_status& status)
```

Decode the given Base64-encoded encoded data. Returns the decoded data and sets `decodedBytes` to the length of the decoded data in bytes and `status` to the status. The decoded version is valid only if `!statusIsError(status)`. A null terminator is appended to the decoded data but is not included in `decodedBytes`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`decodedBytes`: the length of the decoded version.
`status`: the status.

11.3.1.15 `MteMkeDec::decode` (`std::string`) (CPP)

```
mte_status decode(const void *encoded, size_t encodedBytes, std::string& str)
```

Decode the given raw encoded data of the given byte length. Sets `str` to the decoded data up to the length of decoding or first null byte, whichever comes first. Returns the `status`. The decoded version is valid only if `!statusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`encodedBytes`: the length of the encoded data to be decoded (in bytes).
`str`: the decoded string.

11.3.1.16 `MteMkeDec::decodeB64` (`std::string`) (CPP)

```
mte_status decodeB64(const char *encoded, std::string& str)
```

Decode the given Base64-encoded encoded data. Sets `str` to the decoded data up to the length of decoding or first null byte, whichever comes first. Returns the `status`. The decoded version is valid only if `!statusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`str`: the decoded string.

11.3.1.17 `MteMkeDec::decode` (external buffers) (CPP)

```
mte_status decode(const void *encoded, size_t encOff, size_t encBytes, void
*decoded, size_t& decOff, size_t& decBytes)
```

Decode the given raw `encoded` version of the given length at the given offset to the given buffer at the given offset. Returns the status. Adjusts `decOff` to the offset of the decoded version in the `decoded` buffer. Sets `decBytes` to the decoded length in bytes. The `decoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: buffer containing the encoded data.
`encOff`: offset to the start of the encoded data.
`encBytes`: length of the encoded data in bytes.
`decoded`: buffer to hold the decoded data.
`decOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the decoded data.
`decBytes`: set to the length of the decoded data in bytes.

11.3.1.18 `MteMkeDec::decodeB64` (external buffers) (CPP)

```
mte_status decodeB64(const void *encoded, size_t encOff, size_t encBytes, void
*decoded, size_t& decOff, size_t& decBytes)
```

Decode the given Base64 `encoded` version of the given length at the given offset to the given buffer at the given offset. Returns the status. Adjusts `decOff` to the offset of the decoded version in the `decoded` buffer. Sets `decBytes` to the decoded length in bytes. The `decoded` buffer must have sufficient length remaining after the offset. Use `GetBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: buffer containing the encoded data.
`encOff`: offset to the start of the encoded data.
`encBytes`: length of the encoded data in bytes.
`decoded`: buffer to hold the decoded data.
`decOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the decoded data.
`decBytes`: set to the length of the decoded data in bytes.

11.3.1.19 `MteMkeDec::startDecrypt` (CPP)

```
mte_status startDecrypt()
```

Start a chunk-based decryption session. Returns the status.

11.3.1.20 `MteMkeDec::decryptChunk` (CPP)

```
const void *decryptChunk(const void *encrypted, size_t encryptedBytes, size_t&
decryptedBytes)
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data of length `encryptedBytes` is used as input and `decryptedBytes` is set to the amount of decrypted data which is returned. The amount decrypted may be less than the input size. Returns `NULL` on error.

`encrypted`: the encrypted data to decrypt.
`encryptedBytes`: the length of the encrypted data to be decrypted (in bytes).
`decryptedBytes`: the length of data decrypted. This may be less than `encryptedBytes` due to cipher block boundaries and buffering for the tokenized hash.

11.3.1.21 `MteMkeDec::decryptChunk` (external buffers) (CPP)

```
size_t decryptChunk(const void *encrypted, size_t encOff, size_t encBytes, void
*decrypted, size_t decOff)
```

Decrypt a chunk of data starting at `encOff` and of length `encBytes` in the `encrypted` buffer in a chunk-based decryption session. Some decrypted data is written to the `decrypted` buffer starting at `decOff`. The amount decrypted may be less than the input size and is returned. Returns `ULONG_MAX` on error.

`encrypted`: buffer containing encrypted data to decrypt.
`encOff`: offset to the start of the encrypted data.
`encBytes`: length of the encrypted data in bytes.
`decrypted`: buffer to hold decrypted data.
`decOff`: offset to start writing decrypted data.

11.3.1.22 `MteMkeDec::finishDecrypt` (CPP)

```
const void *finishDecrypt(size_t& decryptedBytes, mte_status& status)
```

Finish the chunk-based decryption session. Returns the final decrypted data and sets `decryptedBytes` to the length of the final part of the decrypted data and `status` to the status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`decryptedBytes`: the length of data decrypted.
`status`: the status.

11.3.1.23 `MteMkeDec::getEncTs` (CPP)

```
uint64_t getEncTs() const
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

11.3.1.24 `MteMkeDec::getDecTs` (CPP)

```
uint64_t getDecTs() const
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

11.3.1.25 `MteMkeDec::getMsgSkipped` (CPP)

```
uint32_t getMsgSkipped() const
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

11.3.1.26 `MteMkeDec::uninstantiate` (CPP)

```
mte_status uninstantiate()
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

11.3.2 Encoder

Encoder-specific functions are declared in `MteMkeEnc.h`.

11.3.2.1 `MteMkeEnc` Constructor (default options) (CPP)

```
MteMkeEnc()
```

Constructor using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase::ourDefault`* constants.

11.3.2.2 `MteMkeEnc` Constructor (runtime options) (CPP)

```
MteMkeEnc(mte_drbgs drbg, size_t tokBytes, mte_verifiers verifiers, mte_ciphers
cipher, mte_hashes hash)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`.

11.3.2.3 `MteMkeEnc` Destructor (CPP)

```
virtual ~MteMkeEnc()
```

Destructor. The `uninstantiate()` method is called.

11.3.2.4 `MteMkeEnc::instantiate(void *)` (CPP)

```
mte_status instantiate(const void *ps, size_t psBytes)
```

Instantiate the encoder given the personalization string, and length of the personalization string in bytes. The `entropyCallback()` and `nonceCallback()` virtual functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.
`psBytes`: length of the personalization string in bytes.

11.3.2.5 `MteMkeEnc::instantiate(std::string)` (CPP)

```
mte_status instantiate(const std::string& ps)
```

Calls `instantiate(ps.data(), ps.size())`. For interoperability, it is assumed the string is UTF-8 encoded.

`ps`: personalization string.

11.3.2.6 `MteMkeEnc::getReseedCounter` (CPP)

```
uint64_t getReseedCounter() const
```

Returns the encoder's DRBG reseed counter.

11.3.2.7 `MteMkeEnc::saveState` (CPP)

```
const void *saveState(size_t& stateBytes)
```

Returns the saved state in raw form and sets `stateBytes` to the length of the saved state in bytes. Returns `NULL` on error.

`stateBytes`: the length of the saved state.

11.3.2.8 `MteMkeEnc::saveStateB64` (CPP)

```
char *saveStateB64()
```

Returns the saved state in Base64-encoded form. Returns `NULL` on error.

11.3.2.9 `MteMkeEnc::restoreState` (CPP)

```
mte_status restoreState(const void *saved)
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

11.3.2.10 `MteMkeEnc::restoreStateB64` (CPP)

```
mte_status restoreStateB64(const char *saved)
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

11.3.2.11 `MteMkeEnc::getBuffBytes` (CPP)

```
size_t getBuffBytes(size_t dataBytes) const
```

Returns the raw encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

11.3.2.12 `MteMkeEnc::getBuffBytesB64` (CPP)

```
size_t getBuffBytesB64(size_t dataBytes) const
```

Returns the Base64 encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

11.3.2.13 `MteMkeEnc::encode` (`void *`) (CPP)

```
const void *encode(const void *data, size_t dataBytes, size_t& encodedBytes,
mte_status& status)
```

Encode the given data of the given byte length in raw form. Returns the encoded version and sets `encodedBytes` to the length of the encoded data in bytes and `status` to the status. The encoded version is valid only if `status == mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`dataBytes`: the length of the data to be encoded (in bytes).
`encodedBytes`: the length of the encoded version.
`status`: the status.

11.3.2.14 `MteMkeEnc::encodeB64` (`void *`) (CPP)

```
const char *encodeB64(const void *data, size_t dataBytes, mte_status& status)
```

Encode the given data of the given byte length in Base64-encoded form. Returns the encoded version and sets `status` to the status. The encoded version is valid only if `status == mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`dataBytes`: the length of the data to be encoded (in bytes).
`status`: the status.

11.3.2.15 `MteMkeEnc::encode` (`std::string`) (CPP)

```
const void *encode(const std::string& str, size_t& encodedBytes, mte_status&
status)
```

Calls `encode(str.data(), str.size(), encodedBytes, status)`. For interoperability, it is assumed the string is UTF-8 encoded.

`str`: the string to encode.
`encodedBytes`: the length of the encoded version.
`status`: the status.

11.3.2.16 `MteMkeEnc::encodeB64` (`std::string`) (CPP)

```
const char *encodeB64(const std::string& str, mte_status& status)
```

Calls `encodeB64(str.data(), str.size(), status)`. For interoperability, it is assumed the string is UTF-8 encoded.

`str`: the string to encode.
`status`: the status.

11.3.2.17 `MteMkeEnc::encode` (external buffers) (CPP)

```
mte_status encode(const void *data, size_t dataOff, size_t dataBytes, void
*encoded, size_t& encOff, size_t& encBytes)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the status. Adjusts `encOff` to the offset of the encoded version in the `encoded` buffer. Sets `encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

11.3.2.18 `MteMkeEnc::encodeB64` (external buffers) (CPP)

```
mte_status encodeB64(const void *data, size_t dataOff, size_t dataBytes, void
*encoded, size_t& encOff, size_t& encBytes)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the status. Adjusts `encOff` to the offset of the encoded version in the `encoded` buffer. Sets `encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

11.3.2.19 `MteMkeEnc::encryptFinishBytes` (CPP)

```
size_t encryptFinishBytes() const
```

Returns the length of the result `finishEncrypt()` will produce. Use this if you need to know that size before you can call it.

11.3.2.20 `MteMkeEnc::startEncrypt` (CPP)

```
mte_status startEncrypt()
```

Start a chunk-based encryption session. Returns the status.

11.3.2.21 `MteMkeEnc::encryptChunk` (CPP)

```
mte_status encryptChunk(void *data, size_t bytes)
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` of length `bytes` is encrypted in place. The `bytes` argument must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: the data to encrypt.
`bytes`: the length of the data to be encrypted (in bytes).

11.3.2.22 `MteMkeEnc::finishEncrypt` (CPP)

```
const void *finishEncrypt(size_t& resultBytes, mte_status& status)
```

Finish the chunk-based encryption session. Returns the tokenized hash and sets `resultBytes` to the length of the tokenized hash and `status` to the status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`resultBytes`: the length of the tokenized hash.
`status`: the status.

11.3.2.23 `MteMkeEnc::uninstantiate` (CPP)

```
mte_status uninstantiate()
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

12 C# Language Interface

12.1 Overview

12.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and zeroize it.

To create, use an `MteMkeEnc` constructor to create an encoder object and one of the `MteMkeEnc.Instantiate()` overloads to instantiate it.

To encode, use any of the `MteMkeEnc.Encode()` or `MteMkeEnc.EncodeB64()` overloads.

To zeroize, call `MteMkeEnc.Uninstantiate()`. The destructor will also do this, but C# cannot guarantee when that will happen, if at all, so for security it should be done explicitly.

12.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and zeroize it.

To create, use an `MteMkeDec` constructor to create a decoder object and one of the `MteMkeDec.Instantiate()` overloads to instantiate it.

To decode, use `MteMkeDec.Decode()`, `MteMkeDec.DecodeB64()`, `MteMkeDec.DecodeStr()`, or `MteMkeDec.DecodeStrB64()`.

To get additional information, use `MteMkeDec.GetEncTs()`, `MteMkeDec.GetDecTs()`, and `MteMkeDec.GetMsgSkipped()`, depending on which verifiers are enabled.

To zeroize, call `MteMkeDec.Uninstantiate()`. The destructor will also do this, but C# cannot guarantee when that will happen, if at all, so for security it should be done explicitly.

12.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `MteMkeEnc.StartEncrypt()` to start a chunk session, `MteMkeEnc.EncryptChunk()` repeatedly to encrypt each chunk, then `MteMkeEnc.FinishEncrypt()` to finish the session.

12.1.4 Chunk Decrypt

The chunk encrypt follows the flow of a decoder, but instead of using a decode method, use `MteMkeDec.StartDecrypt()` to start a chunk session, `MteMkeDec.DecryptChunk()` repeatedly to decrypt each chunk, then `MteMkeEnc.FinishDecrypt()` to finish the session.

12.2 Files

The MTE Managed-Key Encryption Add-On uses the core source files (other than `MteEnc.cs` and `MteDec.cs`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
IMteMkeDec.cs	Interface of the MteMkeDec class.
IMteMkeEnc.cs	Interface of the MteMkeEnc class.
MteDelegatesMkeDec.cs	The delegates for MteMkeDec.
MteDelegatesMkeEnc.cs	The delegates for MteMkeEnc.
MteInteropMkeDec.cs	The MteInterop partial class for MteMkeDec.
MteInteropMkeEnc.cs	The MteInterop partial class for MteMkeEnc.
MteMkeDec.cs	The MteMkeDec class.
MteMkeEnc.cs	The MteMkeEnc class.

12.3 Types

12.3.1 Classes

12.3.1.1 `MteMkeDec`

Defines decoder-specific methods.

12.3.1.2 `MteMkeEnc`

Defines encoder-specific methods.

12.4 Functions

12.4.1 Decoder

Decoder-specific functions are declared in `MteMkeDec.cs`.

12.4.1.1 `MteMkeDec` Constructor (default options) (CS)

```
public MteMkeDec(UInt64 tWindow = 0, Int32 sWindow = 0)
```

Constructor using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants. The default interop is used.

`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

12.4.1.2 `MteMkeDec` Constructor (interop, default options) (CS)

```
public MteMkeDec(MteInterop interop, UInt64 tWindow = 0, Int32 sWindow = 0)
```

Constructor using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants. The provided interop is used.

`interop`: the interop object to use.
`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

12.4.1.3 `MteMkeDec` Constructor (runtime options) (CS)

```
public MteMkeDec(MteDrbgs drbg, int tokBytes, MteVerifiers verifiers, MteCiphers
cipher, MteHashes hash, UInt64 tWindow, Int32 sWindow)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window. The default interop is used.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.
`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

12.4.1.4 `MteMkeDec` Constructor (interop, runtime options) (CS)

```
public MteMkeDec(MteInterop interop, MteDrbgs drbg, int tokBytes, MteVerifiers
verifiers, MteCiphers cipher, MteHashes hash, UInt64 tWindow, Int32 sWindow)
```

Constructor taking a provided interop, the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`interop`: the interop object to use.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.
`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

12.4.1.5 `MteMkeDec` Destructor (CS)

```
~MteMkeDec()
```

Destructor. The `Uninstantiate()` method is called.

12.4.1.6 `MteMkeDec.Instantiate(byte[])` (CS)

```
public MteStatus Instantiate(byte[] ps)
```

Instantiate the decoder given the personalization string. The `EntropyCallback()` and `NonceCallback()` virtual functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

12.4.1.7 `MteMkeDec.Instantiate(string)` (CS)

```
public MteStatus Instantiate(string ps)
```

Calls `Instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

12.4.1.8 `MteMkeDec.GetReseedCounter` (CS)

```
public UInt64 GetReseedCounter()
```

Returns the decoder's DRBG reseed counter.

12.4.1.9 `MteMkeDec.SaveState` (CS)

```
public byte[] SaveState()
```

Returns the saved state in raw form. Returns `null` on error.

12.4.1.10 `MteMkeDec.SaveStateB64` (CS)

```
public string SaveStateB64()
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

12.4.1.11 `MteMkeDec.RestoreState` (CS)

```
public MteStatus RestoreState(byte[] saved)
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

12.4.1.12 `MteMkeDec.RestoreStateB64` (CS)

```
public MteStatus RestoreStateB64(string saved)
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

12.4.1.13 `MteMkeDec.GetBuffBytes` (CS)

```
public int GetBuffBytes(byte[] encoded, int encOff, int encBytes)
```

Returns the decode buffer size in bytes given the raw encoded message at the given offset and size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

12.4.1.14 `MteMkeDec.GetBuffBytesB64` (CS)

```
public int GetBuffBytesB64(byte[] encoded, int encOff, int encBytes)
```

Returns the decode buffer size in bytes given the Base64-encoded message at the given offset and size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

12.4.1.15 `MteMkeDec.Decode` (CS)

```
public byte[] Decode(byte[] encoded, out MteStatus status)
```

Decode the given raw encoded data. Returns the decoded data and sets `status` to the status. The decoded version is valid only if `!StatusIsError(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`status`: the status.

12.4.1.16 `MteMkeDec.DecodeB64` (CS)

```
public byte[] DecodeB64(string encoded, out MteStatus status)
```

Decode the given Base64-encoded encoded data. Returns the decoded data and sets `status` to the status. The decoded version is valid only if `!StatusIsError(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`status`: the status.

12.4.1.17 `MteMkeDec.DecodeStr` (CS)

```
public string DecodeStr(byte[] encoded, out MteStatus status)
```

Decode the given raw encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and sets `status` to the status. The decoded version is valid only if `!StatusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`status`: the status.

12.4.1.18 `MteMkeDec.DecodeStrB64` (CS)

```
public string DecodeStrB64(string encoded, out MteStatus status)
```

Decode the given Base64-encoded encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and sets `status` to the status. The decoded version is valid only if `!StatusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`status`: the status.

12.4.1.19 `MteMkeDec.Decode` (external buffers) (CS)

```
public MteStatus Decode(byte[] encoded, int encOff, int encBytes, byte[] decoded,
ref int decOff, out int decBytes)
```

Decode the given raw `encoded` version of the given length at the given offset to the given buffer at the given offset. Returns the status. Adjusts `decOff` to the offset of the decoded version in the `decoded` buffer. Sets `decBytes` to the decoded length in bytes. The `decoded` buffer must have sufficient length remaining after the offset. Use `GetBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: buffer containing the encoded data.
`encOff`: offset to the start of the encoded data.
`encBytes`: length of the encoded data in bytes.
`decoded`: buffer to hold the decoded data.
`decOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the decoded data.
`decBytes`: set to the length of the decoded data in bytes.

12.4.1.20 `MteMkeDec.DecodeB64` (external buffers) (CS)

```
public MteStatus DecodeB64(byte[] encoded, int encOff, int encBytes, byte[]
decoded, ref int decOff, out int decBytes)
```

Decode the given Base64 `encoded` version of the given length at the given offset to the given buffer at the given offset. Returns the status. Adjusts `decOff` to the offset of the decoded version in the `decoded` buffer. Sets `decBytes` to the decoded length in bytes. The `decoded` buffer must have sufficient length remaining after the offset. Use `GetBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: buffer containing the encoded data.
`encOff`: offset to the start of the encoded data.
`encBytes`: length of the encoded data in bytes.
`decoded`: buffer to hold the decoded data.
`decOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the decoded data.
`decBytes`: set to the length of the decoded data in bytes.

12.4.1.21 `MteMkeDec.StartDecrypt` (CS)

```
public MteStatus StartDecrypt()
```

Start a chunk-based decryption session. Returns the status.

12.4.1.22 `MteMkeDec.DecryptChunk` (CS)

```
public byte[] DecryptChunk(byte[] encrypted)
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data is used as input and some amount of decrypted data is returned. The amount decrypted may be less than the input size. Returns `null` on error.

`encrypted`: the encrypted data to decrypt.

12.4.1.23 `MteMkeDec.DecryptChunk` (external buffers) (CS)

```
public int DecryptChunk(byte[] encrypted, int encOff, int encBytes, byte[]
decrypted, int decOff)
```

Decrypt a chunk of data starting at `encOff` and of length `encBytes` in the `encrypted` buffer in a chunk-based decryption session. Some decrypted data is written to the `decrypted` buffer starting at `decOff`. The amount decrypted may be less than the input size and is returned. Returns `-1` on error.

`encrypted`: buffer containing encrypted data to decrypt.
`encOff`: offset to the start of the encrypted data.
`encBytes`: length of the encrypted data in bytes.
`decrypted`: buffer to hold decrypted data.
`decOff`: offset to start writing decrypted data.

12.4.1.24 `MteMkeDec.FinishDecrypt` (CS)

```
public byte[] FinishDecrypt(out MteStatus status)
```

Finish the chunk-based decryption session. Returns the final decrypted data and sets `status` to the status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`status`: the status.

12.4.1.25 `MteMkeDec.FinishDecrypt` (external buffers) (CS)

```
public MteStatus FinishDecrypt(byte[] decrypted, int off, out int bytes)
```

Finish the chunk-based decryption session. Writes the final decrypted data to the `decrypted` buffer starting at offset `off`. Sets `bytes` to the length of the decrypted data. Returns the status. The `decrypted` buffer must be large enough to hold the chosen cipher's block size in the mode of operation.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`decrypted`: buffer to hold decrypted data.
`off`: offset to start writing decrypted data.
`bytes`: set to the length of the decrypted data in bytes.

12.4.1.26 `MteMkeDec.GetEncTs` (CS)

```
public UInt64 GetEncTs()
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

12.4.1.27 `MteMkeDec.GetDecTs` (CS)

```
public UInt64 GetDecTs()
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

12.4.1.28 `MteMkeDec.GetMsgSkipped` (CS)

```
public UInt32 GetMsgSkipped()
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

12.4.1.29 `MteMkeDec.Uninstantiate` (CS)

```
public MteStatus Uninstantiate()
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

12.4.2 Encoder

Encoder-specific functions are declared in `MteMkeEnc.cs`.

12.4.2.1 `MteMkeEnc` Constructor (default options) (CS)

```
public MteMkeEnc()
```

Constructor using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants. The default interop is used.

12.4.2.2 `MteMkeEnc` Constructor (interop, default options) (CS)

```
public MteMkeEnc(MteInterop interop)
```

Constructor using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants. The provided interop is used.

`interop`: the interop object to use.

12.4.2.3 `MteMkeEnc` Constructor (runtime options) (CS)

```
public MteMkeEnc(MteDrbgs drbg, int tokBytes, MteVerifiers verifiers, MteCiphers
cipher, MteHashes hash)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm. The default interop is used.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.

12.4.2.4 `MteMkeEnc` Constructor (interop, runtime options) (CS)

```
public MteMkeEnc(MteInterop interop, MteDrbgs drbg, int tokBytes, MteVerifiers
verifiers, MteCiphers cipher, MteHashes hash)
```

Constructor taking a provided interop, the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`interop`: the interop object to use.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.

12.4.2.5 `MteMkeEnc` Destructor (CS)

```
~MteMkeEnc()
```

Destructor. The `Uninstantiate()` method is called.

12.4.2.6 `MteMkeEnc.Instantiate` (byte[]) (CS)

```
public MteStatus Instantiate(byte[] ps)
```

Instantiate the encoder given the personalization string. The `EntropyCallback()` and `NonceCallback()` virtual functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

12.4.2.7 `MteMkeEnc.Instantiate` (`string`) (CS)

```
public MteStatus Instantiate(string ps)
```

Calls `Instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

12.4.2.8 `MteMkeEnc.GetReseedCounter` (CS)

```
public UInt64 GetReseedCounter()
```

Returns the encoder's DRBG reseed counter.

12.4.2.9 `MteMkeEnc.SaveState` (CS)

```
public byte[] SaveState()
```

Returns the saved state in raw form. Returns `null` on error.

12.4.2.10 `MteMkeEnc.SaveStateB64` (CS)

```
public string SaveStateB64()
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

12.4.2.11 `MteMkeEnc.RestoreState` (CS)

```
public MteStatus RestoreState(byte[] saved)
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

12.4.2.12 `MteMkeEnc.RestoreStateB64` (CS)

```
public MteStatus RestoreStateB64(string saved)
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

12.4.2.13 `MteMkeEnc.GetBuffBytes` (CS)

```
public int GetBuffBytes(int dataBytes)
```

Returns the raw encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

12.4.2.14 `MteMkeEnc.GetBuffBytesB64` (CS)

```
public int GetBuffBytesB64(int dataBytes)
```

Returns the Base64 encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

12.4.2.15 `MteMkeEnc.Encode(byte[])` (CS)

```
public byte[] Encode(byte[] data, out MteStatus status)
```

Encode the given data in raw form. Returns the encoded version and sets `status` to the status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`status`: the status.

12.4.2.16 `MteMkeEnc.EncodeB64(byte[])` (CS)

```
public string EncodeB64(byte[] data, out MteStatus status)
```

Encode the given data in Base64-encoded form. Returns the encoded version and sets `status` to the status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`status`: the status.

12.4.2.17 `MteMkeEnc.Encode` (`string`) (CS)

```
public byte[] Encode(string str, out MteStatus status)
```

Calls `Encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.
`status`: the status.

12.4.2.18 `MteMkeEnc.EncodeB64` (`string`) (CS)

```
public string EncodeB64(string str, out MteStatus status)
```

Calls `EncodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.
`status`: the status.

12.4.2.19 `MteMkeEnc.Encode` (external buffers) (CS)

```
public MteStatus Encode(byte[] data, int dataOff, int dataBytes, byte[] encoded,
ref int encOff, out int encBytes)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the status. Adjusts `encOff` to the offset of the encoded version in the `encoded` buffer. Sets `encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `GetBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

12.4.2.20 `MteMkeEnc.EncodeB64` (external buffers) (CS)

```
public MteStatus EncodeB64(byte[] data, int dataOff, int dataBytes, byte[] encoded,
ref int encOff, out int encBytes)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the status. Adjusts `encOff` to the offset of the encoded version in the `encoded` buffer. Sets `encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `GetBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

12.4.2.21 `MteMkeEnc.EncryptFinishBytes` (CS)

```
public int EncryptFinishBytes()
```

Returns the length of the result `FinishEncrypt()` will produce. Use this if you need to know that size before you can call it.

12.4.2.22 `MteMkeEnc.StartEncrypt` (CS)

```
public MteStatus StartEncrypt()
```

Start a chunk-based encryption session. Returns the status.

12.4.2.23 `MteMkeEnc.EncryptChunk` (CS)

```
public MteStatus EncryptChunk(byte[] data)
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` is encrypted in place. The `data.Length` must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: the data to encrypt.

12.4.2.24 `MteMkeEnc.EncryptChunk` (slice) (CS)

```
public MteStatus EncryptChunk(byte[] data, int off, int bytes)
```

Encrypt a chunk of data at the given offset of the given length in a chunk-based encryption session. The `data` is encrypted in place. The `bytes` length must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: buffer containing the data to encrypt.
`off`: offset to the start of the data to encrypt.
`bytes`: length of the data to encrypt in bytes.

12.4.2.25 `MteMkeEnc.FinishEncrypt` (CS)

```
public byte[] FinishEncrypt(out MteStatus status)
```

Finish the chunk-based encryption session. Returns the tokenized hash and sets `status` to the status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`status`: the status.

12.4.2.26 `MteMkeEnc.Uninstantiate` (CS)

```
public MteStatus Uninstantiate()
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

13 Go Language Interface

13.1 Overview

13.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use `NewMkeEncDef()` or `NewMkeEncOpt()` to create the encoder object and `(*MkeEnc)Instantiate()` or `(*MkeEnc)InstantiateStr()` to instantiate it.

To encode, use `(*MkeEnc)Encode()`, `(*MkeEnc)EncodeB64()`, `(*MkeEnc)EncodeStr()`, or `(*MkeEnc)EncodeStrB64()`.

To destroy, call `(*MkeEnc)Destroy()`. The Go objects hold on to some global resources, so it is recommended to destroy the object to release those. The destroy method also uninstantiates the encoder for security.

13.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and destroy it.

To create, use `NewMkeDecDef()`, `NewMkeDecWin()`, or `NewMkeDecOpt()` to create the decoder object and `(*MkeDec)Instantiate()` or `(*MkeDec)InstantiateStr()` to instantiate it.

To decode, use `(*MkeDec)Decode()`, `(*MkeDec)DecodeB64()`, `(*MkeDec)DecodeStr()`, or `(*MkeDec)DecodeStrB64()`.

To get additional information, use `(*MkeDec)GetEncTs()`, `(*MkeDec)GetDecTs()`, and `(*MkeDec)GetMsgSkipped()`, depending on which verifiers are enabled.

To destroy, call `(*MkeDec)Destroy()`. The Go objects hold on to some global resources, so it is recommended to destroy the object to release those. The destroy method also uninstantiates the decoder for security.

13.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `(*MkeEnc)StartEncrypt()` to start a chunk session, `(*MkeEnc)EncryptChunk()` repeatedly to encrypt each chunk, then `(*MkeEnc)FinishEncrypt()` to finish the session.

13.1.4 Chunk Decrypt

The chunk decrypt follows the flow of a decoder, but instead of using a decode method, use `(*MkeDec)StartDecrypt()` to start a chunk session, `(*MkeDec)DecryptChunk()` repeatedly to decrypt each chunk, then `(*MkeDec)FinishDecrypt()` to finish the session.

13.2 Files

The MTE Managed-Key Encryption Add-On uses the core source files (other than `MteEnc.go` and `MteDec.go`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteMkeDec.go	The MteMkeDec structure and functions.
MteMkeEnc.go	The MteMkeEnc structure and functions.

13.3 Types

13.3.1 Structures

13.3.1.1 `MkeDec`

Defines decoder-specific data structure.

13.3.1.2 `MkeEnc`

Defines encoder-specific data structure.

13.4 Functions

13.4.1 Decoder

Decoder-specific functions are declared in `MteMkeDec.go`.

13.4.1.1 `NewMkeDecDef` (Go)

```
func NewMkeDecDef() *MkeDec
```

Creates a new `MkeDec` structure using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `default`* constants. The timestamp window and sequence window are set to `0`. It must be destroyed with `Destroy()` when finished to free resources.

13.4.1.2 `NewMkeDecWin` (Go)

```go
func NewMkeDecWin(tWindow uint64, sWindow int) *MkeDec
```

Creates a new `MkeDec` structure using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `default`* constants. It must be destroyed with `Destroy()` when finished to free resources.

`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

13.4.1.3 `NewMkeDecOpt` (Go)

```go
func NewMkeDecOpt(drbg Drbgs, tokBytes int, verifiers Verifiers, cipher Ciphers,
hash Hashes, tWindow uint64, sWindow int) *MkeDec
```

Creates a new `MkeDec` structure using the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window. It must be destroyed with `Destroy()` when finished to free resources.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `Drbgs_mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `Ciphers_mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `Hashes_mte_hashes_none`.
`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

13.4.1.4 `(*MkeDec)Destroy` (Go)

```go
func (dec *MkeDec) Destroy()
```

Releases resources. The `Uninstantiate()` method is called.

13.4.1.5 `(*MkeDec)Instantiate` (Go)

```go
func (dec *MkeDec) Instantiate(ps []byte) Status
```

Instantiate the decoder given the personalization string. The `EntropyCallback()` and `NonceCallback()` functions defined for the current context set with `ContextStore()` will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

13.4.1.6 `(*MkeDec)InstantiateStr` (Go)

```go
func (dec *MkeDec) Instantiate(ps string) Status
```

Calls `Instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

13.4.1.7 `(*MkeDec)GetReseedCounter` (Go)

```go
func (dec *MkeDec) GetReseedCounter() uint64
```

Returns the decoder's DRBG reseed counter.

13.4.1.8 `(*MkeDec)SaveState` (Go)

```go
func (dec *MkeDec) SaveState() []byte
```

Returns the saved state in raw form. Returns `nil` on error.

13.4.1.9 `(*MkeDec)SaveStateB64` (Go)

```go
func (dec *MkeDec) SaveStateB64() string
```

Returns the saved state in Base64-encoded form. Returns `""` on error.

13.4.1.10 `(*MkeDec)RestoreState` (Go)

```go
func (dec *MkeDec) RestoreState(saved []byte) Status
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

13.4.1.11 `(*MkeDec)RestoreStateB64` (Go)

```go
func (dec *MkeDec) RestoreStateB64(saved string) Status
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the string to restore the state from.

13.4.1.12 `(*MkeDec)Decode` (Go)

```go
func (dec *MkeDec) Decode(encoded []byte) ([]byte, Status)
```

Decode the given raw encoded data. Returns the decoded data and status. The decoded version is valid only if `!StatusIsError(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

13.4.1.13 `(*MkeDec)DecodeB64` (Go)

```go
func (dec *MkeDec) DecodeB64(encoded string) ([]byte, Status)
```

Decode the given Base64-encoded encoded data. Returns the decoded data and status. The decoded version is valid only if `!StatusIsError(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

13.4.1.14 `(*MkeDec)DecodeStr` (Go)

```go
func (*MkeDec) DecodeStr(encoded []byte) (string, Status)
```

Decode the given raw encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and status. The decoded version is valid only if `!StatusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

13.4.1.15 `(*MkeDec)DecodeStrB64` (Go)

```go
func (dec *MkeDec) DecodeStrB64(encoded string) (string, Status)
```

Decode the given Base64-encoded encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and status. The decoded version is valid only if `!StatusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

13.4.1.16 `(*MkeDec)StartDecrypt` (Go)

```go
func (dec *MkeDec) StartDecrypt() Status
```

Start a chunk-based decryption session. Returns the status.

13.4.1.17 `(*MkeDec)DecryptChunk` (Go)

```go
func (dec *MkeDec) DecryptChunk(encrypted []byte) []byte
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data is used as input and some amount of decrypted dats is returned. The amount decrypted may be less than the input size. Returns `nil` on error.

`encrypted`: the encrypted data to decrypt.

13.4.1.18 `(*MkeDec)FinishDecrypt` (Go)

```go
func (dec *MkeDec) FinishDecrypt() ([]byte, Status)
```

Finish the chunk-based decryption session. Returns the final decrypted data and status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

13.4.1.19 `(*MkeDec)GetEncTs` (Go)

```go
func (dec *MkeDec) GetEncTs() uint64
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

13.4.1.20 `(*MkeDec)GetMkeDecTs` (Go)

```
func (*MkeDec) GetMkeDecTs() uint64
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

13.4.1.21 `(*MkeDec)GetMsgSkipped` (Go)

```
func (dec *MkeDec) GetMsgSkipped() uint32
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

13.4.1.22 `(*MkeDec)Uninstantiate` (Go)

```
func (dec *MkeDec) Uninstantiate() Status
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

13.4.2 Encoder

Encoder-specific functions are declared in `MteMkeEnc.go`.

13.4.2.1 `NewMkeEncDef` (Go)

```
func NewMkeEncDef() *MkeEnc
```

Creates a new `MkeEnc` structure using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `default`* constants. It must be destroyed with `Destroy()` when finished to free resources.

13.4.2.2 `NewMkeEncOpt` (Go)

```
func NewMkeEncOpt(drbg Drbgs, tokBytes int, verifiers Verifiers, cipher Ciphers,
hash Hashes) *MkeEnc
```

Creates a new `MkeEnc` structure using the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm. It must be destroyed with `Destroy()` when finished to free resources.

Buildtime options: the `drbg`, `tokBytes`, `byteValMin`, `byteValCount`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `Drbgs_mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `Ciphers_mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `Hashes_mte_hashes_none`.

13.4.2.3 `(*MkeEnc)Destroy` (Go)

```go
func (enc *MkeEnc) Destroy()
```

Releases resources. The `Uninstantiate()` method is called.

13.4.2.4 `(*MkeEnc)Instantiate` (Go)

```go
func (enc *MkeEnc) Instantiate(ps []byte) Status
```

Instantiate the encoder given the personalization string. The `EntropyCallback()` and `NonceCallback()` functions defined for the current context set with `ContextStore()` will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

13.4.2.5 `(*MkeEnc)InstantiateStr` (Go)

```go
func (enc *MkeEnc) Instantiate(ps string) Status
```

Calls `Instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

13.4.2.6 `(*MkeEnc)GetReseedCounter` (Go)

```go
func (enc *MkeEnc) GetReseedCounter() uint64
```

Returns the encoder's DRBG reseed counter.

13.4.2.7 `(*MkeEnc)SaveState` (Go)

```go
func (enc *MkeEnc) SaveState() []byte
```

Returns the saved state in raw form. Returns `nil` on error.

13.4.2.8 `(*MkeEnc)SaveStateB64` (Go)

```go
func (enc *MkeEnc) SaveStateB64() string
```

Returns the saved state in Base64-encoded form. Returns `""` on error.

13.4.2.9 `(*MkeEnc)RestoreState` (Go)

```go
func (enc *MkeEnc) RestoreState(saved []byte) Status
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

13.4.2.10 `(*MkeEnc)RestoreStateB64` (Go)

```go
func (enc *MkeEnc) RestoreStateB64(saved string) Status
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the string to restore the state from.

13.4.2.11 `(*MkeEnc)MkeEncode` (Go)

```go
func (enc *MkeEnc) MkeEncode(data []byte) ([]byte, Status)
```

MkeEncode the given data in raw form. Returns the encoded version and status. The encoded version is valid only if `status == Status_mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

13.4.2.12 `(*MkeEnc)MkeEncodeB64` (Go)

```
func (enc *MkeEnc) MkeEncodeB64(data []byte) (string, Status)
```

MkeEncode the given data in Base64-encoded form. Returns the encoded version and status. The encoded version is valid only if `status == Status_mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

13.4.2.13 `(*MkeEnc)MkeEncodeStr` (Go)

```
func (enc *MkeEnc) MkeEncodeStr(str string) ([]byte, Status)
```

Calls `MkeEncode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

13.4.2.14 `(*MkeEnc)MkeEncodeStrB64` (Go)

```
func (enc *MkeEnc) MkeEncodeStrB64(str string) (string, Status)
```

Calls `MkeEncodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

13.4.2.15 `(*MkeEnc)EncryptFinishBytes` (Go)

```
func (enc *MkeEnc) EncryptFinishBytes() int
```

Returns the length of the result `FinishEncrypt()` will produce. Use this if you need to know that size before you can call it.

13.4.2.16 `(*MkeEnc)StartEncrypt` (Go)

```
func (enc *MkeEnc) StartEncrypt() Status
```

Start a chunk-based encryption session. Returns the status.

13.4.2.17 `(*MkeEnc)EncryptChunk` (Go)

```go
func (enc *MkeEnc) EncryptChunk(data []byte) Status
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` is encrypted in place. The `len(data)` must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: the data to encrypt.

13.4.2.18 `(*MkeEnc)FinishEncrypt` (Go)

```go
func (enc *MkeEnc) FinishEncrypt() ([]byte, Status)
```

Finish the chunk-based encryption session. Returns the tokenized hash and status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`status`: the status.

13.4.2.19 `(*MkeEnc)Uninstantiate` (Go)

```go
func (enc *MkeEnc) Uninstantiate() Status
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

14 Java Language Interface

14.1 Overview

14.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and zeroize it.

To create, use an `MteMkeEnc` constructor to create an encoder object and one of the `MteMkeEnc.instantiate()` overloads to instantiate it.

To encode, use any of the `MteMkeEnc.encode()` or `MteMkeEnc.encodeB64()` overloads.

To zeroize, call `MteMkeEnc.uninstantiate()`.

14.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and zeroize it.

To create, use an `MteMkeDec` constructor to create a decoder object and one of the `MteMkeDec.instantiate()` overloads to instantiate it.

To decode, use `MteMkeDec.decode()`, `MteMkeDec.decodeB64()`, `MteMkeDec.decodeStr()`, or `MteMkeDec.decodeStrB64()`.

To get additional information, use `MteMkeDec.getEncTs()`, `MteMkeDec.getDecTs()`, and `MteMkeDec.getMsgSkipped()`, depending on which verifiers are enabled.

To zeroize, call `MteMkeDec.uninstantiate()`.

14.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `MteMkeEnc.startEncrypt()` to start a chunk session, `MteMkeEnc.encryptChunk()` repeatedly to encrypt each chunk, then `MteMkeEnc.finishEncrypt()` to finish the session.

14.1.4 Chunk Decrypt

The chunk encrypt follows the flow of a decoder, but instead of using a decode method, use `MteMkeDec.startDecrypt()` to start a chunk session, `MteMkeDec.decryptChunk()` repeatedly to decrypt each chunk, then `MteMkeEnc.finishDecrypt()` to finish the session.

14.2 Files

The MTE Managed-Key Encryption Add-On uses the core source files (other than `MteEnc.java` and `MteDec.java`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteMkeDec.java	The MteMkeDec class.
MteMkeEnc.java	The MteMkeEnc class.

14.3 Types

14.3.1 Classes

14.3.1.1 `MteMkeDec`

Defines decoder-specific methods.

14.3.1.2 `MteMkeEnc`

Defines encoder-specific methods.

14.4 Functions

14.4.1 Decoder

Decoder-specific functions are declared in `MteMkeDec.java`.

14.4.1.1 `MteMkeDec` *Constructor (default options) (Java)*

```
public MteMkeDec()
```

Constructor using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants. The timestamp window and sequence window are set to `0`.

14.4.1.2 `MteMkeDec` *Constructor (window options) (Java)*

```
public MteMkeDec(long tWindow, int sWindow)
```

Constructor using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants.

`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

14.4.1.3 `MteMkeDec` Constructor (runtime options) (Java)

```
public MteMkeDec(MteDrbgs drbg, int tokBytes, MteVerifiers verifiers, MteCiphers
cipher, MteHashes hash, long tWindow, int sWindow)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.
`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

14.4.1.4 `MteMkeDec.instantiate(byte[])` (Java)

```
public MteStatus instantiate(byte[] ps)
```

Instantiate the decoder given the personalization string. The `entropyCallback()` and `nonceCallback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

14.4.1.5 `MteMkeDec.instantiate(String)` (Java)

```
public MteStatus instantiate(String ps)
```

Calls `instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

14.4.1.6 `MteMkeDec.getReseedCounter` (Java)

```
public long getReseedCounter()
```

Returns the decoder's DRBG reseed counter.

14.4.1.7 `MteMkeDec.saveState` (Java)

```
public byte[] saveState()
```

Returns the saved state in raw form. Returns `null` on error.

14.4.1.8 `MteMkeDec.saveStateB64` (Java)

```
public String saveStateB64()
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

14.4.1.9 `MteMkeDec.restoreState` (Java)

```
public MteStatus restoreState(byte[] saved)
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

14.4.1.10 `MteMkeDec.restoreStateB64` (Java)

```
public MteStatus restoreStateB64(String saved)
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

14.4.1.11 `MteMkeDec.getBuffBytes` (Java)

```
public int getBuffBytes(byte[] encoded, int encOff, int encBytes)
```

Returns the decode buffer size in bytes given the raw encoded message at the given offset and size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

14.4.1.12 `MteMkeDec.getBuffBytesB64` (Java)

```
public int getBuffBytesB64(byte[] encoded, int encOff, int encBytes)
```

Returns the decode buffer size in bytes given the Base64-encoded message at the given offset and size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

14.4.1.13 `MteMkeDec.decode` (Java)

```
public ArrStatus decode(byte[] encoded)
```

Decode the given raw encoded data. Returns the decoded data and status. The decoded version is valid only if `!statusIsError(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

14.4.1.14 `MteMkeDec.decodeB64` (Java)

```
public ArrStatus decodeB64(String encoded)
```

Decode the given Base64-encoded encoded data. Returns the decoded data and status. The decoded version is valid only if `!statusIsError(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

14.4.1.15 `MteMkeDec.decodeStr` (Java)

```
public StrStatus decodeStr(byte[] encoded)
```

Decode the given raw encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and status. The decoded version is valid only if `!statusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

14.4.1.16 `MteMkeDec.decodeStrB64` (Java)

```
public StrStatus decodeStrB64(String encoded)
```

Decode the given Base64-encoded encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and status. The decoded version is valid only if `!statusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

14.4.1.17 `MteMkeDec.decode` (external buffers) (Java)

```
public OffLenStatus decode(byte[] encoded, int encOff, int encBytes, byte[]
decoded, int decOff)
```

Decode the given raw `encoded` version of the given length at the given offset to the given buffer at the given offset. Returns the offset of the decoded version, length of the decoded version, and status. The `decoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: buffer containing the encoded data.
`encOff`: offset to the start of the encoded data.
`encBytes`: length of the encoded data in bytes.
`decoded`: buffer to hold the decoded data.
`decOff`: the offset to the beginning of the usable buffer area.

14.4.1.18 `MteMkeDec.decodeB64` (external buffers) (Java)

```
public OffLenStatus decodeB64(byte[] encoded, int encOff, int encBytes, byte[]
decoded, int decOff)
```

Decode the given Base64 `encoded` version of the given length at the given offset to the given buffer at the given offset. Returns the offset of the decoded version, length of the decoded version, and status. The `decoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: buffer containing the encoded data.
`encOff`: offset to the start of the encoded data.
`encBytes`: length of the encoded data in bytes.
`decoded`: buffer to hold the decoded data.
`decOff`: the offset to the beginning of the usable buffer area.

14.4.1.19 `MteMkeDec.startDecrypt` (Java)

```
public MteStatus startDecrypt()
```

Start a chunk-based decryption session. Returns the status.

14.4.1.20 `MteMkeDec.decryptChunk` (Java)

```
public byte[] decryptChunk(byte[] encrypted)
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data is used as input and some amount of decrypted data is returned. The amount decrypted may be less than the input size. Returns `null` on error.

`encrypted`: the encrypted data to decrypt.

14.4.1.21 `MteMkeDec.decryptChunk` (external buffers) (Java)

```
public int decryptChunk(byte[] encrypted, int encOff, int encBytes, byte[]
decrypted, int decOff)
```

Decrypt a chunk of data starting at `encOff` and of length `encBytes` in the `encrypted` buffer in a chunk-based decryption session. Some decrypted data is written to the `decrypted` buffer starting at `decOff`. The amount decrypted may be less than the input size and is returned. Returns `-1` on error.

`encrypted`: buffer containing encrypted data to decrypt.
`encOff`: offset to the start of the encrypted data.
`encBytes`: length of the encrypted data in bytes.
`decrypted`: buffer to hold decrypted data.
`decOff`: offset to start writing decrypted data.

14.4.1.22 `MteMkeDec.finishDecrypt` (Java)

```
public ArrStatus finishDecrypt()
```

Finish the chunk-based decryption session. Returns the final decrypted data and status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

14.4.1.23 `MteMkeDec.finishDecrypt` (external buffers) (Java)

```
public OffLenStatus finishDecrypt(byte[] decrypted, int off)
```

Finish the chunk-based decryption session. Writes the final decrypted data to the `decrypted` buffer starting at offset `off`. Returns the offset of the decrypted version, length of the decrypted version, and status. The `decrypted` buffer must be large enough to hold the chosen cipher's block size in the mode of operation.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`decrypted`: buffer to hold decrypted data.
`off`: offset to start writing decrypted data.
`bytes`: set to the length of the decrypted data in bytes.

14.4.1.24 `MteMkeDec.getEncTs` (Java)

```
public long getEncTs()
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

14.4.1.25 `MteMkeDec.getDecTs` (Java)

```
public long getDecTs()
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

14.4.1.26 `MteMkeDec.getMsgSkipped` (Java)

```
public int getMsgSkipped()
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

14.4.1.27 `MteMkeDec.uninstantiate` (Java)

```
public MteStatus uninstantiate()
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

14.4.2 Encoder

Encoder-specific functions are declared in `MteMkeEnc.java`.

14.4.2.1 `MteMkeEnc` Constructor (default options) (Java)

```
public MteMkeEnc()
```

Constructor using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants.

14.4.2.2 `MteMkeEnc` Constructor (runtime options) (Java)

```
public MteMkeEnc(MteDrbgs drbg, int tokBytes, MteVerifiers verifiers, MteCiphers
cipher, MteHashes hash)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.

14.4.2.3 `MteMkeEnc.instantiate(byte[])` (Java)

```
public MteStatus instantiate(byte[] ps)
```

Instantiate the encoder given the personalization string. The `entropyCallback()` and `nonceCallback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

14.4.2.4 `MteMkeEnc.instantiate(String)` (Java)

```
public MteStatus instantiate(String ps)
```

Calls `instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

14.4.2.5 `MteMkeEnc.getReseedCounter` (Java)

```
public long getReseedCounter()
```

Returns the encoder's DRBG reseed counter.

14.4.2.6 `MteMkeEnc.saveState` (Java)

```
public byte[] saveState()
```

Returns the saved state in raw form. Returns `null` on error.

14.4.2.7 `MteMkeEnc.saveStateB64` (Java)

```
public String saveStateB64()
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

14.4.2.8 `MteMkeEnc.restoreState` (Java)

```
public MteStatus restoreState(byte[] saved)
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

14.4.2.9 `MteMkeEnc.restoreStateB64` (Java)

```
public MteStatus restoreStateB64(String saved)
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

14.4.2.10 `MteMkeEnc.getBuffBytes` (Java)

```
public int getBuffBytes(int dataBytes)
```

Returns the raw encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

14.4.2.11 `MteMkeEnc.getBuffBytesB64` (Java)

```
public int getBuffBytesB64(int dataBytes)
```

Returns the Base64 encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

14.4.2.12 `MteMkeEnc.encode` (`byte[]`) (Java)

```
public ArrStatus encode(byte[] data)
```

Encode the given data in raw form. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

14.4.2.13 `MteMkeEnc.encodeB64` (`byte[]`) (Java)

```
public StrStatus encodeB64(byte[] data)
```

Encode the given data in Base64-encoded form. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

14.4.2.14 `MteMkeEnc.encode` (`String`) (Java)

```
public ArrStatus encode(String str)
```

Calls `encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

14.4.2.15 `MteMkeEnc.encodeB64` (`String`) (Java)

```
public StrStatus encodeB64(String str)
```

Calls `encodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

14.4.2.16 `MteMkeEnc.encode` (external buffers) (Java)

```
public OffLenStatus encode(byte[] data, int dataOff, int dataBytes, byte[] encoded,
int encOff)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the offset of the encoded version, length of the encoded version, and status. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: the offset to the beginning of the usable buffer area.

14.4.2.17 `MteMkeEnc.encodeB64` (external buffers) (Java)

```
public OffLenStatus encodeB64(byte[] data, int dataOff, int dataBytes, byte[]
encoded, int encOff)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the offset of the encoded version, length of the encoded version, and status. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

14.4.2.18 `MteMkeEnc.encryptFinishBytes` (Java)

```
public int encryptFinishBytes()
```

Returns the length of the result `finishEncrypt()` will produce. Use this if you need to know that size before you can call it.

14.4.2.19 `MteMkeEnc.startEncrypt` (Java)

```
public MteStatus startEncrypt()
```

Start a chunk-based encryption session. Returns the status.

14.4.2.20 `MteMkeEnc.encryptChunk` (Java)

```
public MteStatus encryptChunk(byte[] data)
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` is encrypted in place. The `data.length` must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: the data to encrypt.

14.4.2.21 `MteMkeEnc.encryptChunk` (slice) (Java)

```
public MteStatus encryptChunk(byte[] data, int off, int bytes)
```

Encrypt a chunk of data at the given offset of the given length in a chunk-based encryption session. The `data` is encrypted in place. The `bytes` length must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: buffer containing the data to encrypt.
`off`: offset to the start of the data to encrypt.
`bytes`: length of the data to encrypt in bytes.

14.4.2.22 `MteMkeEnc.finishEncrypt` (Java)

```
public ArrStatus finishEncrypt()
```

Finish the chunk-based encryption session. Returns the tokenized hash and status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`status`: the status.

14.4.2.23 `MteMkeEnc.uninstantiate` (Java)

```
public MteStatus uninstantiate()
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

15 Objective-C Language Interface

15.1 Overview

15.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use an `-init`* message to initialize an encoder object and an `-instantiate:`* message to instantiate it.

To encode, use an `-encode:`* message.

To destroy, release the object (this may be automatic if using ARC or autorelease).

15.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and destroy it.

To create, use an `-init`* message to initialize a decoder object and an `-instantiate:`* message to instantiate it.

To decode, use a `-decode:`* message.

To get additional information, use `-getEncTs`, `-getDecTs`, and `-getMsgSkipped`, depending on which verifiers are enabled.

To destroy, release the object (this may be automatic if using ARC or autorelease).

15.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `-startEncrypt` to start a chunk session, `-encryptChunk:dataBytes:` repeatedly to encrypt each chunk, then `-finishEncrypt:status:` to finish the session.

15.1.4 Chunk Decrypt

The chunk decrypt follows the flow of a decoder, but instead of using a decode method, use `-startDecrypt` to start a chunk session, `-decryptChunk:encryptedBytes:decryptedBytes:` repeatedly to decrypt each chunk, then `-finishDecrypt:status:` to finish the session.

15.2 Files

The MTE Managed-Key Encryption Add-On uses the core header files (other than `MteEnc.h` and `MteDec.h`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteMkeDec.h	The MteMkeDec class declaration.
MteMkeEnc.h	The MteMkeEnc class declaration.

The MTE Managed-Key Encryption Add-On uses the core source files (other than `MteEnc.m` and `MteDec.m`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteMkeDec.m	The MteMkeDec class definition.
MteMkeEnc.m	The MteMkeEnc class definition.

15.3 Types

15.3.1 Classes

15.3.1.1 `MteMkeDec`

Defines decoder-specific methods.

15.3.1.2 `MteMkeEnc`

Defines encoder-specific methods.

15.4 Functions

15.4.1 Decoder

Decoder-specific functions are declared in `MteMkeDec.h`.

15.4.1.1 `MteMkeDec -init` (ObjC)

```
-(id)init
```

Initializer using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MTE_BASE_DEFAULT_`* constants. The timestamp window and sequence window are set to `0`.

15.4.1.2 `MteMkeDec -initWithTWindow:sWindow:` (ObjC)

```
-(id)initWithTWindow:(uint64_t)tWindow sWindow:(int32_t)sWindow
```

Initializer using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MTE_BASE_DEFAULT_*` constants.

`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

15.4.1.3 `MteMkeDec -initWithDrbg:tokBytes:verifiers:cipher:hash:tWindow:sWindow:` (ObjC)

```
-(id)initWithDrbg:(mte_drbgs)drbg
         tokBytes:(size_t)tokBytes
        verifiers:(mte_verifiers)verifiers
           cipher:(mte_ciphers)cipher
             hash:(mte_hashes)hash
          tWindow:(uint64_t)tWindow
          sWindow:(int32_t)sWindow
```

Initializer taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`.
`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

15.4.1.4 `MteMkeDec -dealloc` (ObjC)

```
-(void)dealloc
```

Deallocate. The `-uninstantiate` message is sent.

15.4.1.5 `MteMkeDec -instantiate:bytes:` (ObjC)

```
-(mte_status)instantiate:(const void *)ps bytes:(size_t)psBytes
```

Instantiate the decoder given the personalization string, and length of the personalization string in bytes. The `-entropyCallbackWithMinEntropy:minLength:maxLength:entropyInput:eiBytes:` and `-nonceCallbackWithMinLength:maxLength:nonce:nBytes:` messages will be sent to get the rest of the seeding material. Returns the status.

`ps`: personalization string.
`psBytes`: length of the personalization string in bytes.

15.4.1.6 `MteMkeDec -instantiate:` (ObjC)

```
-(mte_status)instantiate:(NSString *)ps
```

Sends `-instantiate:bytes:` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

15.4.1.7 `MteMkeDec -getReseedCounter` (ObjC)

```
-(uint64_t)getReseedCounter
```

Returns the decoder's DRBG reseed counter.

15.4.1.8 `MteMkeDec -saveState:` (ObjC)

```
-(const void *)saveState:(size_t *)stateBytes
```

Returns the saved state in raw form and sets `*stateBytes` to the length of the saved state in bytes. Returns `nil` on error.

`stateBytes`: pointer to the length of the saved state.

15.4.1.9 `MteMkeDec -saveStateB64` (ObjC)

```
-(NSString *)saveStateB64
```

Returns the saved state in Base64-encoded form. Returns `nil` on error.

15.4.1.10 `MteMkeDec -restoreState:` (ObjC)

```
-(mte_status)restoreState:(const void *)saved
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

15.4.1.11 `MteMkeDec -restoreStateB64:` (ObjC)

```
-(mte_status)restoreStateB64:(NSString *)saved
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

15.4.1.12 `MteMkeDec -getBuffBytes:encOff:encBytes` (ObjC)

```
-(size_t)getBuffBytes:(const void *)encoded
               encOff:(size_t)encOff
             encBytes:(size_t)encodedBytes
```

Returns the decode buffer size in bytes given the raw encoded message at the given offset and size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

15.4.1.13 `MteMkeDec -getBuffBytesB64:encOff:encBytes` (ObjC)

```
-(size_t)getBuffBytesB64:(const void *)encoded
                  encOff:(size_t)encOff
                encBytes:(size_t)encodedBytes
```

Returns the decode buffer size in bytes given the Base64-encoded message at the given offset and size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

15.4.1.14 `MteMkeDec -decode:encodedBytes:decodedBytes:status:` (ObjC)

```
-(void *)decode:(const void *)encoded
   encodedBytes:(size_t)encodedBytes
   decodedBytes:(size_t *)decodedBytes
         status:(mte_status *)status
```

Decode the given raw encoded data of the given byte length. Returns the decoded data and sets `*decodedBytes` to the length of the decoded data in bytes and `*status` to the status. The decoded version is valid only if `![statusIsError:*status]`. A null terminator is appended to the decoded data but is not included in `*decodedBytes`.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`encodedBytes`: the length of the encoded data to be decoded (in bytes).
`decodedBytes`: pointer to the length of the decoded version.
`status`: pointer to the status.

15.4.1.15 `MteMkeDec -decodeB64:decodedBytes:status:` (ObjC)

```
-(void *)decodeB64:(NSString *)encoded
      decodedBytes:(size_t *)decodedBytes
            status:(mte_status *)status
```

Decode the given Base64-encoded encoded data. Returns the decoded data and sets `*decodedBytes` to the length of the decoded data in bytes and `*status` to the status. The decoded version is valid only if `![statusIsError:*status]`. A null terminator is appended to the decoded data but is not included in `*decodedBytes`.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`decodedBytes`: pointer to the length of the decoded version.
`status`: pointer to the status.

15.4.1.16 `MteMkeDec -decode:encodedBytes:status:` (ObjC)

```
-(NSString *)decode:(const void *)encoded
       encodedBytes:(size_t)encodedBytes
             status:(mte_status *)status
```

Decode the given raw encoded data of the given byte length. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and sets `*status` to the status. The decoded version is valid only if `![statusIsError:*status]`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`encodedBytes`: the length of the encoded data to be decoded (in bytes).
`status`: pointer to the status.

15.4.1.17 `MteMkeDec -decodeB64:status:` (ObjC)

```
-(NSString *)decodeB64:(NSString *)encoded
                status:(mte_status *)status
```

Decode the given Base64-encoded encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and sets `status` to the status. The decoded version is valid only if `![statusIsError:*status]`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.
`status`: pointer to the status.

15.4.1.18 `MteMkeDec -decode:encOff:encBytes:decoded:decOff:decBytes:` (ObjC)

```
-(mte_status)decode:(const void *)encoded
             encOff:(size_t)encOff
           encBytes:(size_t)encBytes
            decoded:(void *)decoded
             decOff:(size_t *)decOff
           decBytes:(size_t *)decBytes
```

Decode the given raw `encoded` version of the given length at the given offset to the given buffer at the given offset. Returns the status. Adjusts `*decOff` to the offset of the decoded version in the `decoded` buffer. Sets `*decBytes` to the decoded length in bytes. The `decoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes:encOff:encBytes:` to determine the buffer requirement.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`encoded`: buffer containing the encoded data.
`encOff`: offset to the start of the encoded data.
`encBytes`: length of the encoded data in bytes.
`decoded`: buffer to hold the decoded data.
`decOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the decoded data.
`decBytes`: set to the length of the decoded data in bytes.

15.4.1.19 `MteMkeDec -decodeB64:encOff:encBytes:decoded:decOff:decBytes:` (ObjC)

```
-(mte_status)decodeB64:(const void *)encoded
                 encOff:(size_t)encOff
               encBytes:(size_t)encBytes
                decoded:(void *)decoded
                 decOff:(size_t *)decOff
               decBytes:(size_t *)decBytes
```

Decode the given Base64 `encoded` version of the given length at the given offset to the given buffer at the given offset. Returns the status. Adjusts `*decOff` to the offset of the decoded version in the `decoded` buffer. Sets `*decBytes` to the decoded length in bytes. The `decoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64:encOff:encBytes:` to determine the buffer requirement.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`encoded`: buffer containing the encoded data.
`encOff`: offset to the start of the encoded data.
`encBytes`: length of the encoded data in bytes.
`decoded`: buffer to hold the decoded data.
`decOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the decoded data.
`decBytes`: set to the length of the decoded data in bytes.

15.4.1.20 `MteMkeDec -startDecrypt` (ObjC)

```
-(mte_status)startDecrypt
```

Start a chunk-based decryption session. Returns the status.

15.4.1.21 `MteMkeDec -decryptChunk:encryptedBytes:decryptedBytes:` (ObjC)

```
-(const void *)decryptChunk:(const void *)encrypted
             encryptedBytes:(size_t)encryptedBytes
             decryptedBytes:(size_t *)decryptedBytes
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data of length `encryptedBytes` is used as input and `*decryptedBytes` is set to the amount of decrypted data which is returned. The amount decrypted may be less than the input size. Returns `nil` on error.

`encrypted`: the encrypted data to decrypt.
`encryptedBytes`: the length of the encrypted data to be decrypted (in bytes).
`decryptedBytes`: pointer to the length of data decrypted. This may be less than `encryptedBytes` due to cipher block boundaries and buffering for the tokenized hash.

15.4.1.22 `MteMkeDec -decryptChunk:encOff:encBytes:decrypted:decOff:` (ObjC)

```
-(size_t)decryptChunk:(const void *)encrypted
               encOff:(size_t)encOff
             encBytes:(size_t)encBytes
            decrypted:(void *)decrypted
               decOff:(size_t)decOff
```

Decrypt a chunk of data starting at `encOff` and of length `encBytes` in the `encrypted` buffer in a chunk-based decryption session. Some decrypted data is written to the `decrypted` buffer starting at `decOff`. The amount decrypted may be less than the input size and is returned. Returns `ULONG_MAX` on error.

`encrypted`: buffer containing encrypted data to decrypt.
`encOff`: offset to the start of the encrypted data.
`encBytes`: length of the encrypted data in bytes.
`decrypted`: buffer to hold decrypted data.
`decOff`: offset to start writing decrypted data.

15.4.1.23 `MteMkeDec -finishDecrypt:status:` (ObjC)

```
-(const void *)finishDecrypt:(size_t *)decryptedBytes
                      status:(mte_status *)status
```

Finish the chunk-based decryption session. Returns the final decrypted data and sets `*decryptedBytes` to the length of the final part of the decrypted data and `*status` to the status.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`decryptedBytes`: pointer to the length of data decrypted.
`status`: pointer to the status.

15.4.1.24 `MteMkeDec -getEncTs` (ObjC)

```
-(uint64_t)getEncTs
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

15.4.1.25 `MteMkeDec -getDecTs` (ObjC)

```
-(uint64_t)getDecTs
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

15.4.1.26 `MteMkeDec -getMsgSkipped` (ObjC)

```
-(uint32_t)getMsgSkipped
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

15.4.1.27 `MteMkeDec -uninstantiate` (ObjC)

```
-(mte_status)uninstantiate
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

15.4.2 Encoder

Encoder-specific functions are declared in `MteMkeEnc.h`.

15.4.2.1 `MteMkeEnc -init` (ObjC)

```
-(id)init
```

Initializer using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MTE_BASE_DEFAULT_`* constants.

15.4.2.2 `MteMkeEnc -initWithDrbg:tokBytes:verifiers:cipher:hash:` (ObjC)

```
-(id)initWithDrbg:(mte_drbgs)drbg
         tokBytes:(size_t)tokBytes
        verifiers:(mte_verifiers)verifiers
           cipher:(mte_ciphers)cipher
             hash:(mte_hashes)hash
```

Initializer taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`.

15.4.2.3 `MteMkeEnc -dealloc` (ObjC)

```
-(void)dealloc
```

Deallocate. The `-uninstantiate` message is sent.

15.4.2.4 `MteMkeEnc -instantiate:bytes:` (ObjC)

```
-(mte_status)instantiate:(const void *)ps bytes:(size_t)psBytes
```

Instantiate the encoder given the personalization string, and length of the personalization string in bytes. The `-entropyCallbackWithMinEntropy:minLength:maxLength:entropyInput:eiBytes:` and `-nonceCallbackWithMinLength:maxLength:nonce:nBytes:` messages will be sent to get the rest of the seeding material. Returns the status.

`ps`: personalization string.
`psBytes`: length of the personalization string in bytes.

15.4.2.5 `MteMkeEnc -instantiate:` (ObjC)

```
-(mte_status)instantiate:(NSString *)ps
```

Sends `-instantiate:bytes:` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

15.4.2.6 `MteMkeEnc -getReseedCounter` (ObjC)

```
-(uint64_t)getReseedCounter
```

Returns the encoder's DRBG reseed counter.

15.4.2.7 `MteMkeEnc -saveState:` (ObjC)

```
-(const void *)saveState:(size_t *)stateBytes
```

Returns the saved state in raw form and sets `stateBytes` to the length of the saved state in bytes. Returns `nil` on error.

`stateBytes`: the length of the saved state.

15.4.2.8 `MteMkeEnc -saveStateB64` (ObjC)

```
-(NSString *)saveStateB64
```

Returns the saved state in Base64-encoded form. Returns `nil` on error.

15.4.2.9 `MteMkeEnc -restoreState:` (ObjC)

```
-(mte_status)restoreState:(const void *)saved
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

15.4.2.10 `MteMkeEnc -restoreStateB64:` (ObjC)

```
-(mte_status)restoreStateB64:(NSString *)saved
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the string to restore the state from.

15.4.2.11 `MteMkeEnc -getBuffBytes:` (ObjC)

```
-(size_t)getBuffBytes:(size_t)dataBytes
```

Returns the raw encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

15.4.2.12 `MteMkeEnc -getBuffBytesB64:` (ObjC)

```
-(size_t)getBuffBytesB64:(size_t)dataBytes
```

Returns the Base64 encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

15.4.2.13 `MteMkeEnc -encode:dataBytes:encodedBytes:status:` (ObjC)

```
-(const void *)encode:(const void *)data
            dataBytes:(size_t)dataBytes
         encodedBytes:(size_t *)encodedBytes
               status:(mte_status *)status
```

Encode the given data of the given byte length in raw form. Returns the encoded version and sets `*encodedBytes` to the length of the encoded data in bytes and `*status` to the status. The encoded version is valid only if `*status == mte_status_success`.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`dataBytes`: the length of the data to be encoded (in bytes).
`encodedBytes`: pointer to the length of the encoded version.
`status`: pointer to the status.

15.4.2.14 `MteMkeEnc -encodeB64:dataBytes:status:` (ObjC)

```
-(NSString *)encodeB64:(const void *)data
             dataBytes:(size_t)dataBytes
                status:(mte_status *)status
```

Encode the given data of the given byte length in Base64-encoded form. Returns the encoded version and sets `*status` to the status. The encoded version is valid only if `*status == mte_status_success`.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.
`dataBytes`: the length of the data to be encoded (in bytes).
`status`: pointer to the status.

15.4.2.15 `MteMkeEnc -encode:encodedBytes:status:` (ObjC)

```
-(const void *)encode:(NSString *)str
         encodedBytes:(size_t *)encodedBytes
               status:(mte_status *)status
```

Sends `-encode:dataBytes:encodedBytes:status:` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.
`encodedBytes`: the length of the encoded version.
`status`: pointer to the status.

15.4.2.16 `MteMkeEnc -encodeB64:status:` (ObjC)

```
-(NSString *)encodeB64:(NSString *)str
                status:(mte_status *)status
```

Sends `-encodeB64:dataBytes:status:` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.
`status`: pointer to the status.

15.4.2.17 `MteMkeEnc -encode:dataOff:dataBytes:encoded:encOff:encBytes` (ObjC)

```
-(mte_status)encode:(const void *)data
            dataOff:(size_t)dataOff
          dataBytes:(size_t)dataBytes
            encoded:(void *)encoded
             encOff:(size_t *)encOff
           encBytes:(size_t *)encBytes
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the status. Adjusts `*encOff` to the offset of the encoded version in the `encoded` buffer. Sets `*encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes:` to determine the buffer requirement.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

15.4.2.18 `MteMkeEnc -encodeB64:dataOff:dataBytes:encoded:encOff:encBytes` (ObjC)

```
-(mte_status)encodeB64:(const void *)data
               dataOff:(size_t)dataOff
             dataBytes:(size_t)dataBytes
               encoded:(void *)encoded
                encOff:(size_t *)encOff
              encBytes:(size_t *)encBytes
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the status. Adjusts `*encOff` to the offset of the encoded version in the `encoded` buffer. Sets `*encBytes` to the encoded length in bytes. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64:` to determine the buffer requirement.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: on input, the offset to the beginning of the usable buffer area; on output, the offset of the encoded data.
`encBytes`: set to the length of the encoded data in bytes.

15.4.2.19 `MteMkeEnc -encryptFinishBytes` (ObjC)

```
-(size_t)encryptFinishBytes
```

Returns the length of the result `-finishEncrypt:status:` or `-finishEncrypt:off:bytes:` will produce. Use this if you need to know that size before you can call it.

15.4.2.20 `MteMkeEnc -startEncrypt` (ObjC)

```
-(mte_status)startEncrypt
```

Start a chunk-based encryption session. Returns the status.

15.4.2.21 `MteMkeEnc -encryptChunk:dataBytes:` (ObjC)

```
-(mte_status)encryptChunk:(void *)data dataBytes:(size_t)dataBytes
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` of length `dataBytes` is encrypted in place. The `dataBytes` argument must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: the data to encrypt.
`dataBytes`: the length of the data to be encrypted (in bytes).

15.4.2.22 `MteMkeEnc -finishEncrypt:status:` (ObjC)

```
-(const void *)finishEncrypt:(size_t *)resultBytes status:(mte_status *)status
```

Finish the chunk-based encryption session. Returns the tokenized hash and sets `*resultBytes` to the length of the tokenized hash and `*status` to the status.

The timestamp callback message will be sent to get the timestamp if the timestamp verifier is enabled.

`resultBytes`: pointer to the length of the tokenized hash.
`status`: pointer to the status.

15.4.2.23 `MteMkeEnc -uninstantiate` (ObjC)

```
-(mte_status)uninstantiate
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

16 Python Language Interface

16.1 Overview

16.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and zeroize it.

To create, use the `MteMkeEnc` initializer or `MteMkeEnc.fromdefault()` to create an encoder object and `MteMkeEnc.instantiate()` to instantiate it.

To encode, use `MteMkeEnc.encode()` or `MteMkeEnc.encode_b64()`.

To zeroize, call `MteMkeEnc.uninstantiate()`. The object delete will also do this, but Python cannot guarantee when that will happen, if at all, so for security it should be done explicitly.

16.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and zeroize it.

To create, use the `MteMkeDec` initializer or `MteMkeDec.fromdefault()` to create a decoder object and `MteMkeDec.instantiate()` to instantiate it.

To decode, use `MteMkeDec.decode()`, `MteMkeDec.decode_b64()`, `MteMkeDec.decode_str()`, or `MteMkeDec.decode_str_b64()`.

To get additional information, use `MteMkeDec.get_enc_ts()`, `MteMkeDec.get_dec_ts()`, and `MteMkeDec.get_msg_skipped()`, depending on which verifiers are enabled.

To zeroize, call `MteMkeDec.uninstantiate()`. The object delete will also do this, but Python cannot guarantee when that will happen, if at all, so for security it should be done explicitly.

16.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `MteMkeEnc.start_encrypt()` to start a chunk session, `MteMkeEnc.encrypt_chunk()` repeatedly to encrypt each chunk, then `MteMkeEnc.finish_encrypt()` to finish the session.

16.1.4 Chunk Decrypt

The chunk encrypt follows the flow of a decoder, but instead of using a decode method, use `MteMkeDec.start_decrypt()` to start a chunk session, `MteMkeDec.decrypt_chunk()` repeatedly to decrypt each chunk, then `MteMkeEnc.finish_decrypt()` to finish the session.

16.2 Files

The MTE Managed-Key Encryption Add-On uses the core source files (other than `MteEnc.py` and `MteDec.py`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteMkeDec.py	The MteMkeDec class.
MteMkeEnc.py	The MteMkeEnc class.

16.3 Types

16.3.1 Classes

16.3.1.1 `MteMkeDec`

Defines decoder-specific methods.

16.3.1.2 `MteMkeEnc`

Defines encoder-specific methods.

16.4 Functions

16.4.1 Decoder

Decoder-specific functions are declared in `MteMkeDec.py`.

16.4.1.1 `MteMkeDec` Initializer (Python)

```
def __init__(self, MteDrbgs, tok_bytes: int, verifiers: MteVerifiers, cipher:
MteCiphers, hash_: MteHashes, t_window: int, s_window: int)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash_`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.
`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

16.4.1.2 `MteMkeDec.fromdefault` (Python)

```
@classmethod
def fromdefault(cls, t_window: int = 0, s_window: int = 0)
```

Create a decoder using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise the options chosen are defined by the `MteBase_default_*` constants.

`t_window`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`s_window`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

16.4.1.3 `MteMkeDec` Delete (Python)

```
def __del__(self)
```

Delete the object. Calls `uninstantiate()`.

16.4.1.4 `MteMkeDec.instantiate` (Python)

```
def instantiate(self, ps) -> MteStatus
```

Instantiate the decoder given the personalization string. If `instanceof(ps, str)`, it is converted to an array of UTF-8 encoded bytes; otherwise it must be of type `bytes`. The `entropy_callback()` and `nonce_callback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

16.4.1.5 `MteMkeDec.get_reseed_counter` (Python)

```
def get_reseed_counter(self) -> int
```

Returns the decoder's DRBG reseed counter.

16.4.1.6 `MteMkeDec.save_state` (Python)

```
def save_state(self) -> bytearray
```

Returns the saved state in raw form. Returns an empty array on error.

16.4.1.7 `MteMkeDec.save_state_b64` (Python)

```
def save_state_b64(self) -> str
```

Returns the saved state in Base64-encoded form. Returns an empty string on error.

16.4.1.8 `MteMkeDec.restore_state` (Python)

```
def restore_state(self, saved: bytes) -> MteStatus
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

16.4.1.9 `MteMkeDec.restore_state_b64` (Python)

```
def restore_state_b64(self, saved: str) -> MteStatus
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

16.4.1.10 `MteMkeDec.decode` (Python)

```
def decode(self, encoded: bytes) -> (bytearray, MteStatus)
```

Decode the given raw encoded data. Returns the decoded data and status. The decoded version is valid only if `!status_is_error(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

16.4.1.11 `MteMkeDec.decode_b64` (Python)

```
def decode_b64(self, encoded: str) -> (bytearray, MteStatus)
```

Decode the given Base64-encoded encoded data. Returns the decoded data and status. The decoded version is valid only if `!status_is_error(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

16.4.1.12 `MteMkeDec.decode_str` (Python)

```
def decode_str(self, encoded: bytes) -> (str, MteStatus)
```

Decode the given raw encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and status. The decoded version is valid only if `!status_is_error(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

16.4.1.13 `MteMkeDec.decode_str_b64` (Python)

```
def decode_str_b64(self, encoded: str) -> (str, MteStatus)
```

Decode the given Base64-encoded encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and status. The decoded version is valid only if `!status_is_error(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

16.4.1.14 `MteMkeDec.start_decrypt` (Python)

```
def start_decrypt(self) -> MteStatus
```

Start a chunk-based decryption session. Returns the status.

16.4.1.15 `MteMkeDec.decrypt_chunk` (Python)

```
def decrypt_chunk(self, encrypted: bytes) -> bytearray
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data is used as input and some amount of decrypted data is returned. The amount decrypted may be less than the input size. Returns `None` on error.

`encrypted`: the encrypted data to decrypt.

16.4.1.16 `MteMkeDec.finish_decrypt` (Python)

```
def finish_decrypt(self) -> (bytearray, MteStatus)
```

Finish the chunk-based decryption session. Returns the final decrypted data and status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`status`: the status.

16.4.1.17 `MteMkeDec.get_enc_ts` (Python)

```
def get_enc_ts(self) -> int
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

16.4.1.18 `MteMkeDec.get_dec_ts` (Python)

```
def get_dec_ts(self) -> int
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

16.4.1.19 `MteMkeDec.get_msg_skipped` (Python)

```
def get_msg_skipped(self) -> int
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

16.4.1.20 `MteMkeDec.uninstantiate` (Python)

```
def uninstantiate(self) -> MteStatus
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

16.4.2 Encoder

Encoder-specific functions are declared in `MteMkeEnc.py`.

16.4.2.1 `MteMkeEnc` Initializer (Python)

```
def __init__(self, drbg: mte_drbgs, tok_bytes: int, verifiers: MteVerifiers,
cipher: mte_ciphers, hash_: mte_hashes)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm.

Buildtime options: the `drbg`, `tok_bytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tok_bytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash_`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.

16.4.2.2 `MteMkeEnc.fromdefault` (Python)

```
@classmethod
def fromdefault(cls)
```

Create an encoder using default options. If the library has buildtime options, they are used; otherwise the options chosen are defined by the `MteBase_default_`* constants.

16.4.2.3 `MteMkeEnc` Delete (Python)

```
def __del__(self)
```

Delete the object. Calls `uninstantiate()`.

16.4.2.4 `MteMkeEnc.instantiate` (Python)

```
def instantiate(self, ps) -> MteStatus
```

Instantiate the decoder given the personalization string. If `instanceof(ps, str)`, it is converted to an array of UTF-8 encoded bytes; otherwise it must be of type `bytes`. The `entropy_callback()` and `nonce_callback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

16.4.2.5 `MteMkeEnc.get_reseed_counter` (Python)

```
def get_reseed_counter(self) -> int
```

Returns the encoder's DRBG reseed counter.

16.4.2.6 `MteMkeEnc.save_state` (Python)

```
def save_state(self) -> bytearray
```

*Returns the saved state in raw form and sets `stateBytes` to the length of the saved state in bytes. Returns an empty array on error.

16.4.2.7 `MteMkeEnc.save_state_b64` (Python)

```
def save_state_b64(self) -> str
```

Returns the saved state in Base64-encoded form. Returns an empty string on error.

16.4.2.8 `MteMkeEnc.restore_state` (Python)

```
def restore_state(self, saved: bytes) -> MteStatus
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

16.4.2.9 `MteMkeEnc.restore_state_b64` (Python)

```
def restore_state_b64(self, saved: str) -> MteStatus
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

16.4.2.10 `MteMkeEnc.encode` (Python)

```
def encode(self, data) -> (bytearray, MteStatus)
```

Encode the given data in raw form. If `instanceof(data, str)`, it is converted to an array of UTF-8 encoded bytes; otherwise it must be of type `bytes`. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

16.4.2.11 `MteMkeEnc.encode_b64` (Python)

```
def encode_b64(self, data) -> (str, MteStatus)
```

Encode the given data in Base64-encoded form. If `instanceof(data, str)`, it is converted to an array of UTF-8 encoded bytes; otherwise it must be of type `bytes`. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

16.4.2.12 `MteMkeEnc.encrypt_finish_bytes` (Python)

```
def encrypt_finish_bytes(self) -> int
```

Returns the length of the result `finish_encrypt()` will produce. Use this if you need to know that size before you can call it.

16.4.2.13 `MteMkeEnc.start_encrypt` (Python)

```
def start_encrypt(self) -> MteStatus
```

Start a chunk-based encryption session. Returns the status.

16.4.2.14 `MteMkeEnc.encrypt_chunk` (Python)

```
def encrypt_chunk(self, data: bytearray) -> MteStatus
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` is encrypted in place. The `len(data)` must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: the data to encrypt.

16.4.2.15 `MteMkeEnc.finish_encrypt` (Python)

```
def finish_encrypt(self) -> (bytearray, MteStatus)
```

Finish the chunk-based encryption session. Returns the tokenized hash and status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

16.4.2.16 `MteMkeEnc.uninstantiate` (Python)

```
def uninstantiate(self) -> MteStatus
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

17 Swift Language Interface

17.1 Overview

17.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, and destroy it.

To create, use an `MteMkeEnc` initializer to create an encoder object and one of the `MteMkeEnc.instantiate()` overloads to instantiate it.

To encode, use any of the `MteMkeEnc.encode()` or `MteMkeEnc.encodeB64()` overloads.

To destroy, cause the `MteMkeEnc` deinitializer to be invoked (e.g., by removing the last strong reference to the object).

17.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, and destroy it.

To create, use an `MteMkeDec` initializer to create a decoder object and one of the `MteMkeDec.instantiate()` overloads to instantiate it.

To decode, use `MteMkeDec.decode()`, `MteMkeDec.decodeB64()`, `MteMkeDec.decodeStr()`, or `MteMkeDec.decodeStrB64()`.

To get additional information, use `MteMkeDec.getEncTs()`, `MteMkeDec.getDecTs()`, and `MteMkeDec.getMsgSkipped()`, depending on which verifiers are enabled.

To destroy, cause the `MteMkeDec` deinitializer to be invoked (e.g., by removing the last strong reference to the object).

17.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `MteMkeEnc.startEncrypt()` to start a chunk session, `MteMkeEnc.encryptChunk()` repeatedly to encrypt each chunk, then `MteMkeEnc.finishEncrypt()` to finish the session.

17.1.4 Chunk Decrypt

The chunk encrypt follows the flow of a decoder, but instead of using a decode method, use `MteMkeDec.startDecrypt()` to start a chunk session, `MteMkeDec.decryptChunk()` repeatedly to decrypt each chunk, then `MteMkeEnc.finishDecrypt()` to finish the session.

17.2 Files

The MTE Managed-Key Encryption Add-On uses the core source files (other than `MteEnc.swift` and `MteDec.swift`) documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteMkeDec.swift	The MteMkeDec class.
MteMkeEnc.swift	The MteMkeEnc class.

17.3 Types

17.3.1 Classes

17.3.1.1 `MteMkeDec`

Defines decoder-specific methods.

17.3.1.2 `MteMkeEnc`

Defines encoder-specific methods.

17.4 Functions

17.4.1 Decoder

Decoder-specific functions are declared in `MteMkeDec.swift`.

17.4.1.1 `MteMkeDec` Initializer (default options) (Swift)

```
public convenience init(_ tWindow: UInt64 = 0, _ sWindow: Int = 0) throws
```

Initializer using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants.

`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

17.4.1.2 `MteMkeDec` Initializer (runtime options) (Swift)

```
public init(_ drbg: mte_drbgs, _ tokBytes: Int, _ verifiers: mte_verifiers, _
cipher: mte_ciphers, _ hash: mte_hashes, _ tWindow: UInt64, _ sWindow: Int) throws
```

Initializer taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window.

Buildtime options: the `drbg`, `tokBytes`, `byteValMin`, `byteValCount`, and `verifiers` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes_none`.
`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

17.4.1.3 `MteMkeDec` Deinitializer (Swift)

```
deinit
```

Deallocate. The `uninstantiate()` method is called.

17.4.1.4 `MteMkeDec.instantiate([UInt8])` (Swift)

```
public func instantiate(_ ps: [UInt8]) -> mte_status
```

Instantiate the decoder given the personalization string. The `entropyCallback()` and `nonceCallback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

17.4.1.5 `MteMkeDec.instantiate(String)` (Swift)

```
public func instantiate(_ ps: String) -> mte_status
```

Calls `instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

17.4.1.6 `MteMkeDec.getReseedCounter` (Swift)

```
public func getReseedCounter() -> UInt64
```

Returns the decoder's DRBG reseed counter.

17.4.1.7 `MteMkeDec.saveState` (Swift)

```
public func saveState() -> [UInt8]?
```

Returns the saved state in raw form. Returns `nil` on error.

17.4.1.8 `MteMkeDec.saveStateB64` (Swift)

```
public func saveStateB64() -> String?
```

Returns the saved state in Base64-encoded form. Returns `nil` on error.

17.4.1.9 `MteMkeDec.restoreState` (Swift)

```
public func restoreState(_ saved: [UInt8]) -> mte_status
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

17.4.1.10 `MteMkeDec.restoreStateB64` (Swift)

```
public func restoreStateB64(_ saved: String) -> mte_status
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

17.4.1.11 `MteMkeDec.getBuffBytes` (Swift)

```
public func getBuffBytes(_ encoded: [UInt8], _ encOff: Int, _ encBytes: Int) -> Int
```

Returns the decode buffer size in bytes given the raw encoded message size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

17.4.1.12 `MteMkeDec.getBuffBytesB64` (Swift)

```
public func getBuffBytesB64(_ encoded: [UInt8], _ encOff: Int, _ encBytes: Int) ->
Int
```

Returns the decode buffer size in bytes given the Base64-encoded message size in bytes.

`encoded`: buffer containing the encoded message.
`encOff`: offset to the start of the encoded message.
`encBytes`: the encoded message size in bytes.

17.4.1.13 `MteMkeDec.decode` (Swift)

```
public func decode(_ encoded: [UInt8]) -> (decoded: ArraySlice<UInt8>, status:
mte_status)
```

Decode the given raw encoded data. Returns the decoded data and `status`. The decoded version is valid only if `!statusIsError(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

17.4.1.14 `MteMkeDec.decodeB64` (Swift)

```
public func decodeB64(_ encoded: String) -> (decoded: ArraySlice<UInt8>, status:
mte_status)
```

Decode the given Base64-encoded encoded data. Returns the decoded data and `status`. The decoded version is valid only if `!statusIsError(status)`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

17.4.1.15 `MteMkeDec.decodeStr` (Swift)

```
public func decodeStr(_ encoded: [UInt8]) -> (str: String, status: mte_status)
```

Decode the given raw encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and the status. The decoded version is valid only if `!statusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

17.4.1.16 `MteMkeDec.decodeStrB64` (Swift)

```
public func decodeStrB64(_ encoded: String) -> (str: String, status: mte_status)
```

Decode the given Base64-encoded encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and the status. The decoded version is valid only if `!statusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

17.4.1.17 `MteMkeDec.startDecrypt` (Swift)

```
public func startDecrypt() -> mte_status
```

Start a chunk-based decryption session. Returns the status.

17.4.1.18 `MteMkeDec.decryptChunk` (Swift)

```
public func decryptChunk(_ encrypted: [UInt8]) -> (data: ArraySlice<UInt8>, status:
mte_status)
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data is used as input and some amount of decrypted data is returned along with the status. The amount decrypted may be less than the input size.

`encrypted`: the encrypted data to decrypt.

17.4.1.19 `MteMkeDec.decryptChunk` (external buffers) (Swift)

```
public func decryptChunk(_ encrypted: [UInt8], _ encOff: Int, _ encBytes: Int, _
decrypted: inout [UInt8], _ decOff: Int) -> Int
```

Decrypt a chunk of data starting at `encOff` and of length `encBytes` in the `encrypted` buffer in a chunk-based decryption session. Some decrypted data is written to the `decrypted` buffer starting at `decOff`. The amount decrypted may be less than the input size and is returned. Returns `-1` on error.

`encrypted`: the encrypted data to decrypt.
`encOff`: offset to the start of the encrypted data.
`encBytes`: length of the encrypted data in bytes.
`decrypted`: buffer to hold decrypted data.
`decOff`: offset to start writing decrypted data.

17.4.1.20 `MteMkeDec.finishDecrypt` (Swift)

```
public func finishDecrypt() -> (data: ArraySlice<UInt8>, status: mte_status)
```

Finish the chunk-based decryption session. Returns the final decrypted data and status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`status`: the status.

17.4.1.21 `MteMkeDec.getEncTs` (Swift)

```
public func getEncTs() -> UInt64
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

17.4.1.22 `MteMkeDec.getDecTs` (Swift)

```
public func getDecTs() -> UInt64
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `0` is returned.

17.4.1.23 `MteMkeDec.getMsgSkipped` (Swift)

```
public func getMsgSkipped() -> UInt32
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

17.4.1.24 `MteMkeDec.uninstantiate` (Swift)

```
public func uninstantiate() -> mte_status
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

17.4.2 Encoder

Encoder-specific functions are declared in `MteMkeEnc.swift`.

17.4.2.1 `MteMkeEnc` Initializer (default options) (Swift)

```
public override convenience init() throws
```

Initializer using default options. If the library has buildtime options, they are used; otherwise, the options chosen are defined by the `MteBase.ourDefault`* constants.

17.4.2.2 `MteMkeEnc` Initializer (runtime options) (Swift)

```
public init(_ drbg: mte_drbgs, _ tokBytes: Int, _ verifiers: mte_verifiers, _
cipher: mte_ciphers, _ hash: mte_hashes) throws
```

Initializer taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.

17.4.2.3 `MteMkeEnc` Deinitializer (Swift)

```
deinit
```

Destructor. The `uninstantiate()` method is called.

17.4.2.4 `MteMkeEnc.instantiate(_[UInt8])` (Swift)

```
public func instantiate(_ ps: [UInt8]) -> mte_status
```

Instantiate the encoder given the personalization string. The `entropyCallback()` and `nonceCallback()` functions will be called to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

17.4.2.5 `MteMkeEnc.instantiate(String)` (Swift)

```
public func instantiate(_ ps: String) -> mte_status
```

Calls `instantiate()` with the string as an array of UTF-8 encoded bytes.

`ps`: personalization string.

17.4.2.6 `MteMkeEnc.getReseedCounter` (Swift)

```
public func getReseedCounter() -> UInt64
```

Returns the encoder's DRBG reseed counter.

17.4.2.7 `MteMkeEnc.saveState` (Swift)

```
public func saveState() -> [UInt8]?
```

Returns the saved state in raw form. Returns `nil` on error.

17.4.2.8 `MteMkeEnc.saveStateB64` (Swift)

```
public func saveStateB64() -> String?
```

Returns the saved state in Base64-encoded form. Returns `nil` on error.

17.4.2.9 `MteMkeEnc.restoreState` (Swift)

```
public func restoreState(_ saved: [UInt8]) -> mte_status
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

17.4.2.10 `MteMkeEnc.restoreStateB64` (Swift)

```
public func restoreStateB64(_ saved: String) -> mte_status
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

17.4.2.11 `MteMkeEnc.getBuffBytes` (Swift)

```
public func getBuffBytes(_ dataBytes: Int) -> Int
```

Returns the raw encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

17.4.2.12 `MteMkeEnc.getBuffBytesB64` (Swift)

```
public func getBuffBytesB64(_ dataBytes: Int) -> Int
```

Returns the Base64 encode buffer size in bytes given the data size in bytes.

`dataBytes`: the data size in bytes.

17.4.2.13 `MteMkeEnc.encode` (`[UInt8]`) (Swift)

```
public func encode(_ data: [UInt8]) -> (encoded: ArraySlice<UInt8>, status:
mte_status)
```

Encode the given data in raw form. Returns the encoded version and `status`. The encoded version is valid only if `status == mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

17.4.2.14 `MteMkeEnc.encodeB64` (`[UInt8]`) (Swift)

```
public func encodeB64(_ data: [UInt8]) -> (encoded: String, status: mte_status)
```

Encode the given data in Base64-encoded form. Returns the encoded version and `status`. The encoded version is valid only if `status == mte_status_success`.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: the data to encode.

17.4.2.15 `MteMkeEnc.encode` (`String`) (Swift)

```
public func encode(_ str: String) -> (encoded: ArraySlice<UInt8>, status:
mte_status)
```

Calls `encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

17.4.2.16 `MteMkeEnc.encodeB64` (`String`) (Swift)

```
public func encodeB64(_ str: String) -> (encoded: String, status: mte_status)
```

Calls `encodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

17.4.2.17 `MteMkeEnc.encode` (external buffers) (Swift)

```
public func encode(_ data: [UInt8], _ dataOff: Int, _ dataBytes: Int, _ encoded:
inout [UInt8], _ encOff: Int) -> (encOff: Int, encBytes: Int, status: mte_status)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in raw form. Returns the offset of the encoded version, length of the encoded version, and status. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytes()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: the offset to the beginning of the usable buffer area.

17.4.2.18 `MteMkeEnc.encodeB64` (external buffers) (Swift)

```
public func encodeB64(_ data: [UInt8], _ dataOff: Int, _ dataBytes: Int, _ encoded:
inout [UInt8], _ encOff: Int) -> (encOff: Int, encBytes: Int, status: mte_status)
```

Encode the given data of the given length at the given offset to the given buffer at the given offset in Base64-encoded form. Returns the offset of the encoded version, length of the encoded version, and status. The `encoded` buffer must have sufficient length remaining after the offset. Use `getBuffBytesB64()` to determine the buffer requirement.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

`data`: buffer containing the data.
`dataOff`: offset to the start of the data.
`dataBytes`: length of the data in bytes.
`encoded`: buffer to hold the encoded data.
`encOff`: the offset to the beginning of the usable buffer area.

17.4.2.19 `MteMkeEnc.encryptFinishBytes` (Swift)

```
public func encryptFinishBytes() -> Int
```

Returns the length of the result `finishEncrypt()` will produce. Use this if you need to know that size before you can call it.

17.4.2.20 `MteMkeEnc.startEncrypt` (Swift)

```
public func startEncrypt() -> mte_status
```

Start a chunk-based encryption session. Returns the status.

17.4.2.21 `MteMkeEnc.encryptChunk` (Swift)

```
public func encryptChunk(_ data: inout [UInt8]) -> mte_status
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` is encrypted in place. The `data.count` must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: the data to encrypt.

17.4.2.22 `MteMkeEnc.encryptChunk` (partial) (Swift)

```
public func encryptChunk(_ data: inout [UInt8], _ off: Int, _ bytes: Int) ->
mte_status
```

Encrypt a chunk of data in a chunk-based encryption session. The `data` is encrypted in place. The `data.count` must be a multiple of the cipher block size in the chosen mode of operation. Returns the status.

`data`: the data to encrypt.
`off`: offset to the start of the data to encrypt.
`bytes`: length of the data to encrypt in bytes.

17.4.2.23 `MteMkeEnc.finishEncrypt` (Swift)

```
public func finishEncrypt() -> (encoded: ArraySlice<UInt8>, status: mte_status)
```

Finish the chunk-based encryption session. Returns the tokenized hash and status.

The timestamp callback will be called to get the timestamp if the timestamp verifier is enabled.

17.4.2.24 `MteMkeEnc.uninstantiate` (Swift)

```swift
public func uninstantiate() -> mte_status
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

18 WASM Language Interface

18.1 Overview

18.1.1 Encoder

The basic flow for an encoder is to create it, use it to encode, zeroize, and destruct it.

To create, use the `MteMkeEnc` constructor or `MteMkeEnc.fromdefault()` to create an encoder object and `MteMkeEnc.instantiate()` to instantiate it.

To encode, use `MteMkeEnc.encode()`, `MteMkeEnc.encodeB64()`, `MteMkeEnc.encodeStr()`, or `MteMkeEnc.encodeStrB64()`.

To zeroize, call `MteMkeEnc.uninstantiate()`.

To destruct, call `MteMkeEnc.destruct()` to free WASM resources.

18.1.2 Decoder

The basic flow for a decoder is to create it, use it to decode, get additional information about decoding, zeroize, and destruct it.

To create, use the `MteMkeDec` constructor or `MteMkeDec.fromdefault()` to create a decoder object and `MteMkeDec.instantiate()` to instantiate it.

To decode, use `MteMkeDec.decode()`, `MteMkeDec.decodeB64()`, `MteMkeDec.decodeStr()`, or `MteMkeDec.decodeStrB64()`.

To get additional information, use `MteMkeDec.getEncTs()`, `MteMkeDec.getDecTs()`, and `MteMkeDec.getMsgSkipped()`, depending on which verifiers are enabled.

To zeroize, call `MteMkeDec.uninstantiate()`.

To destruct, call `MteMkeDec.destruct()` to free WASM resources.

18.1.3 Chunk Encrypt

The chunk encrypt follows the flow of an encoder, but instead of using an encode method, use `MteMkeEnc.startEncrypt()` to start a chunk session, `MteMkeEnc.encryptChunk()` repeatedly to encrypt each chunk, then `MteMkeEnc.finishEncrypt()` to finish the session.

18.1.4 Chunk Decrypt

The chunk encrypt follows the flow of a decoder, but instead of using a decode method, use `MteMkeDec.startDecrypt()` to start a chunk session, `MteMkeDec.decryptChunk()` repeatedly to decrypt each chunk, then `MteMkeEnc.finishDecrypt()` to finish the session.

18.2 Files

The MTE Managed-Key Encryption Add-On uses the core source files documented in the *MTE Core Developer's Guide*.

18.3 Types

18.3.1 Classes

18.3.1.1 `MteMkeDec`

Defines decoder-specific methods.

18.3.1.2 `MteMkeEnc`

Defines encoder-specific methods.

18.4 Functions

18.4.1 Decoder

18.4.1.1 `MteMkeDec` Constructor (WASM)

```
constructor(wasm: MteWasm, drbg: mte_drbgs | string, tokBytes: number, verifiers:
MteVerifiers | string, cipher: mte_ciphers | string, hash: mte_hashes | string,
tWindow: string, sWindow: number)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, hash algorithm, timestamp window, and sequence window.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`wasm`: the instantiated WASM.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.

`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

18.4.1.2 `MteMkeDec.fromdefault` (WASM)

```
public static fromdefault(wasm: MteWasm, tWindow = "0", sWindow = 0): MteMkeDec
```

Constructor using default options, timestamp window, and sequence window. If the library has buildtime options, they are used; otherwise the options chosen are defined by the `MteBase.default`* constants.

`tWindow`: the timestamp window. If the timestamp verifier is not enabled, this argument is ignored.
`sWindow`: the sequence window. If the sequence verifier is not enabled, this argument is ignored.

18.4.1.3 `MteMkeDec.destruct` (WASM)

```
destruct = (): void
```

Destruct. The WASM memory will be freed and the object can no longer be used.

18.4.1.4 `MteMkeDec.instantiate` (WASM)

```
instantiate = (ps: string): MteStatus
```

Instantiate the decoder given the personalization string. The personalization string is converted to an array of UTF-8 encoded bytes for use. The entropy and nonce will be used, depending how they were set, to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

18.4.1.5 `MteMkeDec.getReseedCounter` (WASM)

```
getReseedCounter = (): string
```

Returns the decoder's DRBG reseed counter. The string will contain a decimal number which could exceed the range of Javascript's `Number` type.

18.4.1.6 `MteMkeDec.saveState` (WASM)

```
saveState = (): Uint8Array | null
```

Returns the saved state in raw form. Returns `null` on error.

18.4.1.7 `MteMkeDec.saveStateB64` (WASM)

```
saveStateB64 = (): string | null
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

18.4.1.8 `MteMkeDec.restoreState` (WASM)

```
restoreState = (saved: Uint8Array): MteStatus
```

Restores the decoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

18.4.1.9 `MteMkeDec.restoreStateB64` (WASM)

```
restoreStateB64 = (saved: string): MteStatus
```

Restores the decoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

18.4.1.10 `MteMkeDec.decode` (WASM)

```
decode = (encoded: Uint8Array): MteArrStatus
```

Decode the given raw encoded data. Returns the decoded data and status. The decoded version is valid only if `!statusIsError(status)`.

The timestamp callback will be called if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

18.4.1.11 `MteMkeDec.decodeB64` (WASM)

```
decodeB64 = (encoded: string): MteArrStatus
```

Decode the given Base64-encoded encoded data. Returns the decoded data and status. The decoded version is valid only if `!statusIsError(status)`.

The timestamp callback will be called if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

18.4.1.12 `MteMkeDec.decodeStr` (WASM)

```
decodeStr = (encoded: Uint8Array): MteStrStatus
```

Decode the given raw encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and status. The decoded version is valid only if `!statusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

18.4.1.13 `MteMkeDec.decodeStrB64` (WASM)

```
decodeStrB64 = (encoded: string): MteStrStatus
```

Decode the given Base64-encoded encoded data. Returns the decoded data up to the length of decoding or first null byte, whichever comes first, as a string and status. The decoded version is valid only if `!statusIsError(status)`. For interoperability, it is assumed the string is UTF-8 encoded.

The timestamp callback will be called if the timestamp verifier is enabled.

`encoded`: the encoded data to decode.

18.4.1.14 `MteMkeDec.startDecrypt` (WASM)

```
startDecrypt = (): MteStatus
```

Start a chunk-based decryption session. Returns the status.

18.4.1.15 `MteMkeDec.decryptChunk` (WASM)

```
decryptChunk = (encrypted: Uint8Array): Uint8Array | null
```

Decrypt a chunk of data in a chunk-based decryption session. The `encrypted` data is used as input and some amount of decrypted data is returned. The amount decrypted may be less than the input size. Returns `null` on error.

`encrypted`: the encrypted data to decrypt.

18.4.1.16 `MteMkeDec.finishDecrypt` (WASM)

```
finishDecrypt = (): MteArrStatus
```

Finish the chunk-based decryption session. Returns the final decrypted data and status.

The timestamp callback will be called if the timestamp verifier is enabled.

18.4.1.17 `MteMkeDec.getEncTs` (WASM)

```
getEncTs = (): string
```

Returns the encode timestamp from the most recent decode. If the timestamp verifier is not enabled, `"0"` is returned. The string will contain a decimal number which could exceed the range of Javascript's `Number` type.

18.4.1.18 `MteMkeDec.getDecTs` (WASM)

```
getDecTs = (): string
```

Returns the decode timestamp from the most recent decode. If the timestamp verifier is not enabled, `"0"` is returned. The string will contain a decimal number which could exceed the range of Javascript's `Number` type.

18.4.1.19 `MteMkeDec.getMsgSkipped` (WASM)

```
getMsgSkipped = (): number
```

If the sequence window is non-negative, returns the messages skipped from the most recent decode; otherwise it returns the number of messages ahead of the base sequence from the most recent decode. If the sequence verifier is not enabled, `0` is returned.

18.4.1.20 `MteMkeDec.uninstantiate` (WASM)

```
uninstantiate = (): MteStatus
```

Uninstantiate the decoder. The decoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

18.4.2 Encoder

18.4.2.1 `MteMkeEnc` Constructor (WASM)

```
constructor(wasm: MteWasm, drbg: mte_drbgs | string, tokBytes: number, verifiers:
MteVerifiers | string, cipher: mte_ciphers | string, hash: mte_hashes | string)
```

Constructor taking the DRBG algorithm, token size in bytes, verifiers algorithm, cipher algorithm, and hash algorithm.

Buildtime options: the `drbg`, `tokBytes`, `verifiers`, `cipher`, and `hash` arguments must match the buildtime-chosen options.

`wasm`: the instantiated WASM.
`drbg`: the MTE-provided DRBG. Do not pass `mte_drbgs.mte_drbgs_none`.
`tokBytes`: the token size in bytes.
`verifiers`: the verifiers.
`cipher`: the MTE-provided cipher. Do not pass `mte_ciphers.mte_ciphers_none`.
`hash`: the MTE-provided hash. Do not pass `mte_hashes.mte_hashes_none`.

18.4.2.2 `MteMkeEnc.fromdefault` (WASM)

```
public static fromdefault(wasm: MteWasm): MteMkeEnc
```

Constructor using default options. If the library has buildtime options, they are used; otherwise the options chosen are defined by the `MteBase.default`* constants.

`wasm`: the instantiated WASM.

18.4.2.3 `MteMkeEnc.destruct` (WASM)

```
destruct = (): void
```

Destruct. The WASM memory will be freed and the object can no longer be used.

18.4.2.4 `MteMkeEnc.instantiate` (WASM)

```
instantiate = (ps: string): MteStatus
```

Instantiate the decoder given the personalization string. The personalization string is converted to an array of UTF-8 encoded bytes for use. The entropy and nonce will be used, depending how they were set, to get the rest of the seeding material. Returns the status.

`ps`: personalization string.

18.4.2.5 `MteMkeEnc.getReseedCounter` (WASM)

```
getReseedCounter = (): string
```

Returns the encoder's DRBG reseed counter. The string will contain a decimal number which could exceed the range of Javascript's `Number` type.

18.4.2.6 `MteMkeEnc.saveState` (WASM)

```
saveState = (): Uint8Array | null
```

Returns the saved state in raw form. Returns `null` on error.

18.4.2.7 `MteMkeEnc.saveStateB64` (WASM)

```
saveStateB64 = (): string | null
```

Returns the saved state in Base64-encoded form. Returns `null` on error.

18.4.2.8 `MteMkeEnc.restoreState` (WASM)

```
restoreState = (saved: Uint8Array): MteStatus
```

Restores the encoder state from the given buffer in raw form. Returns the status.

`saved`: the buffer to restore the state from.

18.4.2.9 `MteMkeEnc.restoreStateB64` (WASM)

```
restoreStateB64 = (saved: string): MteStatus
```

Restores the encoder state from the given buffer in Base64-encoded form. Returns the status.

`saved`: the buffer to restore the state from.

18.4.2.10 `MteMkeEnc.encode` (WASM)

```
encode = (data: Uint8Array): MteArrStatus
```

Encode the given data in raw form. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

The timestamp callback will be called if the timestamp verifier is enabled.

`data`: the data to encode.

18.4.2.11 `MteMkeEnc.encodeB64` (WASM)

```
encodeB64 = (data: Uint8Array): MteStrStatus
```

Encode the given data in Base64-encoded form. Returns the encoded version and status. The encoded version is valid only if `status == MteStatus.mte_status_success`.

The timestamp callback will be called if the timestamp verifier is enabled.

`data`: the data to encode.

18.4.2.12 `MteMkeEnc.encodeStr` (WASM)

```
encodeStr = (str: string): MteArrStatus
```

Calls `encode()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

18.4.2.13 `MteMkeEnc.encodeStrB64` (WASM)

```
encodeStrB64 = (str: string): MteStrStatus
```

Calls `encodeB64()` with the string as an array of UTF-8 encoded bytes.

`str`: the string to encode.

18.4.2.14 `MteMkeEnc.encryptFinishBytes` (WASM)

```
encryptFinishBytes = (): number
```

Returns the length of the result `finishEncrypt()` will produce. Use this if you need to know that size before you can call it.

18.4.2.15 `MteMkeEnc.startEncrypt` (WASM)

```
startEncrypt = (): MteStatus
```

Start a chunk-based encryption session. Returns the status.

18.4.2.16 `MteMkeEnc.encryptChunk` (WASM)

```
encryptChunk = (data: Uint8Array): Uint8Array | null
```

Encrypt a chunk of data in a chunk-based encryption session. Returns the encrypted data. The `data.length` must be a multiple of the cipher block size in the chosen mode of operation. Returns `null` on error.

`data`: the data to encrypt.

18.4.2.17 `MteMkeEnc.finishEncrypt` (WASM)

```
finishEncrypt = (): MteArrStatus
```

Finish the chunk-based encryption session. Returns the tokenized hash and status.

The timestamp callback will be called if the timestamp verifier is enabled.

18.4.2.18 `MteMkeEnc.uninstantiate` (WASM)

```
uninstantiate = (): MteStatus
```

Uninstantiate the encoder. The encoder's DRBG state is zeroized. Returns the status. The object must not be used until instantiated again.

19 Contact Eclypses

For more information, please contact:

info@eclypses.com

www.eclypses.com

+1.719.323.6680

All trademarks of Eclypses Inc. may not be used without Eclypses Inc.'s prior written consent. No license for any use thereof has been granted without express written consent. Any unauthorized use thereof may violate copyright laws, trademark laws, privacy and publicity laws and communications regulations and statutes. The names, images and likeness of the Eclypses logo, along with all representations thereof, are valuable intellectual property assets of Eclypses, Inc. Accordingly, no party or parties, without the prior written consent of Eclypses, Inc., (which may be withheld in Eclypses' sole discretion), use or permit the use of any of the Eclypses trademarked names or logos of Eclypses, Inc. for any purpose other than as part of the address for the Premises, or use or permit the use of, for any purpose whatsoever, any image or rendering of, or any design based on, the exterior appearance or profile of the Eclypses trademarks and or logo(s).